

Memoria del Proyecto Intermodular

Desarrollo de Interfaces · Programación de Servicios y Procesos
Sistemas de Gestión Empresarial

CICLO FORMATIVO DE GRADO SUPERIOR EN
DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Omar El Yamani Serroukh, Álvaro Baz Rodríguez e Iván Bermejo

CURSO 2026-2027

TÍTULO: NOVA Sneakers · Memoria del Proyecto Intermodular

AUTORES: Omar El Yamani Serroukh, Álvaro Baz Rodríguez e Iván Bermejo

TITULACIÓN: Técnico Superior en Desarrollo de Aplicaciones Multiplataforma (segundo curso)

ASIGNATURAS: Desarrollo de Interfaces (0488); Programación de Servicios y Procesos (0490); Sistemas de Gestión Empresarial (0491)

VERSIÓN DOCUMENTADA: Repositorio omar1676/NOVA, rama main, commit 372f811

FECHA: 13 de noviembre de 2026

RESUMEN

Esta memoria presenta el trabajo de las asignaturas de Desarrollo de Interfaces (0488), Programación de Servicios y Procesos (0490) y Sistemas de Gestión Empresarial (0491) dentro del Proyecto Intermodular de NOVA Sneakers, una fábrica de zapatillas ficticia que vende un catálogo propio y fabrica pedidos personalizados (Custom) para clubes y equipos. El proyecto parte del enunciado «Sneaker Factory» del segundo curso del ciclo de Desarrollo de Aplicaciones Multiplataforma y lo desarrolla con una marca propia.

En Desarrollo de Interfaces se ha construido una web de clientes sin librerías externas, con HTML semántico, CSS organizado según la metodología BEM y JavaScript, que reúne un catálogo de cien modelos, la ficha de producto, un probador virtual, un formulario de solicitud Custom y el contacto, y que se adapta a escritorio y móvil. El equipo la amplió con una tienda online con pago simulado, un panel de administración y un portal del trabajador conectados a PostgreSQL mediante un servidor Node/Express. Las pruebas automáticas en Microsoft Edge y las auditorías con axe y Lighthouse no detectan incumplimientos de WCAG 2.1 AA; la prueba de usabilidad con personas está pendiente.

En Programación de Servicios y Procesos se ha desarrollado una aplicación Java de consola en la que un proceso coordinador lanza tres procesos hijos con `ProcessBuilder` y se comunica con ellos por la entrada y la salida estándar para validar un pedido Custom, agrupar sus pares por talla y generar una ficha auxiliar para el taller. La aplicación supera sus 27 pruebas y las 8 comprobaciones del analizador de tallas.

En Sistemas de Gestión Empresarial se ha implantado Odoo 19 Community en una máquina Ubuntu y se ha recorrido el pedido S00005, de 40 pares, desde la venta hasta la fabricación, la entrega y la factura. Esta parte se acredita con la documentación y las capturas originales de su autor. En el entorno de su autor funcionó también una conexión entre el servidor del proyecto y Odoo mediante su API, que no está activada en el servidor principal; la memoria explica sus límites.

Palabras clave: NOVA Sneakers, HTML5, CSS3, BEM, JavaScript, diseño adaptable, accesibilidad, Java, ProcessBuilder, comunicación entre procesos, Odoo, ERP, fabricación.

ÍNDICE

Capítulo 1. Resumen del proyecto 1
1.1 Contexto y justificación 1
1.2 Planteamiento del problema 1
1.3 Objetivos del proyecto 1
1.4 Resultados obtenidos 2
1.5 Alcance de la memoria y tratamiento de las evidencias 2

Capítulo 2. Antecedentes del proyecto y estado del arte 3
2.1 Estado del arte 3
2.2 El caso de empresa y el pedido Custom 4
2.3 Relación entre las asignaturas 4

Capítulo 3. Desarrollo del proyecto 6
3.1 Desarrollo de Interfaces (0488) 6
3.1.1 Planificación y metodología 6
3.1.2 Descripción de la solución 7
3.1.3 Requisitos del proyecto 23
3.1.4 Resultados del proyecto 23
3.2 Programación de Servicios y Procesos (0490) 26
3.2.1 Planificación y metodología 26
3.2.2 Descripción de la solución 27
3.2.3 Requisitos del proyecto 33
3.2.4 Resultados del proyecto 33
3.3 Sistemas de Gestión Empresarial (0491) 39
3.3.1 Planificación y metodología 39
3.3.2 Descripción de la solución 40
3.3.3 Requisitos del proyecto 45
3.3.4 Resultados del proyecto 46

Capítulo 4. Implementación e integración del proyecto 54
4.1 Qué viaja entre las partes 54
4.2 Modelo de datos común del pedido 54
4.3 Orden de la demostración 55

Capítulo 5. Conclusiones 56
5.1 Conclusiones del trabajo 56
5.2 Reparto del trabajo 56
5.3 Aprendizajes del proyecto 57
5.4 Líneas de mejora 57

Capítulo 6. Anexos 58
6.1 Repositorio y versión documentada 58
6.2 Cómo reproducir las demostraciones 58
6.3 Relación de fuentes 59
6.4 Relación de figuras y su origen 59

ÍNDICE DE FIGURAS

Figura 1. Relación entre las tres asignaturas en la versión entregada (esquema de elaboración propia a partir del código de 372f811 y del documento de Odoo). 5
Figura 2. Portada de la web de clientes en escritorio (1440 px): cabecera con el menú, barra de cuenta y carrito y el escaparate con los filtros por colección. 8
Figura 3. Portada en un móvil de 390 px: menú de tres enlaces y escaparate a dos columnas. 10
Figura 4. Catálogo en móvil (480 px) con el precio y la disponibilidad de cada modelo. 10
Figura 5. «Mis pedidos» en móvil con un pedido «En reparto». 11
Figura 6. Dashboard del panel en móvil: menú horizontal y tarjetas en dos columnas. 11
Figura 7. Catálogo completo: tarjetas de la colección Hombre con el nombre comercial, el precio y la disponibilidad, sobre el color de su colección. 12
Figura 8. Ficha del modelo NOVA Street Vortex: precio, color, tallas de la 38 a la 46, cantidad y acceso al carrito y al probador. 12
Figura 9. Probador con el modelo NOVA Casual Marea puesto, en el escenario Madrid. 13
Figura 10. Figura de complexión media con calzado genérico y la zapatilla elegida al lado, en el escenario Tokio. 13
Figura 11. Sección Custom: los cuatro pasos del proceso, el ejemplo de pedido del enunciado y el formulario con 40 pares repartidos y el mensaje de simulación. 14
Figura 12. Validación del formulario Custom: un envío sin pares se bloquea y el foco pasa a la primera talla. 15
Figura 13. Diálogo de acceso de clientes, diseñado por Álvaro, con el error de credenciales. 15
Figura 14. Registro de una cuenta con el aviso de la longitud mínima de la contraseña. 15
Figura 15. Carrito con seis modelos de las seis colecciones y su total estimado. 16
Figura 16. Paso de pago simulado: líneas y total calculados por la base de datos y los dos botones de simulación. 16
Figura 17. «Mis pedidos»: pedidos confirmados y cancelados con su detalle. 17
Figura 18. Recibo de un pedido, generado por el servidor tras el pago simulado. 17
Figura 19. Login único del personal: el usuario comercial `tienda.demo` no tiene panel web y se le deniega el acceso 17
Figura 20. Dashboard del panel de administración con datos de la base de prueba (captura del 11/11/2026, anterior a la página de análisis). 19
Figura 21. Página de estadísticas: indicadores del mes, ventas por día, evolución de ingresos y modelos más vendidos. 19
Figura 22. Pedidos tras marcar el pedido WEB-F056… en reparto: el aviso confirma el cambio y su estado pasa a «enviado». 20
Figura 23. Auditoría del panel con los movimientos generados por compras de la tienda online. 20
Figura 24. Página «Análisis de datos» con el panel de Streamlit integrado, sobre la exportación de prueba de PostgreSQL. 21
Figura 25. Resumen del portal del trabajador con sus cuatro indicadores. 21
Figura 26. Pedidos del portal después de avanzar NOVA-0002, que pasa a «en_preparacion». 22
Figura 27. Inventario del portal: cantidad editable y botón «Guardar» en cada referencia. 22
Figura 28. Pedidos personalizados y configuraciones X-Nova (esta última tabla, solo de lectura y sin datos). 23
Figura 29. Informe de Lighthouse 12.8.2 en escritorio: 100 en accesibilidad y en buenas prácticas. 24
Figura 30. Informe de Lighthouse 12.8.2 en móvil, con las mismas puntuaciones. 24
Figura 31. Ejecución del proceso padre con el pedido CUSTOM-40: los tres pasos terminan con código 0, las cuentas coinciden (40 pares) y se genera la ficha del taller. 34
Figura 32. Dos errores controlados: una cantidad 0 rechazada por el hijo 1 (código 2) y un archivo inexistente detectado por el padre (código 1). 35
Figura 33. El validador lanzado por separado con una tubería: lee el CSV por stdin y escribe los datos normalizados por stdout. 35

Figura 34. Ejecución de `PruebasPsp` (primera parte): puntos 1 a 11 de la batería. 36
Figura 35. Ejecución de `PruebasPsp` (segunda parte): datos mal formados (12a-12m) y resultado final, 27 de 27 pruebas superadas. 37
Figura 36. Prueba propia del analizador de tallas de Álvaro: ocho comprobaciones superadas. 37
Figura 37. Máquina `ubuntu` en OrbStack, en ejecución: Ubuntu 24.04 LTS, arm64 y dominio `ubuntu.orb.local` (captura 1 del documento de Berme). 40
Figura 38. `apt update` leyendo el repositorio `nightly.odoo.com/19.0` (captura 2 del documento de Berme). 41
Figura 39. Instalación y actualización del paquete `odoo` (captura 3 del documento de Berme). 41
Figura 40. Servicio `odoo` activo y habilitado al arranque (captura 4 del documento de Berme). 41
Figura 41. Creación de la base `nova_sneaker` en español, España y sin datos de demostración (captura 11 del documento de Berme). 42
Figura 42. Menú con las aplicaciones instaladas (captura 5 del documento de Berme). 42
Figura 43. Empresa «Nova Sneaker» con su logotipo, en Ajustes → Compañías (captura 12 del documento de Berme). 42
Figura 44. Ficha de la Runner X en Inventario: bien con seguimiento de inventario, precio, coste y existencias (captura 6 del documento de Berme). 43
Figura 45. Componente «Suela»: producto con inventario, 100 unidades y coste de 15 € (captura 13 del documento de Berme). 44
Figura 46. Antes: cinco listas de materiales de la Runner X (captura 16 del documento de Berme). 44
Figura 47. Después: una sola lista, «Runner X (nuevo) 4» (captura 15 del documento de Berme). 44
Figura 48. Runner X con el atributo Talla en una sola línea de nueve valores (captura 26 del documento de Berme). 45
Figura 49. Variantes de la Runner X con su referencia interna `NOVA-4-38` a `NOVA-4-46` (captura 27 del documento de Berme). 45
Figura 50. Usuarios internos de Odoo: administrador, Almacén NOVA y Vendedor NOVA (captura 28 del documento de Berme). 45
Figura 51. Orden nueva en borrador, sin componentes en stock (captura 7 del documento de Berme). 47
Figura 52. WH/MO/00001 con los tres componentes disponibles (captura 8 del documento de Berme). 47
Figura 53. WH/MO/00001 en estado Hecho: consumida una unidad de cada componente (captura 9 del documento de Berme). 47
Figura 54. La Runner X pasa de 50 a 51 unidades (captura 10 del documento de Berme). 47
Figura 55. Presupuesto con 40 Runner X para el Club Atletico NOVA (captura 17 del documento de Berme). .. 48
Figura 56. Pedido S00005 confirmado, con el botón de entrega (captura 18 del documento de Berme). 48
Figura 57. Orden de 40 unidades en borrador, con los componentes disponibles (captura 19 del documento de Berme). 49
Figura 58. WH/MO/00002 confirmada (captura 20 del documento de Berme). 49
Figura 59. WH/MO/00002 en estado Hecho: 40/40 unidades (captura 21 del documento de Berme). 49
Figura 60. WH/OUT/00005 en estado Listo, preparada para validar (captura 22 del documento de Berme). 50
Figura 61. WH/OUT/00005 en estado Hecho (13 nov, 12:53) (captura 23 del documento de Berme). 50
Figura 62. Factura INV/2026/00001 del Club Atletico NOVA, en estado Pagado (captura 24 del documento de Berme). 50
Figura 63. Historial de movimientos de la Runner X: +50, +1 (WH/MO/00001), +40 (WH/MO/00002) y −40 (WH/OUT/00005), con saldo de 51 (captura 25 del documento de Berme). 51
Figura 64. Creación de la clave de API «nova-servidor» en Odoo (captura 14 del documento de Berme). 51

ÍNDICE DE TABLAS Y LISTADOS

Tabla 1. Objetivos específicos por asignatura. 2
Tabla 2. Resultado principal de cada asignatura. 2
Tabla 3. Los tres pedidos de 40 pares que aparecen en el proyecto. 4
Tabla 4. Herramientas y tecnologías de la parte de Interfaces. 7
Tabla 5. Secciones de la web de clientes en el orden en que aparecen. 7
Tabla 6. Archivos de la web de clientes (`01_Interfaces/cliente`). 8
Tabla 7. Ejemplos de clases BEM de la web de clientes. 9
Tabla 8. Cambios principales de la web de clientes según el ancho de pantalla. 10
Tabla 9. Precio orientativo y tallas por colección (`datosColeccion`, valores provisionales). 13
Tabla 10. Páginas del panel de administración. 18
Tabla 11. Requisitos de Interfaces y su cobertura. 23
Tabla 12. Baterías de pruebas de Interfaces y resultados registrados. 24
Tabla 13. Clases de la aplicación PSP (carpeta `03_PSP`). 27
Tabla 14. Códigos de salida del padre y de los hijos. 32
Tabla 15. Requisitos de PSP y su cobertura. 33
Tabla 16. Correspondencia entre los puntos de prueba del enunciado y las pruebas de `PruebasPsp`. 35
Tabla 17. Tareas de Odoo y su documentación en el PDF de Berme. 39
Tabla 18. Rutas y valores del servicio de Odoo. 42
Tabla 19. Aplicaciones de Odoo utilizadas. 43
Tabla 20. Requisitos de Sistemas de Gestión Empresarial y su estado. 46
Tabla 21. Documentos del recorrido S00005 en Odoo. 48
Tabla 22. Pedidos creados en Odoo desde la integración, en el equipo de Berme (pág. 13 de su documento). 52
Tabla 23. Intercambios de datos entre las partes del proyecto y su estado. 54
Tabla 24. Datos comunes del pedido en el CSV de fabricación y en Odoo. 55
Tabla 25. Aportaciones verificables de cada miembro del equipo en las tres asignaturas. 56
Tabla 26. Fuentes utilizadas para elaborar la memoria. 59
Tabla 27. Origen de cada figura de la memoria. 59

Listado 1. Esquema de comunicación entre el padre y los tres hijos (`03_PSP/README.md`). 28
Listado 2. Construcción y arranque del proceso hijo (`GestorProcesos.java, líneas 33-48`). 29
Listado 3. Un hilo por tubería (`GestorProcesos.java, líneas 50-71`). 29
Listado 4. CSV de entrada del pedido CUSTOM-40 (`03_PSP/datos/custom40_fabricacion_psp.csv`). 30
Listado 5. Respuesta del validador para CUSTOM-40 (ejecución del 13/11/2026). 31
Listado 6. Comprobación de cada paso (extracto de `ProcesadorPedidosCustom.java, líneas 203-253`). 32
Listado 7. Instalación de Odoo 19 en Ubuntu (págs. 4-5 del documento de Berme). 41
Listado 8. Formato de las llamadas a la API JSON-2 de Odoo (pág. 14 del documento de Berme). 52
Listado 9. Compilación, ejecución y pruebas de PSP en Windows con el JDK 17 (desde la carpeta `03_PSP`). 58

CAPÍTULO 1. RESUMEN DEL PROYECTO

NOVA Sneakers es el nombre comercial con el que el equipo ha desarrollado el Proyecto Intermodular de segundo curso. El enunciado plantea una fábrica de zapatillas, *Sneaker Factory*, que vende un catálogo propio y fabrica pedidos personalizados para clubes y equipos, y pide al alumnado que actúe como una pequeña consultora que le prepara una primera solución digital. Esta memoria recoge la parte del proyecto que corresponde a tres asignaturas: Desarrollo de Interfaces (0488), Programación de Servicios y Procesos (0490) y Sistemas de Gestión Empresarial (0491).

1.1 Contexto y justificación

Una fábrica pequeña que trabaja por encargo necesita resolver tres cosas a la vez. Hacia fuera, tiene que darse a conocer, enseñar sus productos y permitir que un club le pida un modelo personalizado. Hacia dentro, tiene que registrar a sus clientes y sus ventas, planificar la fabricación, controlar el inventario y facturar. Y, entre ambas, necesita automatizar tareas repetitivas que no deben hacerse a mano, como comprobar que un pedido es correcto o preparar la información que necesita el taller.

El enunciado reparte esas necesidades entre las asignaturas y las une con un caso común: un pedido Custom de 40 pares para un club, repartido en varias tallas. La web corresponde a Desarrollo de Interfaces; el registro empresarial, a Sistemas de Gestión Empresarial con Odoo, y el procesamiento automático de una copia del pedido, a Programación de Servicios y Procesos. Trabajar con un mismo caso obliga a que las tres partes compartan datos, nombres y referencias, y a explicar qué se ha conseguido conectar y qué no.

1.2 Planteamiento del problema

El problema que plantea el enunciado se puede formular como una cadena: un cliente conoce el catálogo en la web y solicita un pedido personalizado; el comercial lo registra en el ERP, que encadena la fabricación, la entrega y la factura; y un subsistema técnico procesa una copia del pedido para ayudar al taller, sin sustituir al ERP. Cada eslabón tiene sus propias exigencias técnicas:

- La web debe ser clara, navegable, adaptable a cualquier pantalla y accesible, y debe explicar el proceso Custom con un formulario o un prototipo de solicitud.
- El ERP debe estar instalado y configurado para la empresa, con clientes, productos con variantes, listas de materiales, órdenes de fabricación, movimientos de inventario y facturación, y debe permitir seguir el pedido de principio a fin.
- El subsistema de procesos debe coordinar varios procesos que se comuniquen entre sí, controlar sus códigos de salida, sus errores y sus tiempos de espera, y producir una salida comprensible relacionada con el pedido original.

1.3 Objetivos del proyecto

El **objetivo general** es construir, para la empresa NOVA Sneakers, una solución en la que la presencia web, la gestión empresarial y el procesamiento automático de pedidos personalizados funcionen con

un mismo caso de negocio y se puedan demostrar con evidencias. De él se derivan los objetivos específicos de cada asignatura:

Tabla 1. Objetivos específicos por asignatura.

Asignatura	Objetivos específicos
Desarrollo de Interfaces	Desarrollar una web con inicio, catálogo, ficha de producto, servicio Custom con formulario y contacto; aplicar una organización de CSS mantenible (BEM); adaptarla a escritorio y móvil; cumplir el nivel AA de WCAG 2.1, y definir y ejecutar una estrategia de pruebas.
Programación de Servicios y Procesos	Crear una aplicación Java que coordine varios procesos con `ProcessBuilder`; comunicarlos por la entrada y la salida estándar; validar el pedido, agruparlo por tallas y generar una ficha para el taller; gestionar códigos de salida, errores y tiempos de espera, y probarlo todo de forma automática.
Sistemas de Gestión Empresarial	Instalar Odoo en una máquina Ubuntu, configurarlo para la empresa y recorrer un pedido Custom de 40 pares a través de la venta, la fabricación, la entrega y la factura, documentando cada paso.

1.4 Resultados obtenidos

La tabla 2 resume el resultado principal de cada asignatura y el apartado en que se desarrolla. Los detalles, las pruebas y los límites de cada una se explican en el capítulo 3.

Tabla 2. Resultado principal de cada asignatura.

Asignatura	Resultado	Apartado
Desarrollo de Interfaces (0488)	Web de clientes con las secciones del enunciado, CSS con BEM y diseño adaptable, ampliada con tienda, panel y portal sobre PostgreSQL. La regresión final no tuvo fallos (por ejemplo, 136/136 en los requisitos) y axe no detectó violaciones de WCAG 2.1 AA. Falta la prueba de usabilidad con personas.	3.1
Programación de Servicios y Procesos (0490)	Proceso padre y tres hijos comunicados por stdin y stdout, con control de códigos de salida, errores y tiempos de espera. 27/27 pruebas y 8/8 comprobaciones del analizador.	3.2
Sistemas de Gestión Empresarial (0491)	Odoo 19 instalado y configurado por Berme; pedido S00005 recorrido hasta la factura pagada. La conexión con el servidor funcionó en su entorno, pero no está activada en el servidor principal.	3.3

1.5 Alcance de la memoria y tratamiento de las evidencias

La memoria describe el estado del repositorio en el commit **372f811** de la rama `main`, que el equipo considera la versión definitiva. Solo trata las tres asignaturas indicadas; las de Acceso a Datos y de Python aparecen únicamente cuando hace falta para entender las otras, por ejemplo porque PostgreSQL es el origen del archivo que procesa PSP.

Cada afirmación se apoya en el código o en una evidencia del repositorio, que se cita con su ruta. Las capturas proceden de las carpetas de evidencias del proyecto y del documento de Odoo de Berme; las pocas que se han tomado expresamente para esta memoria (las ejecuciones de PSP, el probador y los informes de Lighthouse) se indican en su pie y en el anexo. Cuando algo no se ha podido comprobar, o depende de un equipo concreto, se dice de forma explícita.

CAPÍTULO 2. ANTECEDENTES DEL PROYECTO Y ESTADO DEL ARTE

2.1 Estado del arte

Las tecnologías del proyecto son las habituales en el desarrollo profesional y las que se trabajan en el ciclo. Se resumen aquí para justificar por qué se eligieron y qué papel cumplen.

Desarrollo web con estándares

La web de clientes se escribió con **HTML5, CSS3 y JavaScript** sin ningún *framework*. HTML5 aporta elementos semánticos (`header`, `nav`, `main`, `section`, `footer`) que describen la función de cada parte de la página, y el elemento `dialog`, que resuelve de forma nativa las ventanas modales. En CSS, **Flexbox** y **Grid** permiten maquetar sin trucos, las *media queries* adaptan el diseño a cada ancho de pantalla y las variables (*custom properties*) centralizan los colores. Para organizar las clases se adoptó **BEM** (*Block, Element, Modifier*), una convención de nombres que hace explícita la relación entre las piezas de un componente y evita que un estilo afecte por accidente a otra parte de la página.

Prescindir de librerías tiene ventajas en un proyecto de este tamaño: la web no depende de recursos externos, carga rápido y todo su código es del propio equipo. Las partes que necesitan datos reales (tienda, panel y portal) usan un servidor **Node.js con Express** que consulta **PostgreSQL**, y en el panel se incorporaron dos librerías muy extendidas con versión fija: **Chart.js** para los gráficos y **Lucide** para los iconos.

Accesibilidad y pruebas de interfaz

Las **Pautas de Accesibilidad para el Contenido Web (WCAG) 2.1** del W3C son el estándar de referencia, y su nivel AA es el que se exige habitualmente a las webs públicas en Europa. Herramientas como **axe-core** y **Lighthouse** comprueban de forma automática una parte de esos criterios (contraste, nombres accesibles, etiquetas de formulario, estructura de encabezados), y **Playwright** permite automatizar un navegador real para comprobar el comportamiento de la página en distintos tamaños de pantalla. Las pruebas automáticas no sustituyen a la prueba con personas, que mide si la web se entiende y se usa con facilidad.

Procesos y comunicación entre procesos en Java

Un **proceso** es un programa en ejecución con su propia memoria; a diferencia de los hilos, dos procesos no comparten variables y, si fallan, no se arrastran entre sí. Java permite crear procesos hijos con la clase `ProcessBuilder`, que construye el comando y lo arranca, y controlarlos con la clase `Process`: escribir en su entrada estándar, leer su salida estándar y su salida de error, esperar a que terminen con `waitFor` y obtener su código de salida o detenerlos con `destroy` y `destroyForcibly`. Comunicar procesos mediante tuberías, como hace la consola al encadenar comandos con `|`, es una forma sencilla y portable de intercambiar datos.

Sistemas ERP y Odoo

Un **ERP** (*Enterprise Resource Planning*) integra en una sola aplicación los procesos de una empresa: ventas, compras, fabricación, inventario y contabilidad, de modo que un documento genera los siguientes y todos quedan enlazados. **Odoo** es un ERP de código abierto organizado en aplicaciones que se instalan según las necesidades. Su edición *Community* es gratuita, se instala sobre Linux con

PostgreSQL como base de datos e incluye aplicaciones de Ventas, Inventario, Fabricación (con listas de materiales y órdenes de fabricación) y Facturación, además de una API para que otros programas consulten o creen datos.

2.2 El caso de empresa y el pedido Custom

El equipo usa la marca **NOVA Sneakers** en todas las partes del proyecto. El catálogo de la web tiene cien modelos en seis colecciones (Street, Luxe, Casual, Hombre, Mujer y Unisex), y el servicio Custom permite a un club pedir un modelo con su propia personalización y un reparto de pares por talla.

En el proyecto aparecen tres pedidos de 40 pares que no deben confundirse, porque cada uno se usa en una asignatura distinta y **no son el mismo pedido**:

Tabla 3. Los tres pedidos de 40 pares que aparecen en el proyecto.

Pedido	Qué es	Dónde se usa
SO024	Ejemplo de pedido Custom del enunciado: Club Halcones, Runner X, 40 pares de la talla 38 a la 46.	Como referencia en la web (texto de ejemplo del formulario). No se ha creado en Odoo.
S00005	Venta real en el Odoo de Berme: Club Atletico NOVA, 40 Runner X, con fabricación, entrega y factura.	Caso documentado de Sistemas de Gestión Empresarial.
CUSTOM-40	Pedido de los datos de prueba de PostgreSQL: Club Atletico NOVA, NOVA Velocity roja, tallas 40 a 43.	Entrada de demostración de PSP, exportada con el script 15.

El equipo decidió presentar S00005 como caso de Odoo, sin crear ni renombrar ninguna venta, y tratar SO024 como lo que es en el enunciado: un ejemplo. El nombre del cliente y la cantidad de S00005 y CUSTOM-40 coinciden, pero no existe ninguna exportación que demuestre que se trata del mismo pedido.

2.3 Relación entre las asignaturas

La figura 1 resume cómo se relacionan las tres asignaturas en la versión entregada. Las aplicaciones de Interfaces trabajan contra un servidor Node/Express que consulta PostgreSQL; de PostgreSQL sale, mediante el script 15, el CSV que procesa PSP, y Odoo lleva el flujo empresarial completo en la máquina de Berme, donde también funcionó el envío de pedidos desde el servidor; ese conector no está activado en el servidor principal. La relación que falta es la exportación de una venta de Odoo hacia PSP.

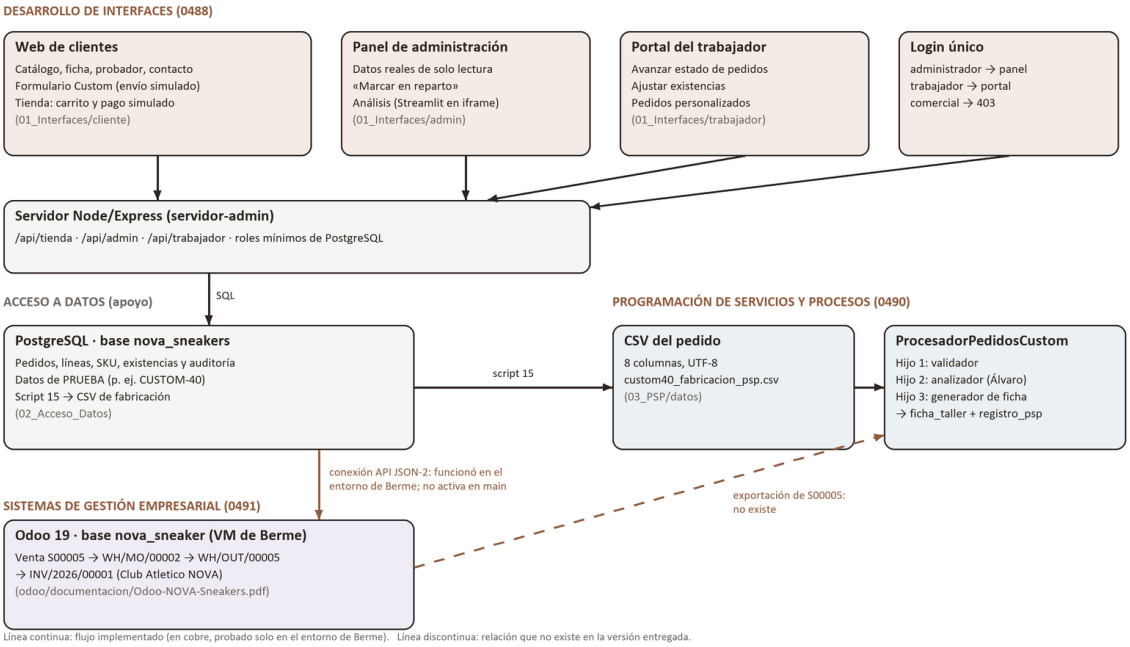


Figura 1. Relación entre las tres asignaturas en la versión entregada (esquema de elaboración propia a partir del código de 372f811 y del documento de Odoo).

CAPÍTULO 3. DESARROLLO DEL PROYECTO

Este capítulo describe el trabajo de cada asignatura con la misma estructura: planificación y metodología, descripción de la solución, requisitos y resultados. Todo lo que se explica corresponde al código y a las evidencias del commit 372f811; cuando una parte no se ha podido comprobar, se indica.

3.1 Desarrollo de Interfaces (0488)

La parte de Interfaces comprende tres aplicaciones web. La **web de clientes** es la que pide el enunciado: presenta la marca, el catálogo, la ficha de cada modelo, el servicio Custom para clubes y el contacto. Sobre esa misma página el equipo añadió una **tienda online** con carrito y pago simulado. Las otras dos son de uso interno: el **panel de administración** y el **portal del trabajador**, a los que se entra con un login único.

> La tienda, el panel y el portal son ampliaciones: ninguna rúbrica los exige y no sustituyen a ningún requisito obligatorio. Se describen porque forman parte del código entregado y porque se probaron igual que la web, por eso en cada apartado se distingue lo que pide el enunciado (estructura, catálogo, Custom y diseño adaptable) de lo que añadió el equipo.

3.1.1 Planificación y metodología

Planificación del trabajo

El desarrollo siguió un orden que va de lo obligatorio a lo opcional:

1. **Escaparate y catálogo.** Omar construyó la portada «El escaparate», el catálogo y la ficha de producto, y más adelante el probador virtual.
2. **Panel de administración.** En paralelo, Álvaro diseñó el panel como un conjunto de páginas HTML con su hoja de estilos, gráficos con Chart.js y varias páginas prototipo. También aportó a la web de clientes el diálogo de inicio de sesión y una paleta de colores por colección para las tarjetas del catálogo.
3. **Cierre de los requisitos.** Se añadieron el precio orientativo, la descripción y las tallas de cada colección, el formulario Custom con su cálculo de pares y la sección de contacto, y se probó todo en varios anchos de pantalla y con herramientas de accesibilidad.
4. **Ampliaciones con datos reales.** Se conectó la web a un servidor Node/Express que consulta PostgreSQL. Así nacieron la tienda (cuentas, carrito, pedidos y pago simulado), el panel con datos reales y el login único. El portal del trabajador, escrito por Berme, se integró detrás de ese login.
5. **Revisión final.** Se unificó el pie de página, se renombraron las clases de la web de clientes a BEM, se corrigieron los contrastes de los colores por colección y se repitió toda la batería de pruebas sobre el árbol final.

Metodología de trabajo

Cada persona trabajó en su propia rama de Git y los cambios se integraron mediante fusiones que conservan la autoría original. Cuando dos ramas modificaban los mismos archivos, como ocurrió con el diálogo de acceso y las tarjetas del catálogo de Álvaro frente a los precios añadidos por Omar, los

conflictos se resolvieron a mano y la decisión quedó explicada en el mensaje de la fusión y en el `README.md` de la web.

Los requisitos se tomaron directamente del enunciado intermodular y de los criterios de evaluación (RA4 sobre usabilidad y accesibilidad, y RA8 sobre pruebas). Para cada uno se definió una comprobación concreta y un criterio de aceptación en `01_Interfaces/memoria/ESTRATEGIA_PRUEBAS_INTERFACES.md`. Como estándar de accesibilidad se eligió **WCAG 2.1 nivel AA**, que es el que miden las herramientas utilizadas, con un criterio propio del equipo: cero violaciones de axe y una puntuación de Lighthouse igual o superior a 90.

Herramientas utilizadas

Tabla 4. Herramientas y tecnologías de la parte de Interfaces.

Herramienta	Uso en el proyecto
HTML5, CSS3 y JavaScript	La web de clientes se escribió sin librerías, sin compilación y sin dependencias externas: no carga ningún recurso de otro dominio.
Node.js 24, Express 5.3 y pg 8.23	Servidor `servidor-admin` que sirve la tienda, el panel y el portal y consulta PostgreSQL (ampliación).
PostgreSQL 17	Base de datos con las funciones de la tienda (script 20), del reparto (22) y del portal (23).
Chart.js 4.4.1 y Lucide 1.62.0	Gráficos e iconos del panel y del portal, cargados desde una CDN con versión fija y comprobación de integridad (SRI).
Streamlit	Panel de datos de la asignatura de Python, que el panel de administración muestra dentro de un `iframe`.
Microsoft Edge 157 y Playwright 1.65	Pruebas automáticas de navegador en escritorio y móvil.
axe-core 4.12.1 y Lighthouse 12.8.2	Auditorías de accesibilidad (reglas WCAG 2.1 A y AA) y de buenas prácticas.
Git y GitHub	Ramas por persona, fusiones y revisión del historial.

3.1.2 Descripción de la solución

Estructura de la web de clientes

La web de clientes es una sola página, `01_Interfaces/cliente/index.html`, organizada en bloques semánticos. Empieza con un enlace oculto «Saltar al escaparate», que aparece al recibir el foco y permite a quien navega con teclado ir directamente al contenido. Le siguen la cabecera (`header`), con el logotipo y un `nav` de cuatro enlaces (Catálogo, Nuestra visión, Custom y Contacto); una barra con la cuenta y el carrito; el contenido principal (`main`) con cuatro secciones, y un único pie (`footer`) que contiene los datos de contacto. La figura 2 muestra la parte superior de la página en escritorio.

Tabla 5. Secciones de la web de clientes en el orden en que aparecen.

Sección	Identificador	Contenido
El escaparate	`#catalogo`	Único h1 de la página. Filtros por colección, buscador y una selección de hasta ocho modelos.
Todos los modelos	`#catalogo-completo`	Catálogo completo con sus filtros, buscador y las tarjetas de los 100 modelos.
Nuestra visión	`#historia`	Presentación de la marca («Muévete a tu manera»).
Zapatillas para tu equipo	`#custom`	Explicación del proceso Custom en cuatro pasos y formulario de solicitud.
Hablemos	`#contacto`	Pie de página con correo, dirección de la fábrica y horario (datos ficticios).

Las ventanas emergentes se construyeron con el elemento nativo `dialog`: la ficha del modelo, el probador, el acceso, el registro, el carrito, el proceso de compra y «Mis pedidos». Se abren con `showModal()`, lo que hace que el resto de la página quede inactivo y que la tecla Escape las cierre sin código adicional. Todas tienen un título enlazado con `aria-labelledby` y un botón de cierre con su `aria-label`, y al cerrar la ficha el foco vuelve a la tarjeta desde la que se abrió.

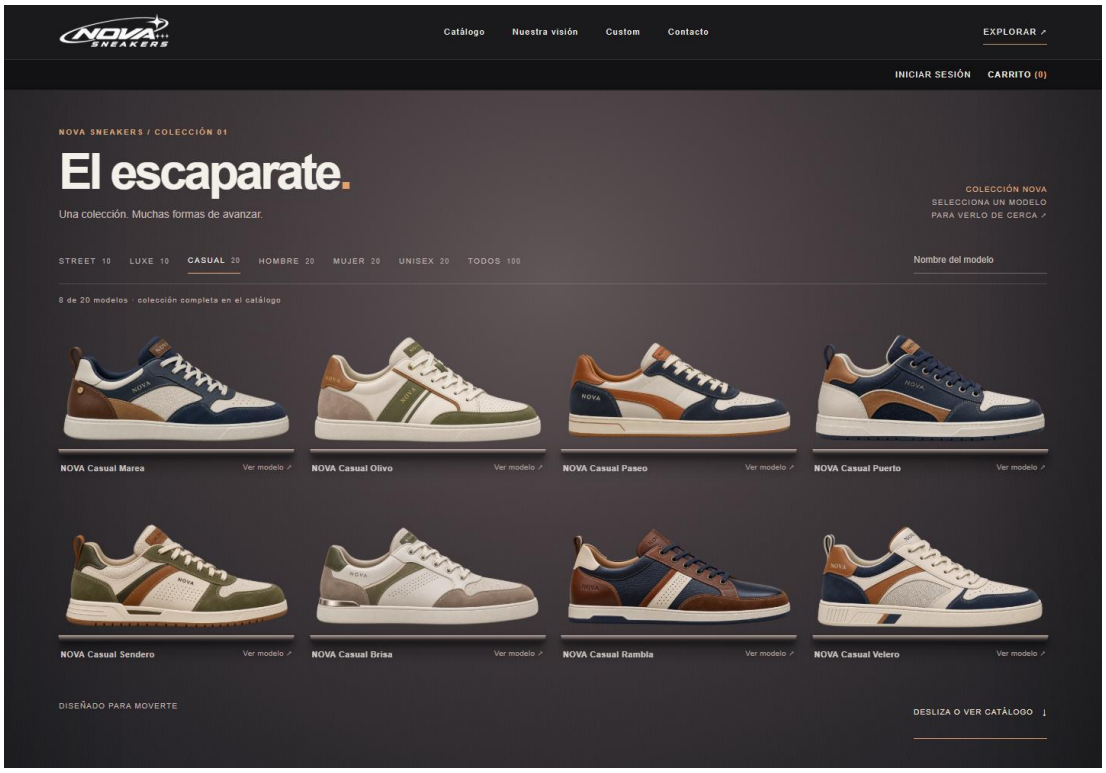


Figura 2. Portada de la web de clientes en escritorio (1440 px): cabecera con el menú, barra de cuenta y carrito y el escaparate con los filtros por colección.

Organización de HTML, CSS y JavaScript

El código de la web de clientes se reparte en siete archivos con responsabilidades separadas (tabla 6). Los datos del catálogo están aislados en `modelos.js`, de forma que cambiar un precio o añadir un modelo no obliga a tocar la lógica de la página.

Tabla 6. Archivos de la web de clientes (`01_Interfaces/cliente`).

Archivo	Líneas	Responsabilidad
`index.html`	820	Todo el marcado de la página y de sus siete diálogos.
`css/inicio.css`	1616	Escaparate, catálogo, ficha, probador, Custom, pie y diálogo de acceso.
`css/tienda.css`	650	Barra de cuenta, bloque de compra, carrito, pago y pedidos.
`js/modelos.js`	675	Datos: los 100 modelos y los datos por colección (precio y tallas).
`js/inicio.js`	479	Escaparate, catálogo, ficha, gesto táctil, probador y diálogo de acceso.
`js/custom.js`	52	Formulario Custom: suma de pares y mensaje de simulación.
`js/tienda.js`	1109	Tienda online: sesión, carrito, compra y «Mis pedidos».

Las hojas de estilo se cargan en orden (`inicio.css` y después `tienda.css`), así que la segunda puede ajustar lo que define la primera. Los cuatro scripts son clásicos, se cargan con `defer` en el orden modelos, inicio, custom y tienda, y comparten el ámbito global: `tienda.js` reutiliza funciones de `inicio.js` como el formato de precios o el pintado del catálogo. Como en el primer pintado

`tienda.js` todavía no se ha ejecutado, `inicio.js` comprueba con `typeof` que sus funciones existen antes de llamarlas.

En el CSS solo hay tres variables globales (`--tinta`, `--suave` y `--brillo`), y cada colección define dos más, `--col-claro` y `--col-oscuro`, en su modificador. La tipografía es Arial a 14 px con interlineado 1,5, y los titulares usan `clamp()` para crecer con la pantalla entre un mínimo y un máximo (por ejemplo, el `h1` va de 2,4 a 5 rem). La maquetación combina Flexbox para las filas de elementos (cabecera, menú, filtros, tarjetas) y CSS Grid para las rejillas: cuatro columnas en el catálogo, dos en la sección de historia y en la ficha, nueve en las tallas del formulario Custom.

Metodología BEM

Las clases de la web de clientes siguen la convención BEM (*Block, Element, Modifier*): un bloque es un componente independiente (`escaparate`, `catalogo`, `compra`), un elemento es una parte que solo tiene sentido dentro de su bloque y se escribe con dos guiones bajos (`catalogo__tarjeta`), y un modificador es una variante o un estado y se escribe con dos guiones (`catalogo__tarjeta--luxe`). El renombrado se hizo en un único commit, `e0d2694` («Rename the shop web classes to BEM»), que tocó el HTML, las dos hojas de estilo y los cuatro scripts; su mensaje indica que los estilos calculados y los resultados de las pruebas eran idénticos antes y después.

Tabla 7. Ejemplos de clases BEM de la web de clientes.

Clase	Qué representa
`escaparate__filtro` / `escaparate__filtro--activo`	Botón de filtro del escaparate y su estado seleccionado.
`catalogo__tarjeta catalogo__tarjeta--street`	Tarjeta del catálogo con los colores de su colección.
`catalogo__estado--agotado` / `--disponible`	Etiqueta de disponibilidad de cada modelo.
`probador__vista--con-figura` / `--solo-zapatilla`	Modo de presentación del probador.
`acceso__caja`, `acceso__campo`, `acceso__error`	Partes del diálogo de inicio de sesión.
`compra__grupo--error`, `compra__stock--pocos`	Estados del bloque de compra de la ficha.
`pedido__estado--enviado`	Estado de un pedido en «Mis pedidos».

No todo el CSS sigue BEM de forma estricta. Quedan algunos selectores por identificador (`#detalle-precio`, `#probador`), selectores de etiqueta dentro de un bloque (`nav a`, `.custom__tallas input`) y estados expresados con atributos de accesibilidad en lugar de con modificadores (`[aria-pressed="true"]`, `[aria-current="step"]`, `[aria-invalid]`). Esto último es intencionado: el atributo ya informa a los lectores de pantalla y el estilo se engancha a él para que estado visual y estado accesible no puedan separarse. El panel y el portal **no usan BEM**: sus clases son planas con el prefijo `adm-` y el commit de renombrado los dejó fuera expresamente.

Diseño adaptable a escritorio y móvil

El diseño adaptable se resuelve con media queries de ancho máximo sobre una maquetación de escritorio. El punto de corte principal está en **760 px**, que aparece en once bloques de las dos hojas de estilo; un segundo corte en **370 px** ajusta los móviles más estrechos, y cinco bloques `min-width: 761px` contienen reglas que solo tienen sentido en pantallas grandes, como la ficha a dos columnas. Además, la regla `prefers-reduced-motion: reduce` desactiva el desplazamiento suave, las transiciones, las animaciones y la animación de entrada para quien lo tenga configurado en su sistema.

Tabla 8. Cambios principales de la web de clientes según el ancho de pantalla.

Ancho	Qué cambia
≤ 760 px	Cabecera de 64 px y logotipo de 115 px; se ocultan «Nuestra visión» y «Explorar» del menú; escaparate y catálogo a dos columnas; historia, Custom, contacto, ficha y probador a una columna; tallas del formulario Custom en tres columnas; botones apilados y líneas del carrito en tres filas.
≤ 370 px	Se oculta la instrucción del escaparate; buscador de 100 px, logotipo de 95 px y menos separación entre enlaces; texto de la barra de la tienda más pequeño.
≥ 761 px	Ficha a dos columnas con áreas de rejilla; probador con la figura anclada abajo y controles con desplazamiento propio.

La web no tiene menú hamburguesa: en el móvil caben tres enlaces y «Nuestra visión» se alcanza bajando por la página. Para que los enlaces sean fáciles de pulsar, llevan un relleno vertical de 10 px que amplía su zona táctil. Las imágenes de producto están en formato WebP (120 archivos, seis carpetas de veinte) y las del catálogo se cargan con `loading="lazy"`: el navegador solo las descarga cuando se acercan a la zona visible. Las figuras 3 a 6 muestran cuatro pantallas en móvil: la portada, el catálogo, «Mis pedidos» y el dashboard del panel de administración, que a menos de 900 px coloca el menú en horizontal y las tarjetas en dos columnas.

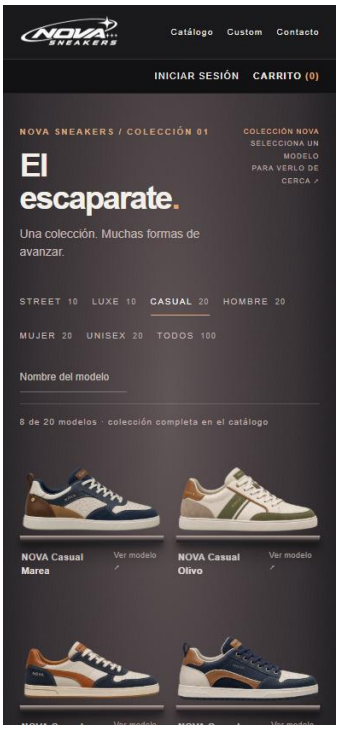


Figura 3. Portada en un móvil de 390 px: menú de tres enlaces y escaparate a dos columnas.

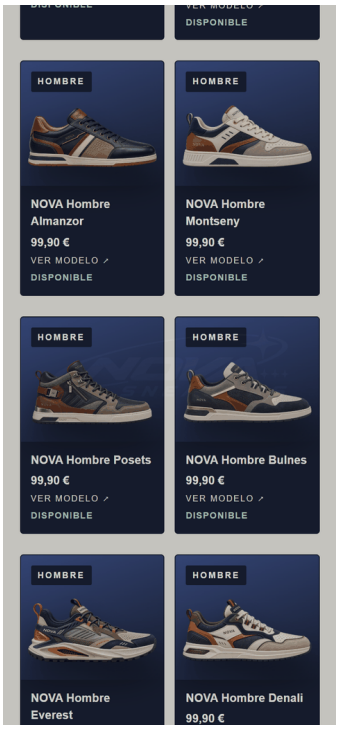


Figura 4. Catálogo en móvil (480 px) con el precio y la disponibilidad de cada modelo.

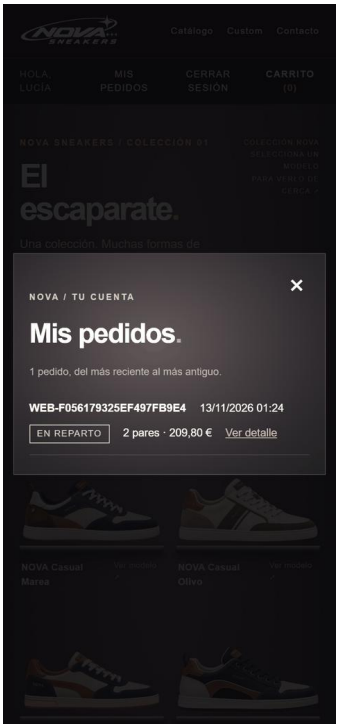


Figura 5. «Mis pedidos» en móvil con un pedido «En reparto».



Figura 6. Dashboard del panel en móvil: menú horizontal y tarjetas en dos columnas.

Navegación e interacción

El menú usa anclas a las secciones con desplazamiento suave (`scroll-behavior: smooth`), y el pie incluye un enlace «Volver arriba». El escaparate tiene siete filtros por colección, con «Casual» activo al cargar, y un buscador que filtra por parte del nombre sin distinguir mayúsculas (`toLocaleLowerCase("es")`). Solo muestra ocho modelos y un texto del tipo «8 de 20 modelos · colección completa en el catálogo» que invita a bajar al catálogo completo, donde se pintan todos los resultados (figura 7). Cada filtro indica cuántos modelos tiene, y su estado se comunica con `aria-pressed`.

En pantallas táctiles, el pie del escaparate tiene un gesto propio: al deslizar hacia arriba más de 65 px, con un movimiento claramente vertical, la página baja hasta el catálogo completo. A partir de 10 px se cancela el desplazamiento por defecto para que la inercia del dedo no se salte el catálogo.

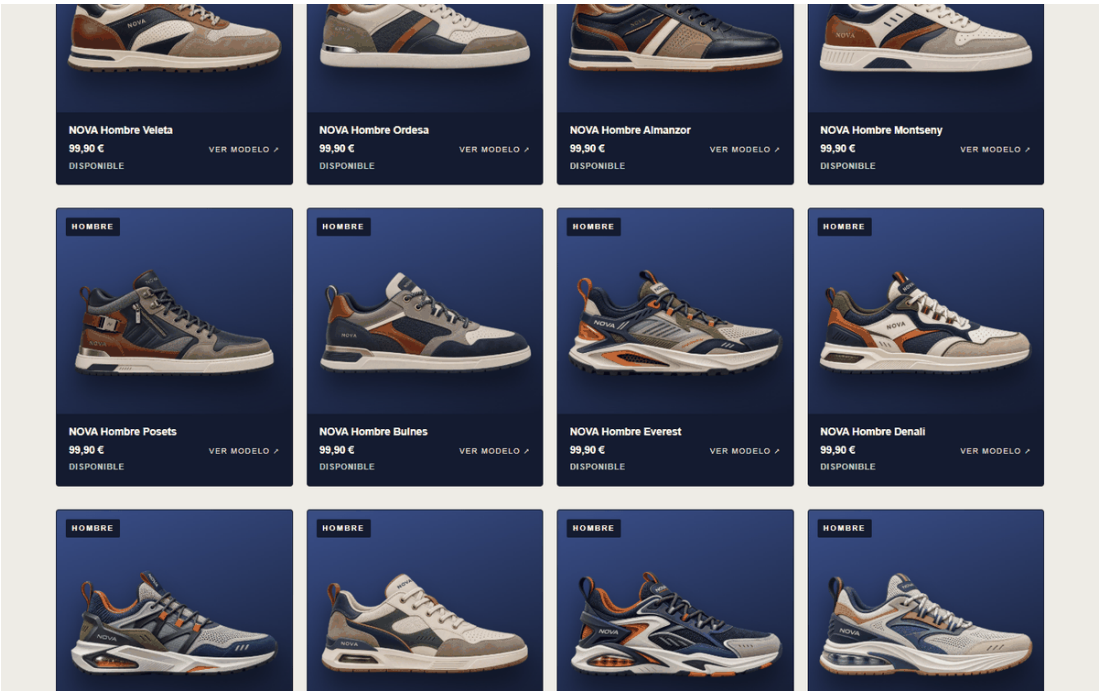


Figura 7. Catálogo completo: tarjetas de la colección Hombre con el nombre comercial, el precio y la disponibilidad, sobre el color de su colección.

Al pulsar una tarjeta se abre la ficha del modelo (figura 8). Muestra la colección, el nombre, una descripción, el precio, el color y las tallas; si la tienda está disponible, añade la elección de talla, la cantidad con botones − y + y el botón «Añadir al carrito». Desde la ficha se accede también al probador.

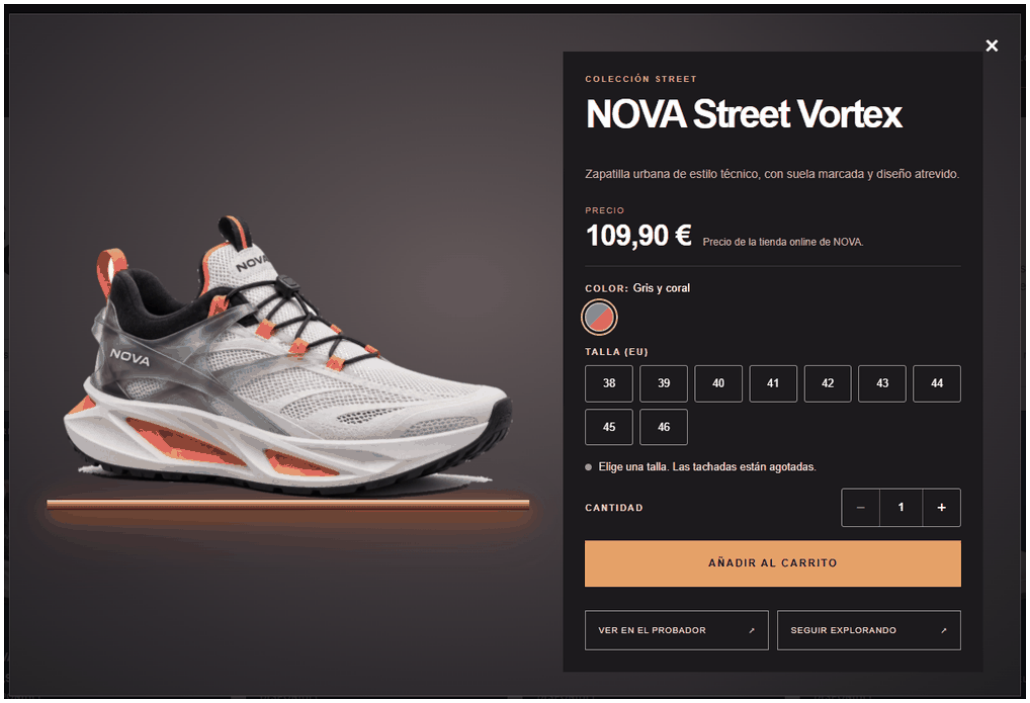


Figura 8. Ficha del modelo NOVA Street Vortex: precio, color, tallas de la 38 a la 46, cantidad y acceso al carrito y al probador.

El probador es una vista conceptual: permite elegir cualquiera de los cien modelos, indicar estatura (de 120 a 220 cm) y peso (de 30 a 200 kg) y cambiar entre dos escenarios, Madrid y Tokio. La estatura escala la figura y, con el peso, se calcula el índice de masa corporal solo para elegir entre tres figuras (delgada, media o amplia), sin ninguna recomendación. Cuatro modelos tienen fotografías con la

zapatilla puesta (figura 9); para los demás se muestra una figura con calzado genérico y la zapatilla elegida en una tarjeta al lado, con un aviso que lo explica (figura 10).



Figura 9. Probador con el modelo NOVA Casual Marea puesto, en el escenario Madrid.

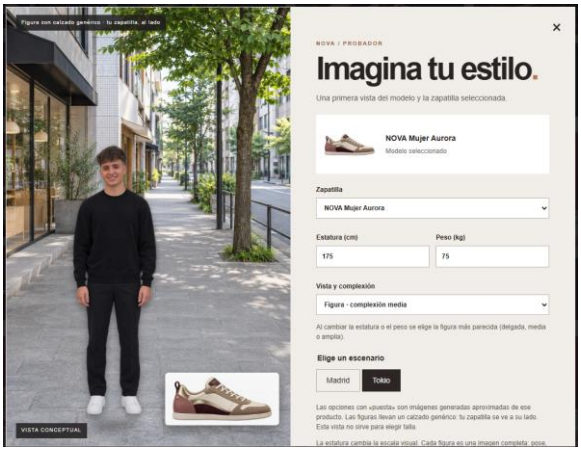


Figura 10. Figura de complexión media con calzado genérico y la zapatilla elegida al lado, en el escenario Tokio.

> Las dos capturas del probador se tomaron el 13/11/2026 con Microsoft Edge sobre la web del commit 372f811, servida como archivos estáticos, porque el repositorio no contenía ninguna captura del probador abierto.

Catálogo

Los datos del catálogo están en `js/modelos.js`: un array `modelos` con 100 objetos (Street 10, Luxe 10, Casual 20, Hombre 20, Mujer 20 y Unisex 20), cada uno con su nombre comercial, su colección y su imagen, y un objeto `datosColeccion` con el precio orientativo, la descripción y las tallas de cada colección. Los precios y las tallas están marcados en el propio archivo como **provisionales**, pendientes de acordarse con Odoo.

Tabla 9. Precio orientativo y tallas por colección (`datosColeccion`, valores provisionales).

Colección	Precio orientativo	Tallas
Street	109,90 €	38-46
Luxe	139,90 €	38-46
Casual	89,90 €	38-46
Hombre	99,90 €	40-46
Mujer	99,90 €	35-42
Unisex	94,90 €	36-46

Cada tarjeta es un elemento `button`, lo que la hace accesible con el teclado sin código adicional, y su `aria-label` incluye el nombre y el precio. Si el servidor de la tienda responde, la tarjeta muestra el precio real de la base de datos; si no, el orientativo de la colección. Para que un modelo se pueda comprar tiene que existir en la base con el mismo nombre (el script `21_catalogo_web.sql` crea los cien), y la tarjeta indica «No disponible» si no está y «Agotado» si no queda stock en ninguna talla.

Formularios

El formulario Custom (figura 11) es el núcleo del requisito de personalización. Pide el club o equipo, la persona de contacto, el correo, el modelo (con una lista de sugerencias generada con los cien nombres,

aunque admite texto libre), la personalización y, de forma opcional, el teléfono y los colores. Las cantidades se introducen en nueve campos numéricos, de la talla 38 a la 46, agrupados en un `fieldset` con su `legend`.

La validación básica la hace el navegador con los atributos `required`, `type="email"`, `min="0"` y `step="1"`. El script `custom.js` añade dos comportamientos: recalcula el total de pares cada vez que cambia una talla y, al enviar, impide la recarga de la página con `preventDefault()`. Si el total es cero muestra «Indica cuántos pares quieres de al menos una talla» y lleva el foco a la talla 38 (figura 12); si es correcto, muestra en una zona `role="status"` un resumen de la solicitud que empieza por «SIMULACIÓN» y aclara que no se ha enviado ningún dato. Un aviso de «Prototipo» encima del formulario lo deja claro desde el principio.



Figura 11. Sección Custom: los cuatro pasos del proceso, el ejemplo de pedido del enunciado y el formulario con 40 pares repartidos y el mensaje de simulación.

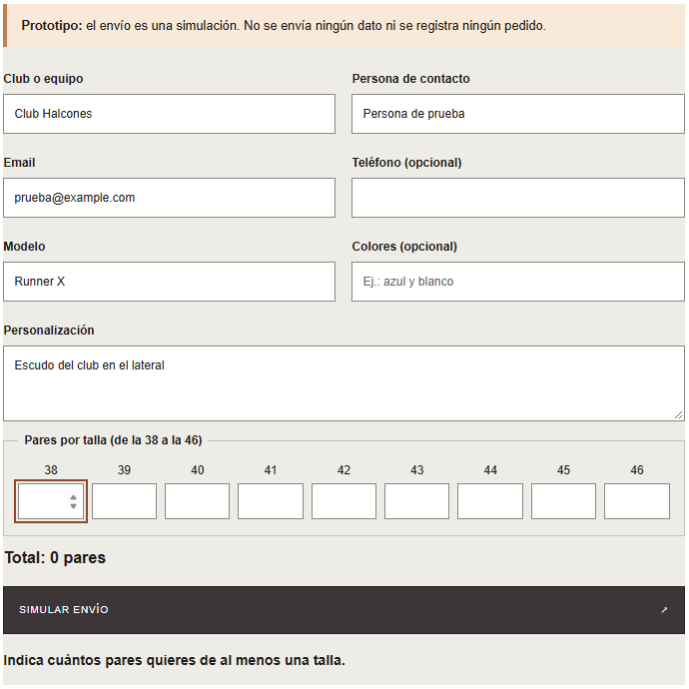


Figura 12. Validación del formulario Custom: un envío sin pares se bloquea y el foco pasa a la primera talla.

La sección de contacto no tiene formulario: muestra los datos en una lista de definiciones (`dl`) con un enlace `mailto:`. Los formularios de acceso y registro de la tienda llevan el atributo `novalidate` porque la validación la hace JavaScript, que coloca cada mensaje junto a su campo, lo anuncia con `role="alert"` y marca el campo con `aria-invalid`. El diálogo de acceso es aportación de Álvaro (figura 13) e incluye un botón para mostrar u ocultar la contraseña que actualiza `aria-pressed`. El registro (figura 14) exige nombre, un correo válido y una contraseña de al menos ocho caracteres; el servidor añade sus propias respuestas, como el aviso de que un correo ya está registrado.



Figura 13. Diálogo de acceso de clientes, diseñado por Álvaro, con el error de credenciales.



Figura 14. Registro de una cuenta con el aviso de la longitud mínima de la contraseña.

Carrito y pedidos

El carrito (figura 15) se guarda en el `localStorage` del navegador con la clave `nova_carrito`, y solo almacena el SKU y la cantidad de cada línea. Al leerlo se sanea (cantidades enteras y positivas, un máximo de 10 pares por talla y 20 líneas) y, al cargar el catálogo, se contrasta con el stock real: lo que ya no existe o está agotado se retira con un aviso. El total que muestra es una estimación; el definitivo lo calcula siempre la base de datos.

La compra sigue tres pasos dentro de un mismo diálogo. Al pulsar «Tramitar pedido» sin sesión iniciada, se abre el acceso y la compra queda pendiente. En el paso de entrega se pide la dirección. Al confirmarla, el servidor crea un pedido **pendiente** con los precios calculados por PostgreSQL, sin descontar stock todavía. En el paso de pago (figura 16) hay dos botones, «Simular pago aceptado» y «Simular pago rechazado»: el primero confirma el pedido y un trigger de la base descuenta el stock; el segundo lo cancela. Para que un doble clic no cree dos pedidos, cada compra lleva una clave generada con `crypto.randomUUID()`.

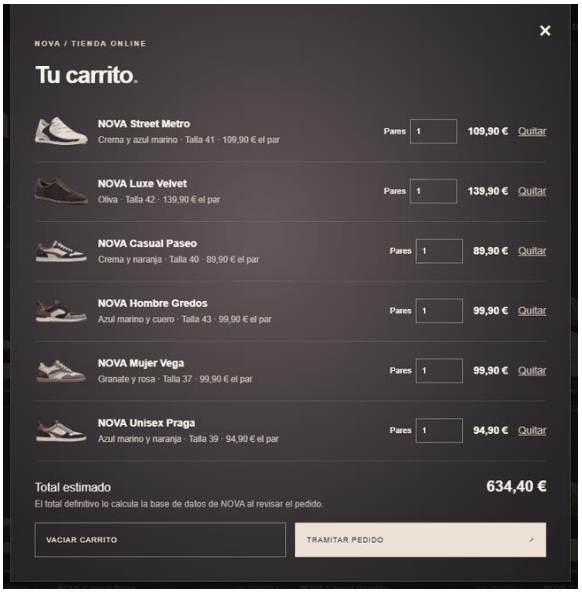


Figura 15. Carrito con seis modelos de las seis colecciones y su total estimado.

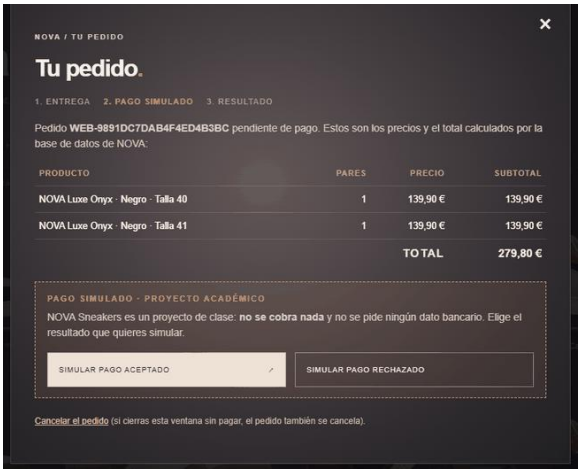


Figura 16. Paso de pago simulado: líneas y total calculados por la base de datos y los dos botones de simulación.

En «Mis pedidos» (figura 17) el cliente ve sus pedidos del más reciente al más antiguo, con el estado traducido a un texto comprensible (el estado técnico `enviado` aparece como «En reparto») y el detalle de las líneas. Tras un pago aceptado, el servidor envía un recibo por correo (figura 18); en el entorno de desarrollo puede usarse un servicio de pruebas (Ethereal), y entonces aparece un enlace «Ver recibo de prueba».



Figura 17. «Mis pedidos»: pedidos confirmados y cancelados con su detalle.



Figura 18. Recibo de un pedido, generado por el servidor tras el pago simulado.

En la tienda hay que distinguir lo simulado de lo real. Son simulados el **pago**, porque no se pide ningún dato bancario y es el navegador quien elige el resultado, y el **envío del formulario Custom**. Es real todo lo demás: las cuentas (con contraseñas cifradas con bcrypt), la sesión (una cookie `httpOnly` y `SameSite=strict` de dos horas), los pedidos, los precios, el stock y su historial, que pasan por funciones de PostgreSQL. El servidor nunca toma del navegador ni precios ni identificadores de cliente.

Acceso del personal

El panel y el portal comparten una página de acceso, «Acceso del personal». El navegador envía usuario y contraseña al servidor, que los comprueba en PostgreSQL y devuelve a qué página debe ir cada perfil: el administrador al panel y el trabajador al portal. Un perfil sin panel web, como el comercial, recibe un error 403 y no obtiene sesión (figura 19). Tras cinco intentos fallidos con un mismo usuario, el acceso se bloquea cinco minutos.

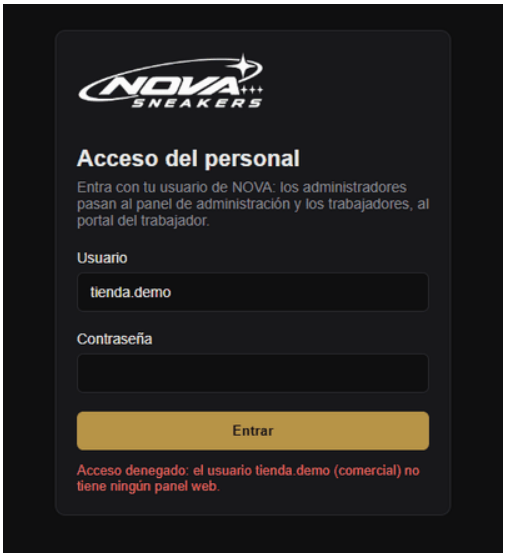


Figura 19. Login único del personal: el usuario comercial `tienda.demo` no tiene panel web y se le deniega el acceso.

Cada petición posterior vuelve a comprobar el rol en la base de datos, de modo que una cuenta desactivada o con otro rol pierde el acceso sin esperar a que caduque la sesión. Además, las páginas

escuchan el evento `pageshow` y se recargan si el navegador las muestra desde su memoria, para que el botón «Atrás» no enseñe datos después de cerrar sesión.

Funcionalidades del panel de administración

El diseño del panel es de Álvaro: una barra lateral de 240 px con el logotipo y el menú, y una zona de contenido con tarjetas de indicadores, tablas y gráficos sobre un tema oscuro con acento dorado. Sus colores se definen como variables en el bloque `.adm` (por ejemplo, `--adm-panel: #18181b` y `--adm-dorado: #c9a24b`). Tiene dos puntos de corte: a 900 px la barra lateral pasa arriba con el menú en horizontal y las tarjetas se colocan en dos columnas, y a 480 px se reducen rellenos y tamaños.

Tabla 10. Páginas del panel de administración.

Página	Contenido	Datos
Dashboard	Cuatro indicadores (ventas acumuladas, pedidos del día, pendientes y sugerencias), últimas operaciones, pedidos por estado, ventas por día y stock bajo.	Reales
Pedidos	Tabla con buscador, filtro por estado y botón «Marcar en reparto».	Reales
Productos, Inventario, Clientes, Trabajadores, Auditoría	Tablas con buscador y filtro.	Reales
Estadísticas	Indicadores del mes, gráfico de barras y de línea, pedidos por estado y modelos más vendidos.	Reales
Análisis de datos	Panel de Streamlit de la asignatura de Python dentro de un `iframe`.	Reales
Sugerencias, Mejoras, Configuración	Tabla de ejemplo, tablero kanban y ajustes.	Prototipo

Las páginas de prototipo llevan un aviso visible que lo indica, y el botón «Guardar» de la configuración solo muestra un mensaje. En las páginas con datos, `script.js` pide al servidor solo las filas filtradas; el buscador espera 300 ms tras la última tecla antes de preguntar y, si llegan varias respuestas, solo se pinta la última. Las celdas se rellenan con `textContent`, nunca con HTML, para que un dato que contenga etiquetas no se ejecute. Las consultas se hacen en PostgreSQL con un rol de solo lectura (`panel_lectura`) dentro de una transacción `READ ONLY`. El dashboard reúne en una sola vista los indicadores principales (figura 20).

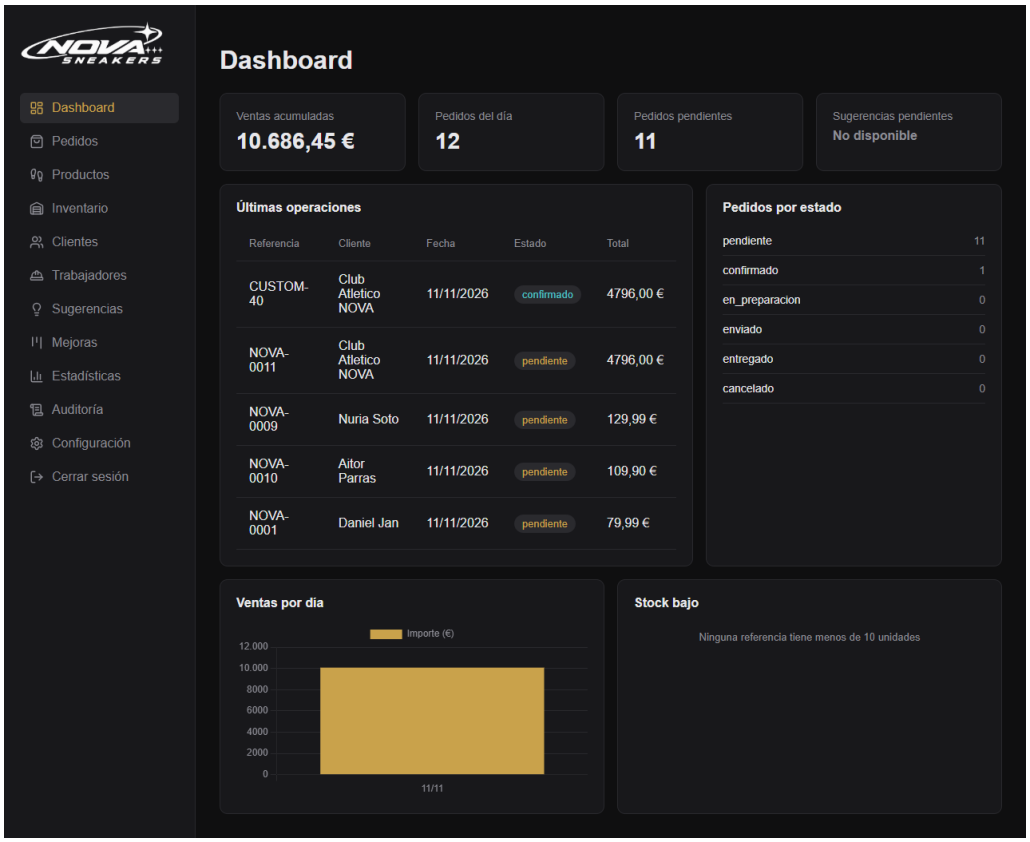


Figura 20. Dashboard del panel de administración con datos de la base de prueba (captura del 11/11/2026, anterior a la página de análisis).

Los gráficos usan Chart.js 4.4.1 (figura 21). Cada `canvas` lleva `role="img"` y un `aria-label` que describe su contenido, y si la librería no se carga, la página muestra un aviso en lugar del gráfico. El gráfico de ventas por día abarca los últimos 30 días y no cuenta los pedidos cancelados.



Figura 21. Página de estadísticas: indicadores del mes, ventas por día, evolución de ingresos y modelos más vendidos.

La única operación de escritura del panel es «Marcar en reparto», que pasa un pedido confirmado al estado `enviado`. El botón solo aparece en los pedidos confirmados, pide confirmación y se desactiva para evitar un segundo envío. El servidor ejecuta la operación con un rol que solo puede llamar a la función `marcar_pedido_en_reparto` del script 22, y el resultado se anuncia en una zona `role="status"` que recibe el foco (figura 22). Si el pedido ya había cambiado de estado, el servidor responde con un error 409 y el panel lo muestra en rojo.

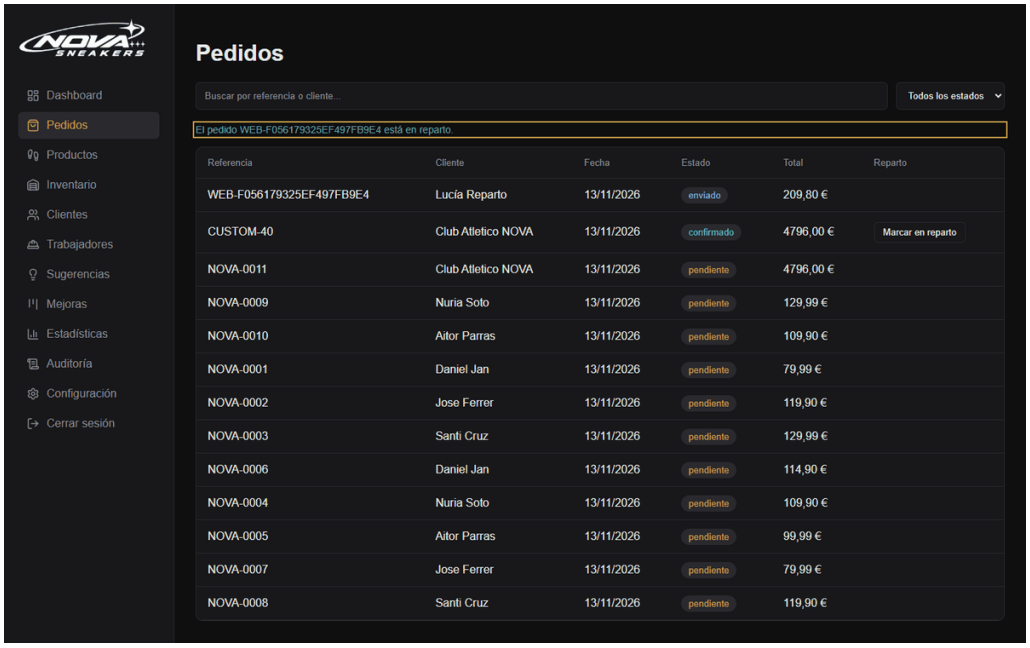


Figura 22. Pedidos tras marcar el pedido WEB-F056… en reparto: el aviso confirma el cambio y su estado pasa a «enviado».

La auditoría (figura 23) muestra los cambios que registran los triggers de la base: altas, cambios de estado y de stock y bajas lógicas, con su fecha, usuario, tabla y detalle. La página de análisis comprueba primero si Streamlit está arrancado y, si no lo está, explica cómo arrancarlo; si responde, lo incrusta con su tema oscuro (figura 24).

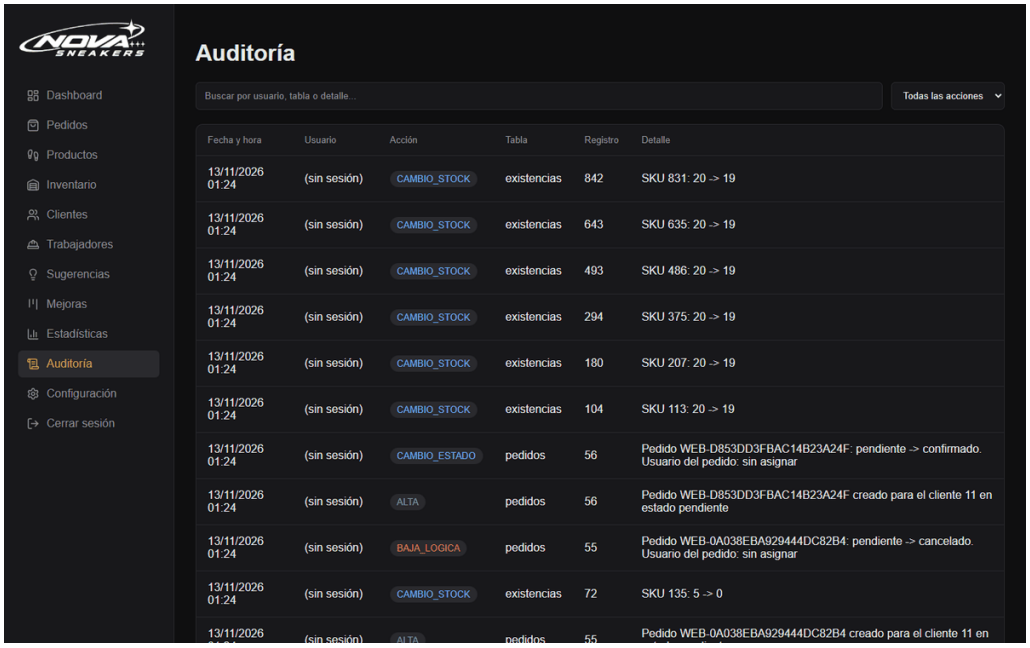


Figura 23. Auditoría del panel con los movimientos generados por compras de la tienda online.

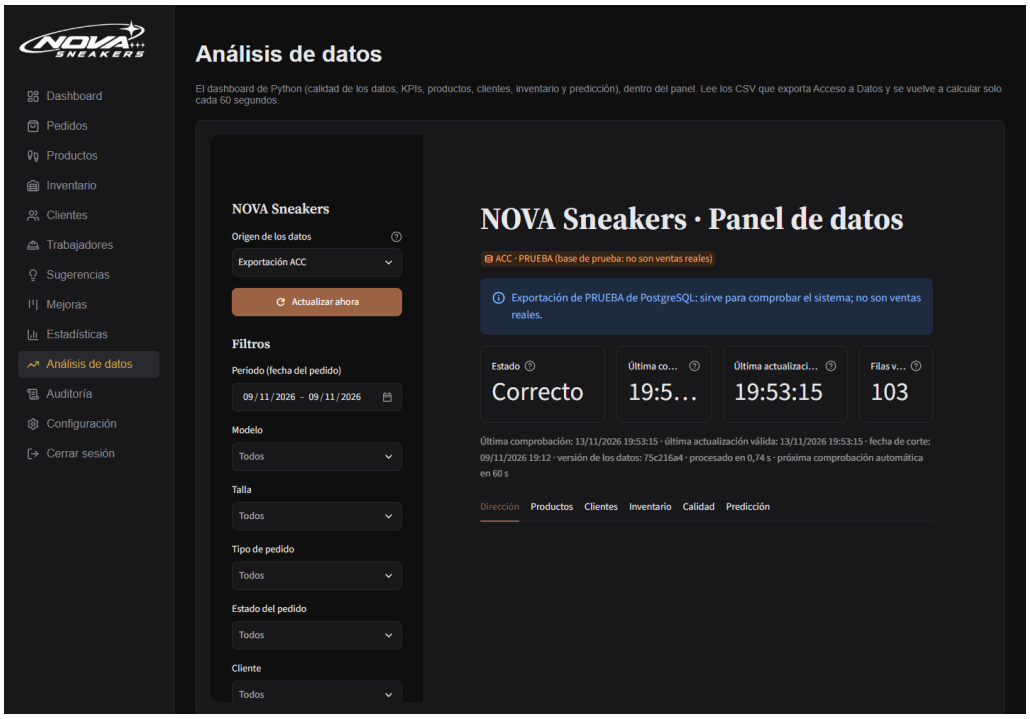


Figura 24. Página «Análisis de datos» con el panel de Streamlit integrado, sobre la exportación de prueba de PostgreSQL.

Funcionalidades del portal del trabajador

El portal del trabajador lo escribió Berme: sus páginas, su JavaScript y sus rutas en el servidor. Reutiliza la estructura visual del panel (barra lateral y tablas) y tiene cuatro páginas: un resumen con cuatro indicadores (pedidos por preparar, en preparación, referencias con stock bajo y pedidos del día; figura 25), la lista de pedidos, el inventario y los pedidos personalizados, que incluyen una tabla de configuraciones X-Nova de solo lectura (figura 28).



Figura 25. Resumen del portal del trabajador con sus cuatro indicadores.

En la página de pedidos, el botón «Avanzar estado» mueve un pedido un único paso en la secuencia confirmado → en preparación → enviado → entregado (figura 26). En el inventario (figura 27), cada fila tiene un campo numérico y un botón «Guardar» para ajustar las existencias, con valores enteros entre 0 y 99 999. Ambas operaciones las ejecutan dos funciones de PostgreSQL del script 23 que comprueban que el usuario es trabajador y dejan el cambio en la auditoría a su nombre.

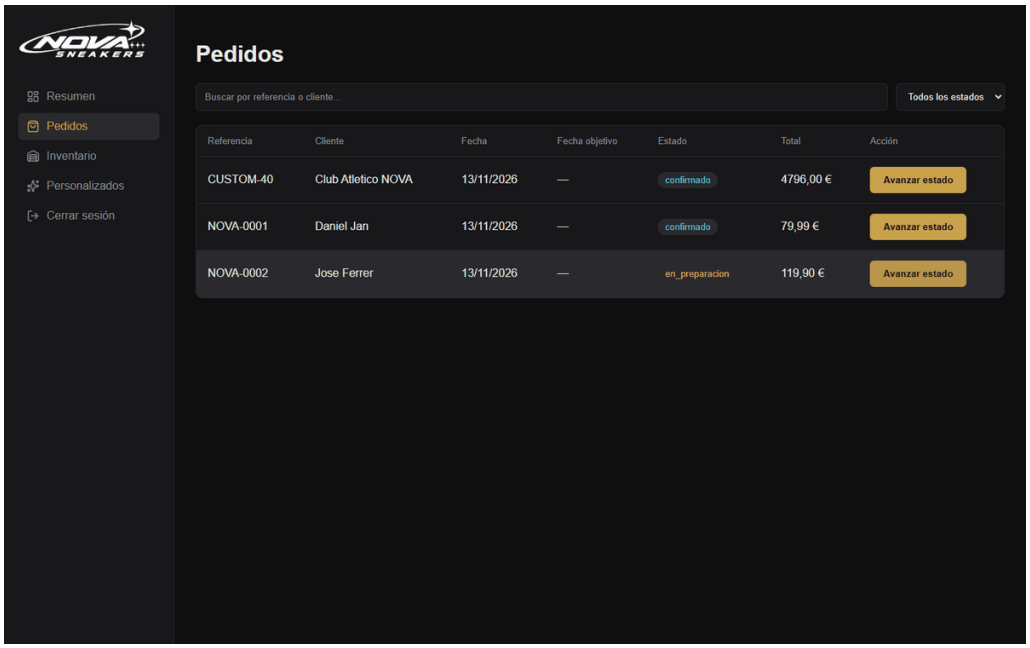


Figura 26. Pedidos del portal después de avanzar NOVA-0002, que pasa a «en_preparacion».

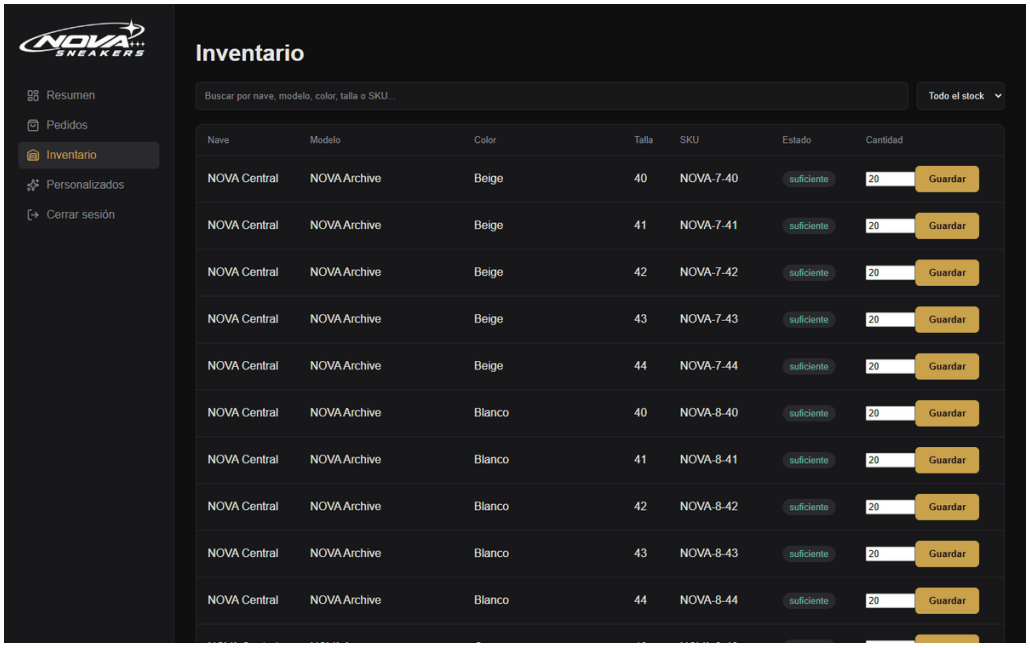


Figura 27. Inventario del portal: cantidad editable y botón «Guardar» en cada referencia.

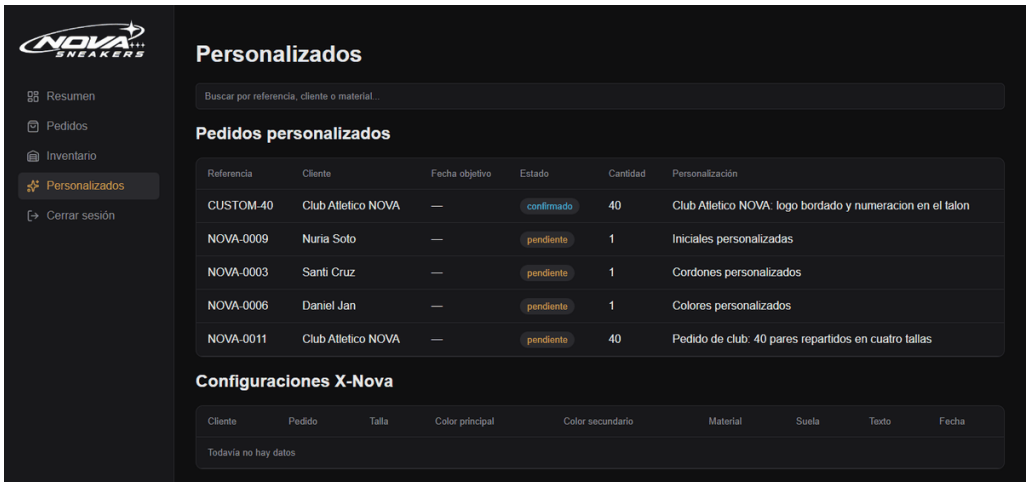


Figura 28. Pedidos personalizados y configuraciones X-Nova (esta última tabla, solo de lectura y sin datos).

En la primera versión de Berme, el portal no tenía inicio de sesión ni roles y modificaba la base con sentencias `UPDATE` directas. Al integrarlo, Omar lo colocó detrás del login único, con un rol mínimo de PostgreSQL (`portal_trabajador`) que solo puede cambiar datos mediante las dos funciones mencionadas. El portal considera stock bajo menos de 5 unidades y el panel, menos de 10; es una diferencia de criterio que sigue abierta.

3.1.3 Requisitos del proyecto

La tabla 11 recoge los requisitos de Interfaces del enunciado intermodular y su estado según la matriz de rúbricas del repositorio (apartado 2.1). De los catorce requisitos con estado, trece están cumplidos y probados; queda pendiente la prueba de usabilidad con personas.

Tabla 11. Requisitos de Interfaces y su cobertura.

Requisito	Cómo se cubre	Estado
Estructura mínima: inicio, catálogo, ficha, Custom y contacto	Secciones de `index.html` y menú con enlaces a cada una	Cumplido
Diseño adaptable a móvil y escritorio	Media queries a 760 y 370 px; probado en 4 anchos y en 11 anchos de 320 a 1440 px	Cumplido
Catálogo con nombre, imagen, precio orientativo, descripción y tallas	`modelos.js` (100 modelos) y ficha de producto	Cumplido
Custom: explicar el proceso y formulario o prototipo	Sección `#custom` con cuatro pasos y formulario simulado	Cumplido
Calidad: jerarquía visual, legibilidad, consistencia y navegación	axe sin violaciones; foco visible; contraste AA	Cumplido
RA4: estándares de usabilidad y accesibilidad	WCAG 2.1 AA con axe-core y Lighthouse	Cumplido
RA4: menús, controles y mensajes claros	Etiquetas, `fieldset`, `role="status"`/`"alert"` y errores junto al campo	Cumplido
RA4: pruebas de usabilidad con personas	Procedimiento preparado, sin realizar	Pendiente
RA8: estrategia, pruebas de integración y documentación	`ESTRATEGIA_PRUEBAS_INTERFACES.md`; web → servidor → PostgreSQL en Edge	Cumplido

3.1.4 Resultados del proyecto

Pruebas realizadas

Las pruebas de navegador se automatizaron con Playwright sobre Microsoft Edge, en escritorio y en móvil simulado, y cada estado de la página se auditó con axe-core. Para la tienda, el panel y el portal

se usó un servidor de prueba conectado a una copia desechable de la base, montada desde cero con los scripts, nunca a la base del equipo. La tabla 12 resume las baterías y sus resultados tal como constan en las evidencias.

Tabla 12. Baterías de pruebas de Interfaces y resultados registrados.

Batería	Resultado	Evidencia
Requisitos de Interfaces en 4 anchos (13/11)	136/136	`regresion-final/05_web/02_interfaces.txt`
Toda la web de clientes (13/11)	114/114	`regresion-final/05_web/03_web.txt`
Mejoras de la web: 100 modelos, colores y probador (13/11)	81/81	`regresion-final/05_web/01_mejoras.txt`
Compra completa en Edge con 4 pedidos pagados (13/11)	139/139	`regresion-final/05_web/04_compra_edge.txt`
Reparto y recibos (13/11)	29/29	`regresion-final/05_web/05_recorrido_reparto.txt`
Login único y portal del trabajador en Edge (13/11)	82/82	`regresion-final/05_web/06_edge_portal.txt`
Servidor: panel, tienda y portal (`node --test`, cierre)	23/23, 39/39 y 24/24	`cierre-entrega-20261113/node_*.txt`
Panel: navegador en escritorio y móvil (11/11)	124/124	`panel-admin/resultados/07_navegador_escritorio_y_movil.txt`
Análisis de datos con y sin Streamlit (13/11)	43/43 y 2/2	`panel-admin/analisis-datos/`
Lighthouse, accesibilidad y buenas prácticas (11/11)	100 y 100 en escritorio y en móvil	`web/lighthouse/`
axe-core, reglas WCAG 2.1 A y AA	0 violaciones en la web, la tienda y el login; en móvil, 2 avisos por tablas desplazables del panel y del portal	Resultados anteriores

La batería de requisitos, por ejemplo, comprueba en los anchos de 1440, 820, 390 y 360 px que no hay desplazamiento horizontal, que la cabecera cabe sin que sus enlaces se pisen, que las cien tarjetas muestran su precio, que las fichas de las seis colecciones tienen precio, descripción y tallas, y que el formulario Custom bloquea los envíos vacíos, los correos mal escritos y las cantidades negativas o con decimales, suma el total al escribir y, en un envío válido, muestra la simulación sin realizar ninguna petición de red. Los informes de Lighthouse de escritorio y móvil (figuras 29 y 30) puntuaron 100 en accesibilidad y en buenas prácticas.



Figura 29. Informe de Lighthouse 12.8.2 en escritorio: 100 en accesibilidad y en buenas prácticas.



Figura 30. Informe de Lighthouse 12.8.2 en móvil, con las mismas puntuaciones.

Las pruebas tienen tres límites. La **prueba de usabilidad con personas no se ha realizado**: existe un procedimiento preparado (`01_Interfaces/memoria/PROCEDIMIENTO_USABILIDAD.md`) para tres a cinco participantes, con seis tareas, métricas de eficacia, tiempo y errores y el cuestionario SUS, pero

sin fecha ni resultados. Los scripts de Playwright y axe no están en el repositorio, solo sus resultados y capturas. Y Lighthouse no se repitió después del 11/11, cuando todavía no existían los cien nombres comerciales ni los colores por colección; esos cambios sí se cubrieron con axe y con una medida de contraste posterior. Tampoco se ha probado con un lector de pantalla, en un móvil físico ni en otros navegadores.

Problemas encontrados y soluciones

- **Contraste de los colores por colección.** Las tarjetas con degradados de Álvaro dejaban el precio con un contraste de entre 1,0:1 y 1,9:1 y el foco en torno a 1,3:1, muy por debajo del mínimo AA de 4,5:1. Además, las herramientas automáticas no pueden medir texto sobre degradados. Al fusionar su rama se conservaron las tarjetas redondeadas y la sombra, y después se recuperaron los colores con correcciones: la etiqueta de colección en una pastilla oscura, una banda de datos opaca y un foco doble (claro y oscuro).
- **Conflictos al integrar ramas.** El diálogo de acceso y el rediseño de las tarjetas de Álvaro modificaban los mismos archivos que los precios añadidos por Omar, y la fusión dio cinco conflictos en la web de clientes. Se resolvieron a mano, conservando la autoría de cada parte.
- **Pie de página duplicado.** El contacto y el pie eran dos bloques oscuros seguidos que parecían dos pies distintos. Se unieron en un único `footer` con el identificador `#contacto`.
- **Probador con figuras ocultas.** En una versión anterior, las tres figuras genéricas estaban ocultas y desactivadas y el peso no tenía efecto. Se corrigió para que solo se oculten las vistas con fotografía propia cuando no corresponden al modelo y para que la figura se elija según el IMC.
- **Recortes de imagen defectuosos.** Al quitar el fondo blanco de las fotografías se borraban también las partes blancas de algunas zapatillas. Se sustituyeron por versiones `-sinfondo.webp` obtenidas rellenando el fondo desde los bordes.
- **Accesibilidad del panel y del portal.** axe detectó en el panel once elementos con tres tipos de problema (desplegables sin nombre, el tablero kanban sin acceso por teclado e interruptores sin etiqueta), y en el portal, 940 campos de inventario sin etiqueta. Se añadieron `aria-label`, `role="region"` y `tabindex="0"`, y el resultado quedó en cero violaciones en escritorio.

Quedan abiertos dos detalles menores: la etiqueta «Usuario o correo» del acceso, que solo admite el correo, y los precios y tallas provisionales del catálogo.

3.2 Programación de Servicios y Procesos (0490)

La parte de Programación de Servicios y Procesos (PSP) es una aplicación Java de consola que trabaja sobre una **copia exportada de un pedido Custom**. Un proceso coordinador lee esa copia y la hace pasar por tres procesos hijos: el primero valida los datos, el segundo agrupa los pares por variante y talla, y el tercero prepara una ficha auxiliar para el taller. Todo el intercambio de datos se hace por la entrada y la salida estándar de cada proceso.

El enunciado deja claro que este subsistema es auxiliar. Por eso la aplicación no escribe en PostgreSQL ni en Odoo, no modifica el stock y no crea órdenes de fabricación: solo lee un archivo CSV y genera dos archivos de texto en la carpeta `salidas/`. La propia ficha lo recuerda en su última línea.

3.2.1 Planificación y metodología

Planificación del trabajo

El trabajo se organizó en cuatro etapas, que se pueden seguir en el `README.md` de la carpeta `03_PSP` y en el historial del repositorio:

1. **Primera versión.** Omar escribió el validador del pedido (`ValidadorPedidoCustom`), el lector de CSV (`LectorCsv`) y un primer lanzador (`LanzadorValidacion`) con un analizador sencillo que solo agrupaba por talla. Esta versión ya usaba `ProcessBuilder` y el protocolo de respuestas `OK;`/`ERROR;`.
2. **Analizador de tallas.** Álvaro desarrolló en su rama el paquete `Psp`, con `AnalizadorTallas`, su lanzador (`LanzadorAnalizador`), la clase `ResultadoProceso` y sus propias pruebas (`PruebasAnalizador`). Su analizador agrupa por variante y por talla, que es lo que necesita la ficha del taller.
3. **Coordinador con tres hijos.** Se añadieron el proceso padre (`ProcesadorPedidosCustom`), la clase de apoyo que lanza cada hijo (`GestorProcesos`), el generador de la ficha (`GeneradorFichaTaller`) y la batería `PruebasPsp`. El validador se modificó para devolver los datos ya normalizados en el formato que leía el analizador de Álvaro, de modo que su clase se pudiera usar sin tocarla.
4. **Integración y revisión.** La rama de Álvaro se fusionó conservando su autoría. Berme aportó el comprobador rápido `scripts/comprobar_psp.py`, un CSV sintético con comas, comillas y saltos de línea dentro de un campo (`pruebas/custom40.csv`) y la descripción de los formatos de entrada (`06_Intermodular/SO024/contrato-fabricacion.md`). En el cierre se incorporó su corrección del comprobador y se repitió toda la batería sobre el árbol final.

Metodología de trabajo

Se partió de un **contrato de datos** antes que del código: el formato del CSV que exporta el script `15_exportar_fabricacion.sql` (ocho columnas en un orden fijo) y el formato de las respuestas entre procesos. Con el contrato fijado, cada hijo se pudo programar y probar por separado, lanzándolo a mano con una tubería de PowerShell, antes de conectarlo al padre.

La base técnica son los ejemplos trabajados en clase. Del mini reto `mini_reto_1_lanzador_estadisticas` se tomó el protocolo de respuesta (`OK;…` por la salida estándar y `ERROR;…` por la salida de error con `System.exit(2)`), la forma de construir el comando con `java.home` y el *classpath* del

propio programa y la costumbre de comprobar que la respuesta empieza por `OK;` antes de usarla. Del ejemplo `EG1_PipelineTexto` se tomó la lectura de la entrada estándar con un `BufferedReader`, y de `LanzadorProcesoFinal2.java`, la espera con límite y la cancelación con `destroyForcibly()`.

Las pruebas se escribieron para **lanzar los procesos de verdad**, no para simular sus respuestas. Cada prueba crea una carpeta temporal, prepara un CSV, ejecuta el coordinador o un hijo y comprueba el código de salida y el texto obtenido. Así se prueban también los errores que solo aparecen al comunicar procesos, como una clase que no se encuentra o un hijo que no termina.

Herramientas utilizadas

- **JDK 17.0.12**, sin Maven ni librerías externas. Se compila y se ejecuta con `javac` y `java` desde la carpeta `03_PSP`, usando la ruta de `JAVA_HOME` porque en el equipo de pruebas no están en el `PATH`.
- **PowerShell** en Windows 11, para lanzar el coordinador, probar cada hijo con tuberías (`Get-Content … | java …`) y comprobar los códigos de salida con `$LASTEXITCODE`.
- **PostgreSQL** y el script `15_exportar_fabricacion.sql`, de Acceso a Datos, como origen del CSV de entrada. PSP no se conecta a la base: solo lee el archivo exportado.
- **Git y GitHub**, con una rama por persona y fusiones que conservan la autoría de cada clase.

3.2.2 Descripción de la solución

Estructura de clases y procesos

Cada proceso es una clase con su propio método `main`. El coordinador no llama a los métodos de los hijos: los arranca como programas independientes y se comunica con ellos por sus flujos estándar. La tabla 13 resume el papel de cada clase.

Tabla 13. Clases de la aplicación PSP (carpeta `03_PSP`).

Clase	Tipo	Responsabilidad	Autoría
`ProcesadorPedidosCustom`	Padre	Comprueba el archivo, lanza los tres hijos uno tras otro, les pasa los datos, revisa sus códigos y respuestas, guarda la ficha y escribe el registro.	Omar
`GestorProcesos`	Apoyo del padre	Lanza un hijo con `ProcessBuilder`, le escribe por stdin, recoge stdout y stderr por separado y espera con un tiempo máximo.	Omar
`ValidadorPedidoCustom`	Hijo 1	Valida el CSV y devuelve los datos normalizados (`PEDIDO`, `LINEA`, `FIN`).	Omar
`LectorCsv`	Apoyo del hijo 1	Parte el CSV en registros y campos respetando las comillas.	Omar
`Psp.AnalizadorTallas`	Hijo 2	Agrupa los pares por modelo, color y talla y por talla. Se usa sin cambios.	Álvaro
`GeneradorFichaTaller`	Hijo 3	Comprueba que las cantidades cuadran y escribe la ficha del taller.	Omar
`pruebas/PruebasPsp`	Pruebas	Las 27 pruebas, que lanzan los programas reales.	Omar
`pruebas/HijoDePrueba`	Pruebas	Hijo que tarda, falla o devuelve un código raro a propósito.	Omar

En el reparto de tareas del equipo, la clase padre (PSP-01) y el generador de la ficha (PSP-04) estaban asignados a Berme, pero las subió Omar; el `README.md` de PSP lo deja escrito y deja pendiente de acordar quién defiende cada clase. Además de la versión actual se conserva la primera

(`LanzadorValidacion` y el `AnalizadorTallas` de la raíz), que sigue funcionando porque el validador mantiene la misma primera línea de respuesta.

Proceso principal y procesos hijos

El listado 1 reproduce el esquema de comunicación que acompaña al código. Los datos nunca pasan directamente de un hijo a otro: siempre vuelven al padre, que los revisa, los separa y prepara la entrada del paso siguiente.

```
                        CSV del pedido
                              |
                              v
ProcesadorPedidosCustom --stdin--> ValidadorPedidoCustom      (hijo 1)
                        <--stdout-- OK;... + datos normalizados
                        <--stderr-- ERROR;motivo
        |
        | el padre separa el bloque PEDIDO / LINEA / FIN
        v
ProcesadorPedidosCustom --stdin--> Psp.AnalizadorTallas       (hijo 2)
                        <--stdout-- OK;... + VARIANTE_TALLA / TALLA / TOTAL
        |
        | el padre añade cliente y personalización
        v
ProcesadorPedidosCustom --stdin--> GeneradorFichaTaller       (hijo 3)
                        <--stdout-- OK;... + la ficha
        |
        v
salidas/ficha_taller_CUSTOM-40.txt   y   salidas/registro_psp.txt
```

Listado 1. Esquema de comunicación entre el padre y los tres hijos (`03_PSP/README.md`).

El método `main` del padre sigue siempre el mismo orden. Primero comprueba el argumento y el archivo (que exista, que sea un archivo normal, que termine en `.csv`, que no esté vacío y que esté en UTF-8) y solo entonces lo lee entero con `Files.readString`. Lo lee de una vez porque un registro del CSV puede ocupar varias líneas cuando la personalización lleva un salto de línea entre comillas.

Después lanza el paso 1 con todo el CSV. De la respuesta del validador separa dos grupos de líneas: las de `PEDIDO`, `LINEA` y `FIN`, que son justo la entrada que espera el analizador de Álvaro, y las de `CLIENTE` y `PERSONALIZACION`, que guarda para la ficha. Con el primer grupo lanza el paso 2 y, con el resultado del analizador más los datos guardados, el paso 3. Tras cada paso llama a `comprobarPaso`, que decide si se puede seguir; si algo falla, el programa termina sin lanzar el paso siguiente.

Uso de ProcessBuilder

La clase `GestorProcesos` concentra todo lo relacionado con crear y vigilar un proceso. El listado 2 muestra cómo se construye el comando del hijo. En lugar de escribir `java` a secas, se usa el ejecutable de la misma máquina virtual que está corriendo (`java.home`) y el mismo *classpath* del padre (`java.class.path`). De este modo el hijo encuentra sus clases sin configurar nada y se evita lanzar, por error, otra versión de Java instalada en el equipo.

```
String java = Path.of(System.getProperty("java.home"), "bin", "java").toString();
String classpath = System.getProperty("java.class.path");

ProcessBuilder pb = new ProcessBuilder(java, "-cp", classpath, clase);

Process proceso;

try {
    proceso = pb.start();

} catch (IOException e) {
    // Ni siquiera ha arrancado: no hay salida ni codigo que recoger.
    errores.add("No se ha podido arrancar el proceso: " + e.getMessage());
    codigoSalida = NO_ARRANCA;
    milisegundos = System.currentTimeMillis() - inicio;
    return;
}
```

Listado 2. Construcción y arranque del proceso hijo (`GestorProcesos.java, líneas 33-48`).

Si `start()` lanza una `IOException`, el hijo no llega a existir y, por tanto, no tiene código de salida. Para distinguir ese caso de cualquier código real, `GestorProcesos` usa dos valores propios: **–1** (`NO_ARRANCA`) cuando el proceso no se ha podido crear y **–2** (`TIEMPO_AGOTADO`) cuando se ha cancelado por tardar demasiado.

Comunicación mediante stdin y stdout

Cada proceso hijo tiene tres tuberías: su entrada estándar, por la que el padre le envía los datos; su salida estándar, por la que contesta, y su salida de error, por la que explica los fallos. Estas tuberías tienen un tamaño limitado. Si el padre escribiera todo el CSV y después se pusiera a leer, podría quedarse bloqueado escribiendo mientras el hijo, a su vez, está bloqueado porque nadie vacía su salida. Es un interbloqueo clásico entre procesos.

Para evitarlo, `GestorProcesos` dedica **un hilo a cada tubería** (listado 3). El hilo escritor envía los datos y cierra la entrada del hijo; cerrarla es la señal de que no llegarán más datos, y es lo que permite al hijo terminar su bucle de lectura. Los dos hilos lectores vacían la salida y la salida de error en dos listas distintas. El `README.md` explica el origen de esta decisión: con la primera versión, sin hilos, el programa se colgaba al enviar un CSV de 6 MB o con un hijo que escribía mucho.

```
Thread escritor = new Thread(() -> {
    try (BufferedWriter entradaDelHijo = proceso.outputWriter(StandardCharsets.UTF_8)) {
        entradaDelHijo.write(entrada);
    } catch (IOException e) {
        errorDeEscritura = e.getMessage();
    }
});

Thread lectorSalida = new Thread(() -> leerTodo(
        proceso.inputReader(StandardCharsets.UTF_8), salida));

Thread lectorError = new Thread(() -> leerTodo(
        proceso.errorReader(StandardCharsets.UTF_8), errores));

escritor.setDaemon(true);
lectorSalida.setDaemon(true);
lectorError.setDaemon(true);

escritor.start();
lectorSalida.start();
lectorError.start();
```

Listado 3. Un hilo por tubería (`GestorProcesos.java, líneas 50-71`).

No hace falta sincronizar las listas porque cada hilo escribe solo en la suya y el hilo principal no las lee hasta después de esperar al proceso y hacer `join()` sobre los tres hilos. Los hilos se marcan como demonios para que, si alguno quedara bloqueado, no impida que el programa termine. Todos los flujos se abren con `StandardCharsets.UTF_8`, y los hijos fijan también UTF-8 en su `System.out` y `System.err`: así un nombre de cliente o una personalización con tildes llega igual al otro lado.

La espera se hace en el hilo principal con `waitFor(segundosMaximos, TimeUnit.SECONDS)`, con un máximo de 10 segundos por hijo. Si el hijo termina a tiempo se guarda `exitValue()`; si no, se le pide que termine con `destroy()` y, si al cabo de un segundo sigue vivo, se fuerza con `destroyForcibly()`. El orden es importante: primero se espera con límite y después se recoge lo leído. De este modo el padre recupera el control pase lo que pase dentro del hijo.

Entradas y salidas

La entrada es el CSV que genera el script 15 de Acceso a Datos, en UTF-8, separado por comas y con una cabecera fija de ocho columnas: `referencia, sku, cliente, modelo, color, personalizacion, talla, cantidad`. El archivo de demostración es `03_PSP/datos/custom40_fabricacion_psp.csv`, una copia sin cambios de la exportación del pedido CUSTOM-40 sobre la base de prueba (listado 4).

```
referencia,sku,cliente,modelo,color,personalizacion,talla,cantidad
CUSTOM-40,NOVA-4-40,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y numeracion en el
talon,40,10
CUSTOM-40,NOVA-4-41,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y numeracion en el
talon,41,10
CUSTOM-40,NOVA-4-42,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y numeracion en el
talon,42,10
CUSTOM-40,NOVA-4-43,Club Atletico NOVA,NOVA Velocity,Rojo,Club Atletico NOVA: logo bordado y numeracion en el
talon,43,10
```

Listado 4. CSV de entrada del pedido CUSTOM-40 (`03_PSP/datos/custom40_fabricacion_psp.csv`).

CUSTOM-40 es un pedido de los datos de prueba de PostgreSQL: Club Atletico NOVA, modelo NOVA Velocity en rojo, tallas 40 a 43 con 10 pares cada una. Su cliente se llama igual que el del pedido S00005 documentado en Odoo (apartado 3.3), y los dos suman 40 pares, pero **no son el mismo pedido**: no existe ninguna exportación de S00005 que haya pasado por PSP, y coincidir en el nombre o en la cantidad no acredita la equivalencia.

La aplicación produce dos salidas. La ficha se guarda en `salidas/ficha_taller_<referencia>.txt`; antes, el método `nombreParaArchivo` sustituye por un guion bajo cualquier carácter de la referencia que no sea letra, número, guion o guion bajo, para que el nombre sea válido en cualquier sistema. El registro `salidas/registro_psp.txt` recibe un bloque por ejecución, siempre al final del archivo (`StandardOpenOption.APPEND`), con la hora, el archivo de origen, el código y la duración de cada paso y el resultado. La referencia del pedido aparece en el CSV, en cada respuesta `OK;`, en el nombre de la ficha, dentro de ella y en el registro, lo que permite seguir el pedido de principio a fin.

Validación de pedidos

`ValidadorPedidoCustom` lee toda su entrada con `System.in.readAllBytes()` y se la entrega a `LectorCsv`, que recorre el texto carácter a carácter. Dentro de un campo entre comillas, la coma y el salto de línea son texto normal, y dos comillas seguidas representan una comilla literal. Si encuentra una comilla sin cerrar o texto pegado detrás de una comilla de cierre, lanza una `IllegalArgumentException` que el validador convierte en un rechazo.

Sobre los registros ya separados, el validador aplica ocho comprobaciones numeradas en el propio código: que llegue algo; que la cabecera sea exactamente la esperada; que haya al menos un registro además de la cabecera; que cada registro tenga ocho campos; que no falte la referencia, el SKU, el cliente, el modelo, el color, la talla ni la cantidad; que todas las filas sean del mismo pedido y del mismo cliente; que la talla sea un entero positivo, y que la cantidad sea un entero mayor que cero. Además, los textos que se van a reenviar no pueden contener `;` ni saltos de línea, porque romperían el formato de líneas que se usa entre procesos.

Cuando todo es correcto, el validador no se limita a decir que sí: devuelve los datos normalizados (listado 5). La primera línea, `OK;referencia;líneas;pares`, es la misma que leía la primera versión del proyecto, y el bloque `PEDIDO … FIN` es exactamente la entrada del analizador de Álvaro. Si la personalización ocupa varias líneas, se emite una línea `PERSONALIZACION` por cada una para no perder texto. Cuando encuentra un error, no escribe nada en la salida estándar: escribe `ERROR;motivo` en la salida de error y termina con código 2.

```
OK;CUSTOM-40;4;40
CLIENTE;Club Atletico NOVA
PERSONALIZACION;Club Atletico NOVA: logo bordado y numeracion en el talon
PEDIDO;CUSTOM-40
LINEA;NOVA-4-40;NOVA Velocity;Rojo;40;10
LINEA;NOVA-4-41;NOVA Velocity;Rojo;41;10
LINEA;NOVA-4-42;NOVA Velocity;Rojo;42;10
LINEA;NOVA-4-43;NOVA Velocity;Rojo;43;10
FIN
```

Listado 5. Respuesta del validador para CUSTOM-40 (ejecución del 13/11/2026).

Análisis de tallas

El segundo hijo es la clase `Psp.AnalizadorTallas` de Álvaro, que se usa tal como la escribió. Lee la primera línea, que tiene que ser `PEDIDO;referencia`, y después cada `LINEA;sku;modelo;color;talla;cantidad` hasta encontrar `FIN`. Rechaza con código 2 las líneas de tipo desconocido, las que no tienen seis campos, los textos vacíos, los números mal escritos, las cantidades que no son mayores que cero y las tallas fuera del rango 20-55; también rechaza un pedido sin `FIN` o sin líneas.

La agrupación se apoya en dos mapas. Para las combinaciones de modelo, color y talla usa un `LinkedHashMap`, que conserva el orden de llegada; para las tallas, un `TreeMap`, que las devuelve ordenadas. En ambos acumula con `merge(clave, cantidad, Integer::sum)`, de forma que dos líneas iguales se suman en una sola. La respuesta tiene tres tipos de líneas: `VARIANTE_TALLA;modelo;color;talla;pares`, `TALLA;talla;pares` y `TOTAL;pares`.

Al recibir esa respuesta, el padre cuenta las combinaciones y las tallas y comprueba que el `TOTAL` del analizador coincide con los pares que contó el validador. Si no coinciden, termina con código 3 y el mensaje «Los procesos no coinciden». Es una comprobación sencilla, pero detecta cualquier pérdida de datos al pasar de un proceso a otro.

Generación de la ficha

`GeneradorFichaTaller` recibe del padre `PEDIDO`, `ORIGEN` (el nombre del CSV), `CLIENTE`, las líneas de `PERSONALIZACION`, la respuesta del analizador y `FIN`. Antes de escribir nada comprueba que haya cliente, que existan filas por variante y por talla, que haya llegado el `TOTAL` y, sobre todo, que **las tres cuentas coincidan**: la suma por variante, la suma por talla y el total. Si alguna difiere, rechaza los datos con código 2 indicando las tres cifras.

Si todo cuadra, contesta `OK;referencia` seguido de la ficha. Las columnas se alinean con `String.format` (por ejemplo, `%-20s` deja el modelo a la izquierda en veinte posiciones y `%6d` coloca los pares a la derecha en seis), y el documento termina con la aclaración de que es un documento auxiliar que no sustituye a la orden de fabricación de Odoo. El padre descarta la línea `OK;`, guarda el resto en el archivo de la ficha y la muestra también por pantalla.

Códigos de retorno y gestión de errores

Los códigos de salida siguen un convenio común, compartido con el analizador de Álvaro, que se resume en la tabla 14. El padre traduce cada situación a uno de cuatro códigos para que quien lo ejecute (una persona o un script) sepa qué ha pasado sin leer los mensajes.

Tabla 14. Códigos de salida del padre y de los hijos.

Código	Proceso padre (`ProcesadorPedidosCustom`)	Procesos hijos
0	Correcto: ficha generada.	Correcto.
1	Problema con un archivo: el de entrada (no se lanza ningún hijo) o al guardar.	Java no encuentra la clase o hay una excepción sin capturar.
2	Un hijo ha rechazado los datos del pedido.	Datos incorrectos: `ERROR;motivo` por stderr.
3	Un hijo ha fallado: no arranca, código inesperado, tiempo agotado, respuesta no válida o cuentas que no cuadran.	Error inesperado (solo el analizador de Álvaro).
−1 / −2	Valores internos de `GestorProcesos`: el hijo no arrancó / se canceló por tiempo.	—

Toda la lógica de decisión está en `comprobarPaso` (listado 6). El orden de las preguntas es deliberado: primero los casos en los que no hay código real (tiempo agotado y proceso que no arranca), después el rechazo de datos, después cualquier otro código distinto de cero y, por último, la forma de la respuesta. Que el hijo termine con 0 no basta: la primera línea tiene que empezar por `OK;` y, a partir del paso 2, hablar del mismo pedido que descubrió el validador.

```
int codigo = paso.getCodigoSalida();

if (codigo == GestorProcesos.TIEMPO_AGOTADO) {
    terminar(FALLO_PROCESO, "El paso " + numero + " ha superado el tiempo de espera");
}
if (codigo == GestorProcesos.NO_ARRANCA) {
    terminar(FALLO_PROCESO, "No se ha podido lanzar el paso " + numero);
}
// El hijo ha rechazado los datos: su motivo viene por stderr.
if (codigo == HIJO_RECHAZA_DATOS) {
    terminar(DATOS_RECHAZADOS, "El paso " + numero + " ha rechazado los datos del pedido");
}
// Cualquier otro codigo distinto de 0 es un fallo del hijo.
if (codigo != 0) { ... terminar(FALLO_PROCESO, mensaje); }

String respuesta = primeraLinea(paso.getSalida());
String[] partes = respuesta.split(";");
boolean valida = respuesta.startsWith("OK;") && partes.length >= 2;
if (valida && referencia != null && !partes[1].equals(referencia)) {
    valida = false;
}
if (!valida) {
    terminar(FALLO_PROCESO, "La respuesta del paso " + numero + " no es la esperada");
}
```

Listado 6. Comprobación de cada paso (extracto de `ProcesadorPedidosCustom.java, líneas 203-253`).

El método `terminar` es el único punto de salida: añade una línea final `Resultado: …` con el estado y el código, escribe el bloque en el registro y llama a `System.exit(codigo)`. Si la clase de un hijo no

está compilada, el mensaje lo dice expresamente («No se encuentra la clase … hay que compilarla»), lo que ahorra tiempo al preparar la demostración.

3.2.3 Requisitos del proyecto

La tabla 15 relaciona los requisitos de PSP del enunciado intermodular con la parte del código que los cubre y con su estado, tal como figura en la matriz de rúbricas del repositorio (`06_Intermodular/rubricas/MATRIZ_RUBRICAS.md`, apartado 2.3). De los 19 requisitos con estado, 17 están cumplidos y probados y 2 quedan parciales.

Tabla 15. Requisitos de PSP y su cobertura.

Requisito	Cómo se cubre	Estado
Recibir la exportación de un pedido Custom y coordinar varios procesos	`ProcesadorPedidosCustom` con tres hijos; CUSTOM-40 termina con código 0	Cumplido
No sustituir a Ventas ni a Fabricación de Odoo	No escribe en ninguna base; la ficha lo indica expresamente	Cumplido
Proceso coordinador que lanza los hijos con `ProcessBuilder`	`GestorProcesos.ejecutar`	Cumplido
Procesos especializados (validar, resumir por talla, ficha)	Validador, analizador de Álvaro y generador	Cumplido
Comunicación por stdin/stdout	Tres hilos por hijo; pruebas 06a y 06b	Cumplido
Lectura separada de stdout y stderr	Dos listas independientes; prueba 07	Cumplido
`waitFor` y tratamiento de los códigos de salida	Códigos 0-3 y −1/−2; pruebas 08a y 08b	Cumplido
Procesos bloqueados o que tardan demasiado	`waitFor` con límite y `destroyForcibly`; prueba 10	Cumplido
Separación de responsabilidades y código legible	Un gestor y tres hijos probados por separado	Cumplido
Salida comprensible relacionada con el pedido original de Odoo	Ficha y registro con la referencia del pedido exportado	Parcial
Registro que permita relacionar el pedido con su referencia de Odoo	`registro_psp.txt`; falta una exportación real desde Odoo	Parcial

Los dos requisitos parciales tienen la misma causa: la ficha y el registro conservan la referencia del pedido que reciben, pero el CSV procede de PostgreSQL y no hay ninguna exportación de Odoo que haya pasado por PSP. La relación con el número de venta de Odoo, por tanto, no se puede demostrar con las evidencias actuales.

3.2.4 Resultados del proyecto

Las ejecuciones que se muestran en este apartado se repitieron el 13 de noviembre de 2026 sobre una copia del commit 372f811, compilada con el JDK 17.0.12 en Windows 11, y coinciden con las evidencias guardadas en `03_PSP/evidencias/`. Las capturas son de una consola real; en ellas el indicador de PowerShell se ha simplificado a `PS NOVA\03_PSP>` para no mostrar rutas personales.

Procesamiento completo de CUSTOM-40

La figura 31 muestra una ejecución correcta. Cada paso indica la clase lanzada, su código de salida y su duración, seguidos de la primera línea de su respuesta. El validador cuenta 4 líneas y 40 pares; el analizador encuentra 4 combinaciones y 4 tallas que suman los mismos 40 pares, y el generador devuelve la ficha. El programa termina con «Resultado: CORRECTO (codigo 0)», y `$LASTEXITCODE` confirma el 0 en PowerShell.

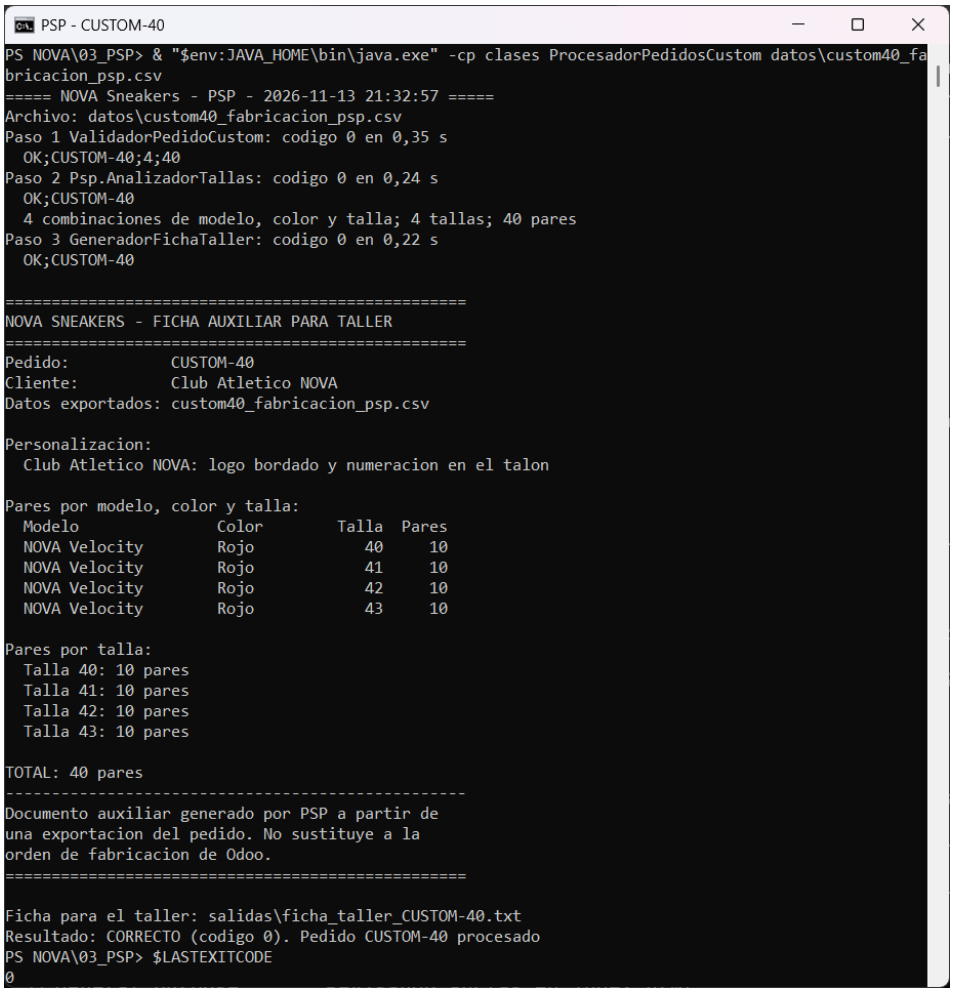


Figura 31. Ejecución del proceso padre con el pedido CUSTOM-40: los tres pasos terminan con código 0, las cuentas coinciden (40 pares) y se genera la ficha del taller.

Errores controlados

En la figura 32 se ven dos fallos provocados. Con `prueba_cantidad_cero.csv`, el validador rechaza el registro 2 porque su cantidad es 0; el padre muestra el motivo recibido por stderr, no lanza los pasos 2 y 3 y termina con código 2. Con un archivo que no existe, el padre detecta el problema antes de crear ningún proceso y termina con código 1. Las evidencias del repositorio incluyen un tercer caso: el mismo pedido correcto sin el analizador de Álvaro en el *classpath*, en el que el paso 2 falla y el padre termina con código 3 avisando de que hay que compilar la clase.

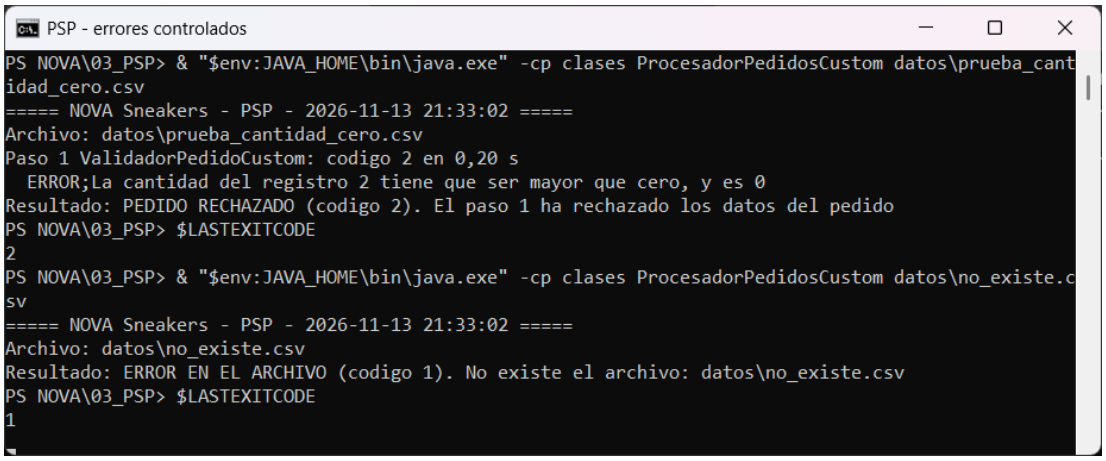


Figura 32. Dos errores controlados: una cantidad 0 rechazada por el hijo 1 (código 2) y un archivo inexistente detectado por el padre (código 1).

Un hijo ejecutado por separado

Como cada hijo es un programa independiente, se puede probar con una tubería de PowerShell sin pasar por el padre. La figura 33 muestra el validador recibiendo el CSV por su entrada estándar y devolviendo el bloque normalizado del listado 5, con código 0. Esta forma de trabajar fue la que se usó para desarrollar cada hijo antes de integrarlo.

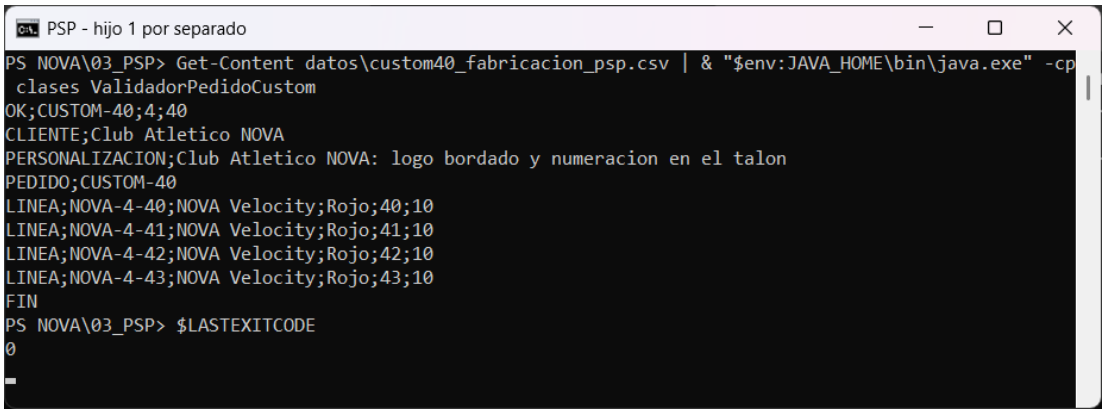


Figura 33. El validador lanzado por separado con una tubería: lee el CSV por stdin y escribe los datos normalizados por stdout.

Batería de 27 pruebas

`PruebasPsp` organiza sus 27 pruebas según los doce puntos que pide el enunciado (tabla 16). Trabaja en una carpeta temporal nueva, por lo que no modifica el repositorio, y termina con código 1 si alguna prueba falla. En la ejecución del 13/11/2026 superó las 27 (figuras 34 y 35), el mismo resultado que figura en `03_PSP/evidencias/04_resultado_pruebas.txt` y en la regresión final del repositorio.

Tabla 16. Correspondencia entre los puntos de prueba del enunciado y las pruebas de `PruebasPsp`.

Punto pedido	Pruebas	Qué se comprueba
1. Pedido Custom válido	01	CUSTOM-40 de principio a fin con código 0.
2. Varias tallas y cantidades	02	Nueve tallas (38 a 46), dos colores y una talla repetida: la 42 suma 6 + 2 + 4 = 12; total 40.
3. Validación correcta	03	El validador, solo, normaliza las 4 líneas: `OK;CUSTOM-40;4;40`.
4. Agrupación de tallas	04	Validador → analizador: Rojo 42 = 6 + 2 = 8; talla 42 = 12.
5. Ficha auxiliar	05	Ficha de 29 líneas y registro del procesamiento.
6. Comunicación entre procesos	06a, 06b	Eco por stdin/stdout; personalización de dos líneas, con coma y comillas, que cruza los tres procesos.

Punto pedido	Pruebas	Qué se comprueba
7. stdout y stderr por separado	07	Cada flujo llega a su lista.
8. Códigos de salida	08a, 08b	Se recoge el código exacto (7); el validador rechaza con 2 sin escribir en stdout.
9. Fallo controlado de un proceso	09a, 09b	Un hijo que lanza una excepción y una clase que no existe: código 1 detectado.
10. Tiempo de espera agotado	10	Un hijo que duerme 60 s se cancela a los 2 s (código −2).
11. Archivo inexistente	11	El padre avisa con código 1 y no lanza ningún hijo.
12. Datos mal formados	12a-12m	Comilla sin cerrar, cabecera distinta, cantidad no numérica, cero o negativa, falta la talla o el modelo, solo cabecera, campos de menos, dos pedidos, talla 99 (la rechaza el paso 2), archivo vacío y formato `.json`.

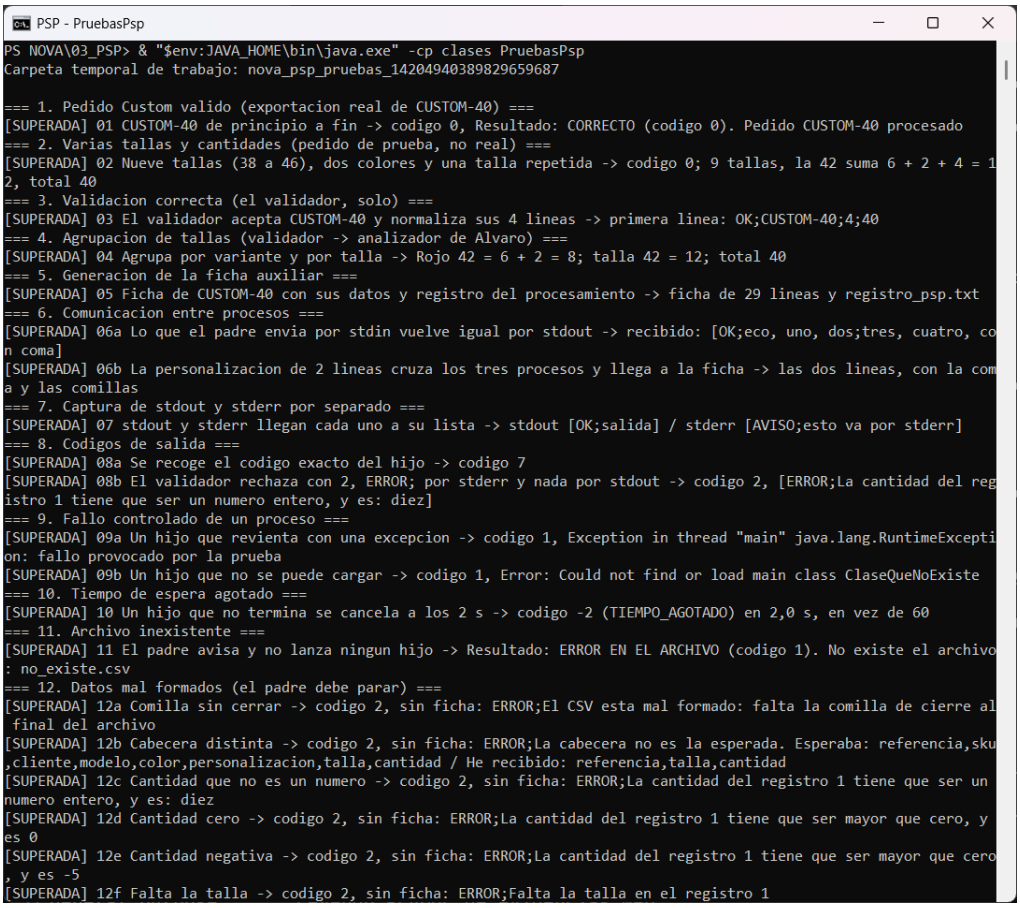


Figura 34. Ejecución de `PruebasPsp` (primera parte): puntos 1 a 11 de la batería.

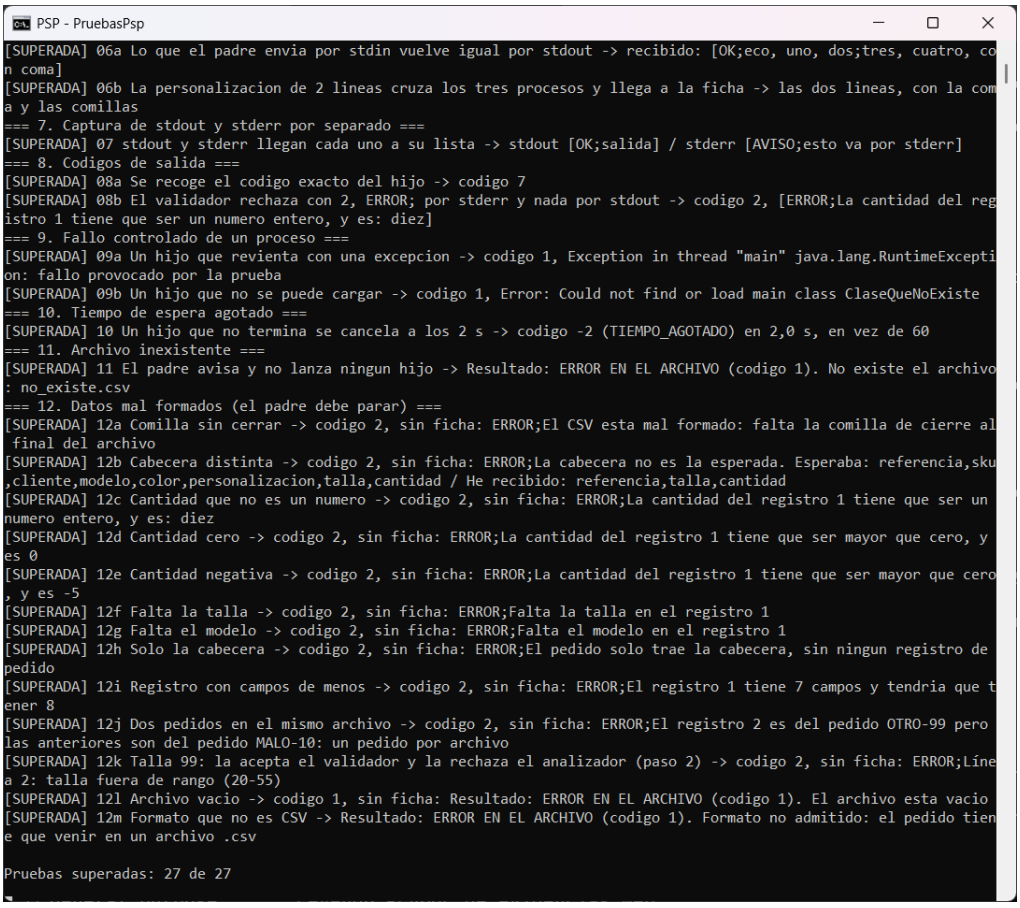


Figura 35. Ejecución de `PruebasPsp` (segunda parte): datos mal formados (12a-12m) y resultado final, 27 de 27 pruebas superadas.

Las ocho comprobaciones del analizador

El analizador de Álvaro tiene su propia prueba, `Psp.PruebasAnalizador`, que se mantiene sin cambios. Usa su lanzador (`LanzadorAnalizador`) y hace ocho comprobaciones: un pedido correcto que debe aceptarse (con una variante repetida que tiene que sumar 3 pares en la talla 42), seis pedidos incorrectos que deben rechazarse con código 2 (datos incompletos, cantidad cero, cantidad no numérica, primera línea incorrecta, falta de `FIN` y talla fuera de rango) y un hijo que no arranca porque la clase no existe. La figura 36 muestra el resultado: 8 de 8.

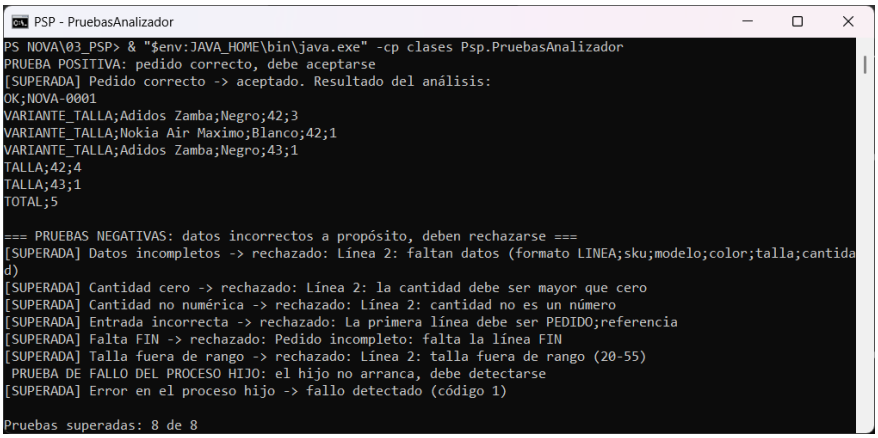


Figura 36. Prueba propia del analizador de tallas de Álvaro: ocho comprobaciones superadas.

Ensayo con un pedido de nueve tallas

El repositorio conserva también un ensayo rotulado **SO024 PROVISIONAL** (`03_PSP/evidencias/so024_provisional/`). Se hizo con datos inventados (cliente Club Halcones, modelo Runner X, color «PROVISIONAL» y un reparto de 40 pares entre las tallas 38 y 46) para comprobar la cadena PostgreSQL → script 15 → PSP con un pedido de nueve tallas. PSP lo procesó con código 0 (`OK;SO024;9;40`). Se cita solo como prueba de la cadena técnica: no es el pedido SO024 del enunciado, que es un ejemplo, ni tiene relación con la venta S00005 de Odoo.

Problemas encontrados y soluciones

- **Bloqueo al comunicar procesos.** La primera versión leía las salidas del hijo después de escribirle; con un CSV de 6 MB o con un hijo que escribía mucho, el programa se quedaba colgado. Se resolvió con un hilo por tubería y la espera con límite en el hilo principal.
- **Tiempo de espera que no actuaba.** Al revisar el `LanzadorAnalizador` de Álvaro se midió que, con un hijo que tarda 20 segundos, esperaba 20,1 s y devolvía código 0, porque lee la salida con `readAllBytes()` (que espera a que el hijo termine) antes de llamar a `waitFor`. El coordinador usa `GestorProcesos` en su lugar, y el problema se dejó documentado como propuesta para que lo revise Álvaro, sin modificar su archivo.
- **Caracteres con tilde.** Los textos se deformaban al pasar de un proceso a otro con la codificación por defecto de Windows. Se fijó UTF-8 en los tres flujos de cada hijo y en la lectura del archivo. Las pruebas deben lanzarse sin `-Dfile.encoding=UTF-8`, porque con esa opción comparan los textos en otro juego de caracteres y fallan.
- **Campos con comas, comillas y saltos de línea.** Una personalización puede contener cualquiera de los tres. Partir las líneas con `split(",")` no servía, así que `LectorCsv` recorre el texto carácter a carácter y el validador reparte la personalización en varias líneas `PERSONALIZACION`.
- **Comprobador desactualizado.** El comprobador rápido de Berme (`scripts/comprobar_psp.py`) esperaba la respuesta antigua del validador y falló cuando este pasó a devolver datos normalizados. En el cierre se incorporó su propia corrección, conservando su autoría, y el comprobador volvió a terminar correctamente.

Como límites conocidos quedan los que recoge el propio `README.md`: solo se admite CSV con coma como separador, se procesa un pedido por archivo (procesar varios a la vez sería la ampliación que propone el enunciado) y el padre guarda en memoria toda la salida de cada hijo, sin un tope.

3.3 Sistemas de Gestión Empresarial (0491)

En el planteamiento del enunciado, Odoo es el núcleo empresarial de la fábrica: en él se registran los clientes, las ventas, la fabricación, el inventario y la facturación, y los demás módulos trabajan como apoyo. Esta parte la desarrolló Berme (Iván Bermejo) en su propio equipo y la documentó en `odoo/documentacion/Odoo-NOVA-Sneakers.pdf`, un documento de 19 páginas cuya portada firma junto con Omar y Álvaro. El documento recoge las tareas OD-01 a OD-06 y una prueba de conexión entre Odoo y el servidor del proyecto.

Este apartado resume y explica ese trabajo con sus capturas originales, que se identifican con el número que tienen en el documento de Berme. El PDF se conserva sin modificar en el repositorio. Odoo se ejecuta en una máquina virtual del ordenador de Berme, y en ese entorno se hicieron tanto el recorrido del pedido S00005 como la prueba de **conexión entre Odoo y el servidor del proyecto**, que funcionó. Desde los demás equipos la instancia no es accesible: la evidencia que se conserva en el repositorio es el documento con sus capturas, y la demostración en directo requiere el ordenador de Berme.

3.3.1 Planificación y metodología

Planificación del trabajo

El trabajo se dividió en las seis tareas del reparto, más una séptima, la conexión con el servidor, que el propio documento presenta como una mejora no exigida. La tabla 17 muestra cada tarea y el apartado del documento en que se documenta.

Tabla 17. Tareas de Odoo y su documentación en el PDF de Berme.

Tarea	Contenido	Dónde se documenta
OD-01	Instalación y configuración de Odoo	Apartados 1 a 3 (págs. 3-6)
OD-02	Variantes por talla, referencias internas y usuarios	Apartado 8 (págs. 16-17)
OD-03	Venta del pedido Custom	Apartado 5 (pág. 10)
OD-04	Producto, lista de materiales y fabricación	Apartados 4 y 5 (págs. 7-11)
OD-05	Entrega y stock	Apartados 5 y 8 (págs. 11 y 17)
OD-06	Factura y pago	Apartado 5 (pág. 12)
—	Conexión del servidor con Odoo (API JSON-2)	Apartados 6 y 7 (págs. 13-15)

Metodología de trabajo

El orden de trabajo sigue el del propio ERP: primero se prepara el entorno y se configura la empresa, después se definen los productos y su forma de fabricarlos y, por último, se recorre el flujo comercial completo, desde el presupuesto hasta la factura. Cada paso se documentó con una captura y una explicación de qué demuestra. Las contraseñas (la maestra de la base de datos y la del administrador) y la clave de la API no aparecen en el documento ni en el repositorio.

Herramientas utilizadas

- **OrbStack** en un Mac, para crear una máquina Linux con un dominio local propio (`ubuntu.orb.local`), accesible desde el navegador del Mac sin configurar la red.
- **Ubuntu 24.04.5 LTS** (arquitectura arm64) como sistema operativo de la máquina virtual.

- **Odoo 19.0 Community**, instalado desde el paquete `.deb` oficial del repositorio *nightly*, y **PostgreSQL** del propio Ubuntu como gestor de la base de datos de Odoo.
- Las aplicaciones **Contactos, Ventas, Inventario, Fabricación y Facturación** de Odoo.
- Para la prueba de conexión, un script de Python con XML-RPC y el servidor Node del proyecto con la API JSON-2 de Odoo.

3.3.2 Descripción de la solución

Entorno: máquina Ubuntu en OrbStack

Odoo se ejecuta en una máquina llamada `ubuntu` creada con OrbStack (figura 37). Para crearla basta con instalar OrbStack, elegir en *Linux* → *Machines* una Ubuntu 24.04 y darle ese nombre; después se entra en ella desde la terminal del Mac con `orb -m ubuntu`. El acceso a Odoo queda en `http://ubuntu.orb.local:8069`.

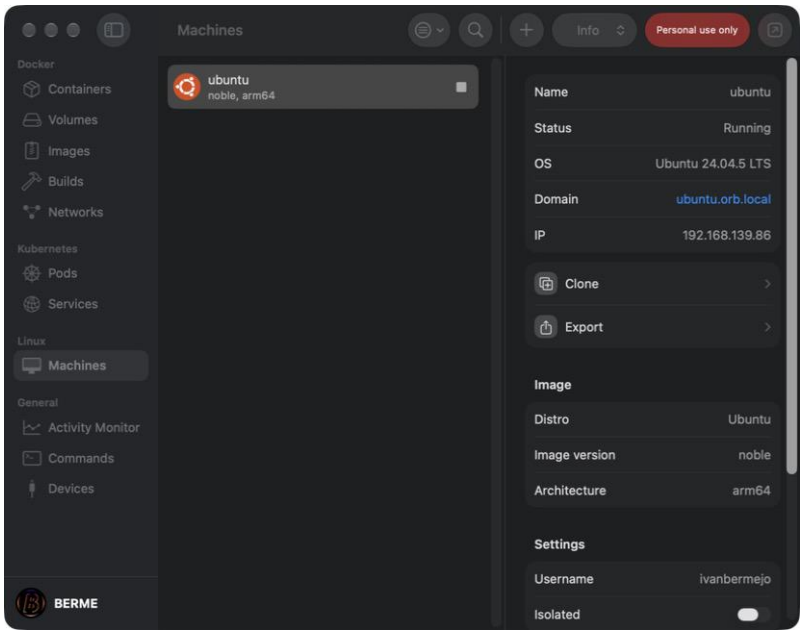


Figura 37. Máquina `ubuntu` en OrbStack, en ejecución: Ubuntu 24.04 LTS, arm64 y dominio `ubuntu.orb.local` (captura 1 del documento de Berme).

Instalación de Odoo 19 (OD-01)

La instalación usa el paquete oficial de Odoo 19, con lo que Odoo queda registrado como un servicio del sistema y se actualiza con `apt` como cualquier otro programa. El listado 7 reúne los comandos documentados por Berme: instalar PostgreSQL, añadir la clave y el repositorio de Odoo, instalar el paquete y comprobar el servicio.

```
# Paso 1 · PostgreSQL
sudo apt update
sudo apt install -y postgresql

# Paso 2 · Repositorio de Odoo 19
wget -q -O - https://nightly.odoo.com/odoo.key | sudo gpg --dearmor -o \
    /usr/share/keyrings/odoo-archive-keyring.gpg
echo 'deb [signed-by=/usr/share/keyrings/odoo-archive-keyring.gpg] \
    https://nightly.odoo.com/19.0/nightly/deb/ ./' | sudo tee /etc/apt/sources.list.d/odoo.list
sudo apt update

# Paso 3 · Instalar Odoo
sudo apt install -y odoo

# Paso 4 · Comprobar el servicio
sudo systemctl status odoo --no-pager
```

Listado 7. Instalación de Odoo 19 en Ubuntu (págs. 4-5 del documento de Berme).

La clave GPG permite a `apt` comprobar que los paquetes descargados son los publicados por Odoo. En las capturas, Odoo ya estaba instalado en la versión 19.0.20261005 y el mismo comando lo actualizó a la 19.0.20261105 (figuras 38 y 39). El paquete crea el servicio `odoo.service`, activado al arrancar, y la comprobación debe mostrar el estado *active (running)* (figura 40).

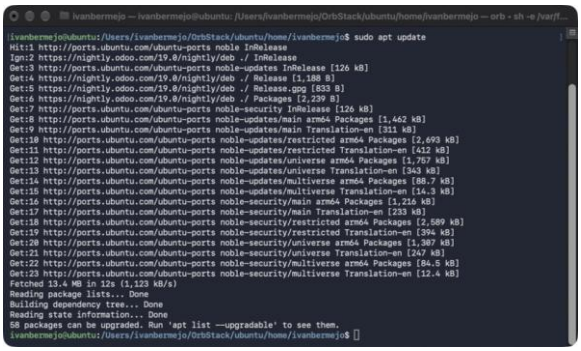


Figura 38. `apt update` leyendo el repositorio `nightly.odoo.com/19.0` (captura 2 del documento de Berme).

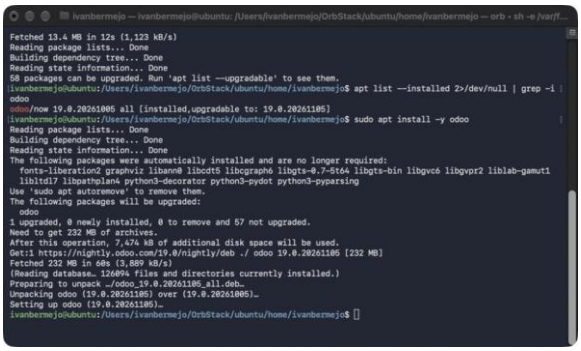


Figura 39. Instalación y actualización del paquete `odoo` (captura 3 del documento de Berme).

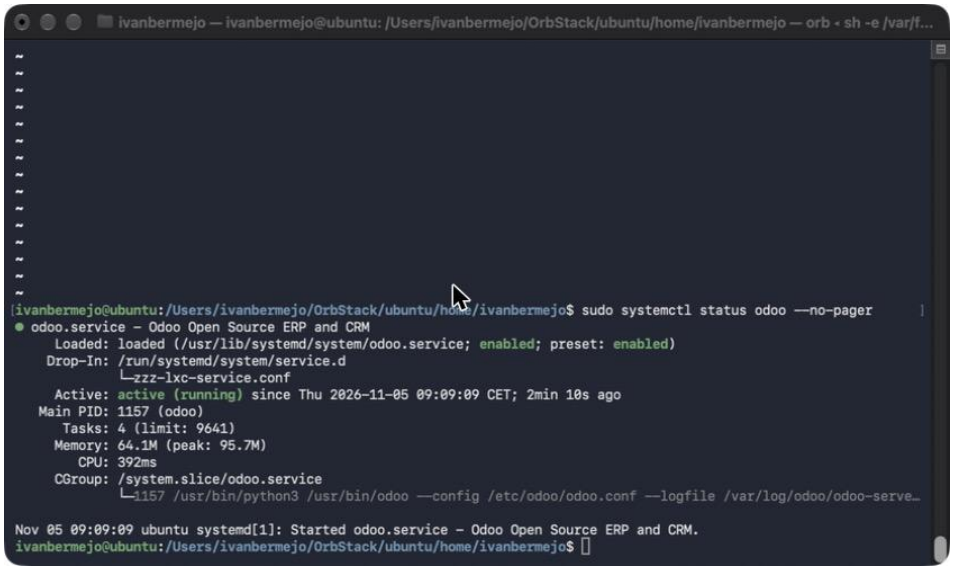


Figura 40. Servicio `odoo` activo y habilitado al arranque (captura 4 del documento de Berme).

Tabla 18. Rutas y valores del servicio de Odoo.

Elemento	Ruta o valor
Servicio	`/usr/lib/systemd/system/odoo.service`
Configuración	`/etc/odoo/odoo.conf`
Registro (log)	`/var/log/odoo/odoo-server.log`
Puerto	8069
Usuario de PostgreSQL	odoo (lo crea el paquete)

Base de datos, empresa y aplicaciones

En el primer acceso se creó la base de datos de Odoo, `nova_sneaker`, en español, con España como país y **sin datos de demostración** (figura 41), y se configuró la empresa «Nova Sneaker» con su logotipo (figura 43), el euro como moneda y el IVA del 21 %. Esta base convive en el mismo servidor PostgreSQL con `nova_sneakers`, la base académica del proyecto, pero son independientes: Odoo no lee las tablas del proyecto ni el proyecto las de Odoo. Los dos nombres solo se diferencian en una letra, pero corresponden a bases distintas. Las aplicaciones se instalaron desde el menú Aplicaciones, pulsando «Activar» en cada una (figura 42); la tabla 19 indica para qué se usa cada una.

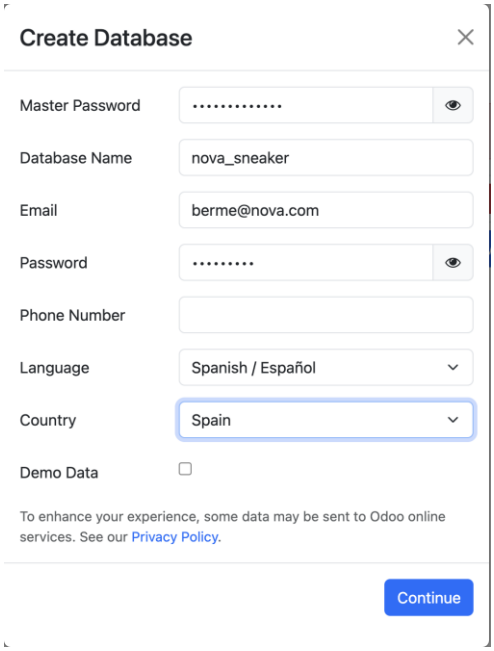


Figura 41. Creación de la base `nova_sneaker` en español, España y sin datos de demostración (captura 11 del documento de Berme).

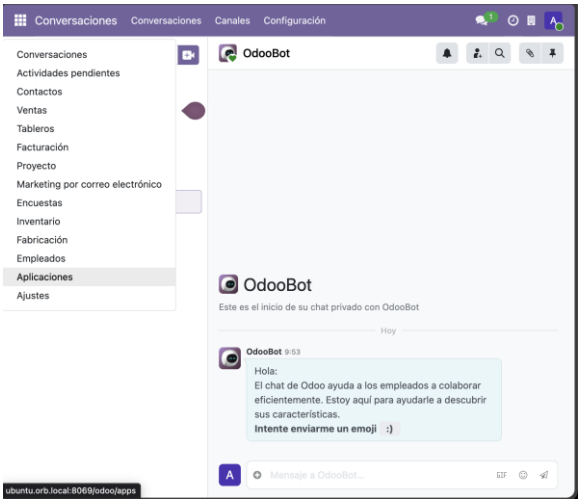


Figura 42. Menú con las aplicaciones instaladas (captura 5 del documento de Berme).

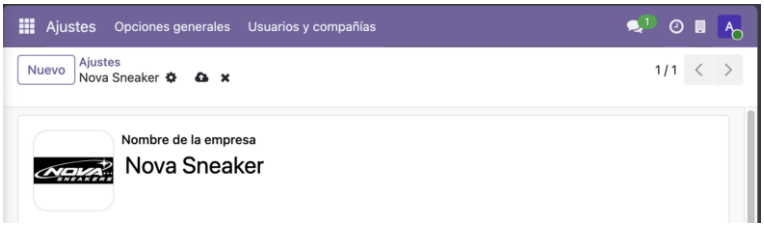


Figura 43. Empresa «Nova Sneaker» con su logotipo, en Ajustes → Compañías (captura 12 del documento de Berme).

Tabla 19. Aplicaciones de Odoo utilizadas.

Aplicación	Para qué se usa
Contactos	Clientes de la empresa, que se buscan por su correo.
Ventas	Presupuestos y pedidos de venta.
Inventario	Productos, existencias y movimientos.
Fabricación	Listas de materiales y órdenes de fabricación.
Facturación	Facturas de los pedidos y registro de pagos (OD-06).

Producto y lista de materiales (OD-04)

El producto terminado es la **Runner X**, creado en *Inventario → Productos* como un bien que se puede vender y comprar, con el seguimiento de inventario activado. Se le asignó un precio de venta de 89 €, un coste de 35 €, la política de facturación «Cantidades pedidas» y un stock inicial de 50 unidades (figura 44). Más tarde, el precio se cambió a 99,90 € para igualarlo con el de PostgreSQL.

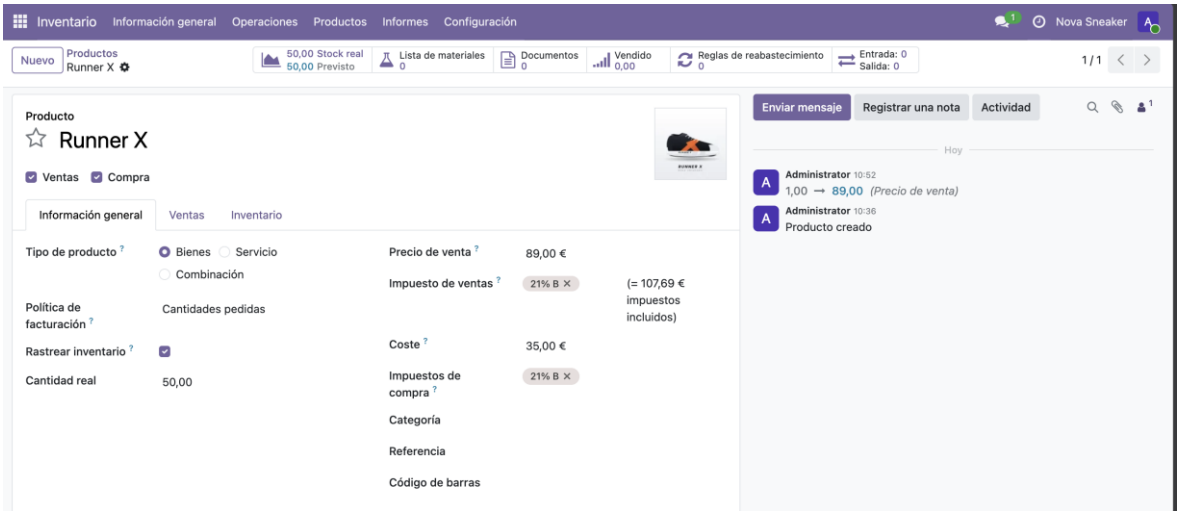


Figura 44. Ficha de la Runner X en Inventario: bien con seguimiento de inventario, precio, coste y existencias (captura 6 del documento de Berme).

La lista de materiales, creada en *Fabricación → Productos → Listas de materiales*, indica qué consume un par de Runner X: **una suela, una tela y unos cordones**. Los tres componentes se crearon también como productos con inventario; la suela, por ejemplo, con 100 unidades y un coste de 15 € (figura 45). Durante las pruebas llegaron a existir cinco listas de materiales para la Runner X; Berme archivó cuatro y dejó solo «Runner X (nuevo) 4», para que todas las órdenes de fabricación usaran la misma (figuras 46 y 47).

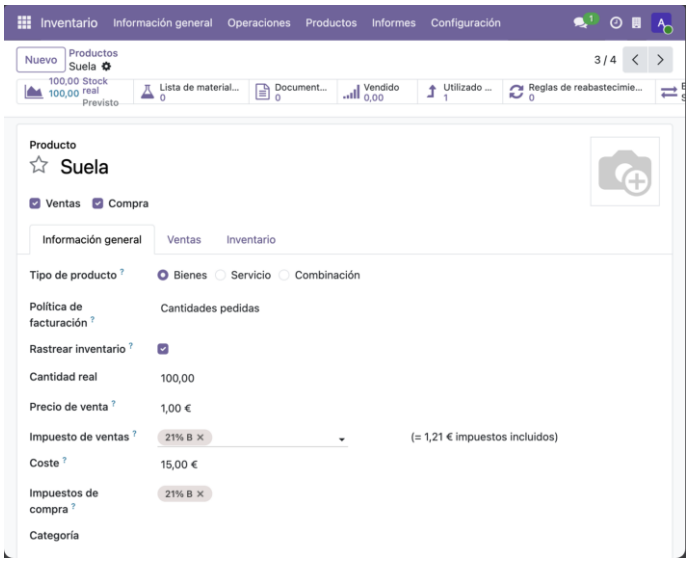


Figura 45. Componente «Suela»: producto con inventario, 100 unidades y coste de 15 € (captura 13 del documento de Berme).

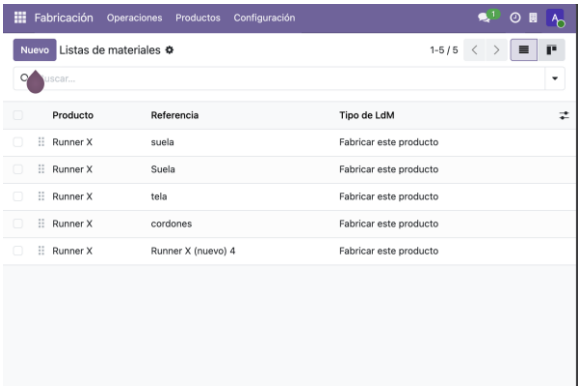


Figura 46. Antes: cinco listas de materiales de la Runner X (captura 16 del documento de Berme).

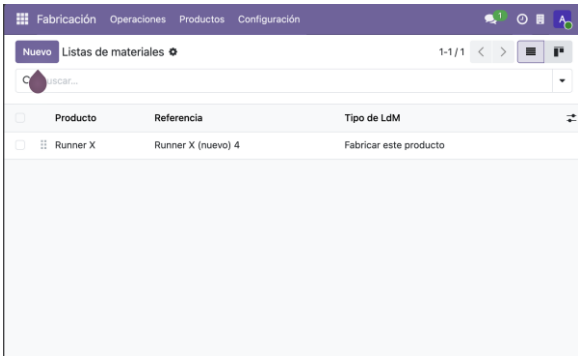


Figura 47. Después: una sola lista, «Runner X (nuevo) 4» (captura 15 del documento de Berme).

Variantes, referencias y usuarios (OD-02)

Para vender por tallas se activaron las variantes en *Inventario → Configuración → Ajustes* y, en la ficha de la Runner X, se creó el atributo **Talla** con los valores 38 a 46, todos en una misma línea. El documento señala la dificultad encontrada: con una línea por talla Odoo no genera variantes, mientras que con una sola línea de nueve valores crea nueve variantes, una por talla (figura 48).

A cada variante se le puso como referencia interna el SKU del proyecto, de `NOVA-4-38` a `NOVA-4-46` (figura 49), porque esa referencia era la clave con la que el servidor buscaría el producto. Antes se archivaron unos productos sueltos de prueba con las referencias `NOVA-4-40` a `NOVA-4-43`, para que no hubiera referencias repetidas. El stock original de 51 unidades quedó en la variante de la talla 40 y las demás quedaron a cero.

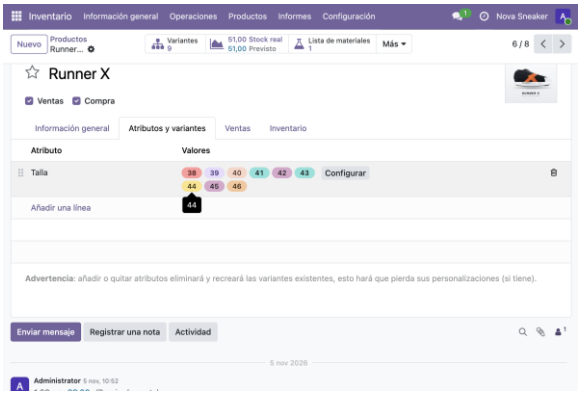


Figura 48. Runner X con el atributo Talla en una sola línea de nueve valores (captura 26 del documento de Berme).

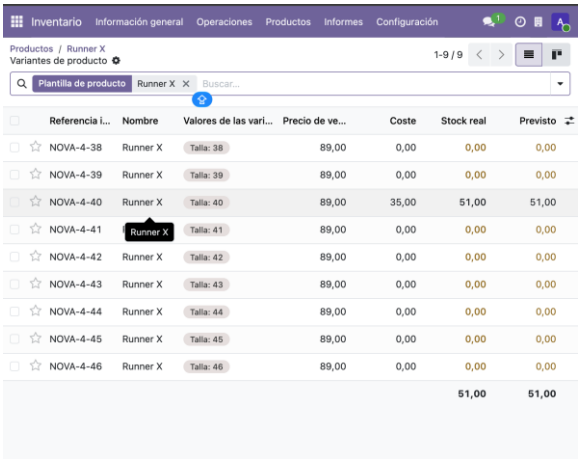


Figura 49. Variantes de la Runner X con su referencia interna `NOVA-4-38` a `NOVA-4-46` (captura 27 del documento de Berme).

Además del administrador, se crearon dos usuarios internos con los permisos mínimos para su función (figura 50): «Vendedor NOVA», con permiso de usuario en Ventas, y «Almacén NOVA», con permiso de usuario en Inventario y en Fabricación. Separar los perfiles evita que quien vende pueda, por ejemplo, validar entregas o modificar existencias.

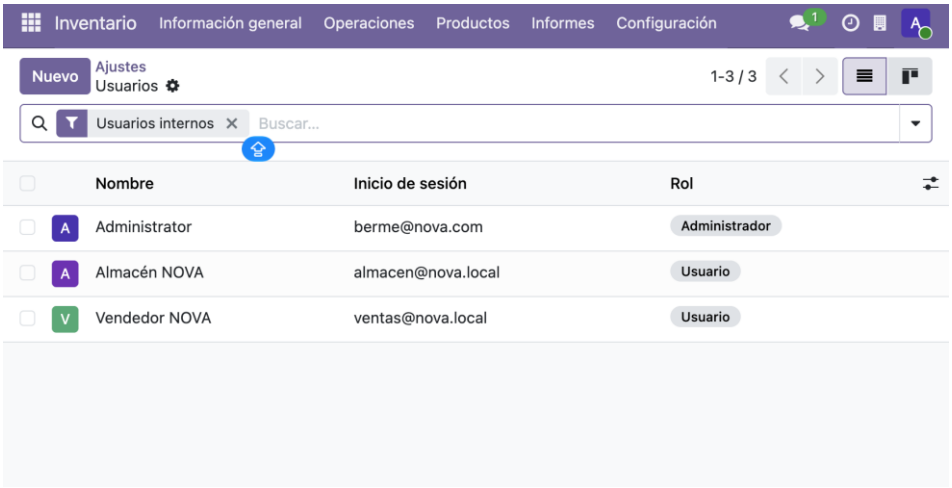


Figura 50. Usuarios internos de Odoo: administrador, Almacén NOVA y Vendedor NOVA (captura 28 del documento de Berme).

3.3.3 Requisitos del proyecto

La matriz de rúbricas del repositorio (apartado 2.5) valora los requisitos de Odoo de forma distinta a los de las otras asignaturas: como la instancia no es accesible desde el entorno de pruebas, los que el PDF documenta figuran como **evidencia documental**, y no como cumplidos y probados. De sus diecisiete filas, doce tienen evidencia documental, tres son parciales y dos están pendientes. La tabla 20 resume los más relevantes.

Tabla 20. Requisitos de Sistemas de Gestión Empresarial y su estado.

Requisito	Evidencia	Estado
Máquina virtual con Ubuntu y Odoo instalado, configurado y operativo	Págs. 3-6: OrbStack, Ubuntu 24.04, Odoo 19 como servicio	Documental
Odoo como sistema central: la venta encadena el resto del flujo	Págs. 10-12: S00005 → fabricación → entrega → factura	Documental
Clientes y contactos	Club Atletico NOVA como cliente de S00005	Documental
Presupuesto y pedido de venta con líneas, cantidades y variantes	S00005 con una línea de 40 pares; variantes en págs. 16-17	Documental
Fabricación: productos, listas de materiales y órdenes	Págs. 7-11: lista de materiales, WH/MO/00001 y WH/MO/00002	Documental
Inventario, movimientos y entrega	WH/OUT/00005 e historial de movimientos	Documental
Facturación o simulación documentada	INV/2026/00001 pagada	Documental
Trazabilidad del mismo pedido de principio a fin	Dentro de Odoo, sí; entre Odoo y los demás módulos, no	Documental
Compañía y perfiles de usuario	Empresa y dos usuarios internos; no todos los perfiles de la rúbrica	Parcial
Verificar el entorno y documentar incidencias y recuperación	Entorno documentado; sin registro de copia de seguridad restaurada	Parcial
Oportunidades de CRM antes de la venta	Sin evidencia (depende del alcance que fije el profesor)	Pendiente
Exportar datos e informes desde Odoo	No hay exportación de S00005	Pendiente
Conexión del servidor con Odoo (mejora no exigida)	Pedidos S00003 y S00004 creados desde el servidor de Berme, sin duplicados y con reintento	Funcionó en el entorno de Berme; no activado en `main`

3.3.4 Resultados del proyecto

Orden de fabricación de prueba WH/MO/00001

Antes del pedido completo, Berme comprobó la lista de materiales con una orden de una sola unidad. Al crearla en *Fabricación → Órdenes de fabricación*, Odoo rellenó la lista de materiales y sus componentes, que aparecían como **No disponible** porque todavía no había existencias (figura 51). Tras dar entrada a suelas, telas y cordones desde Inventario, los componentes pasaron a **Disponible** (figura 52). Después se confirmó la orden y se pulsó *Producir*: WH/MO/00001 quedó en estado **Hecho**, consumió una unidad de cada componente y la Runner X pasó de 50 a 51 unidades (figuras 53 y 54).

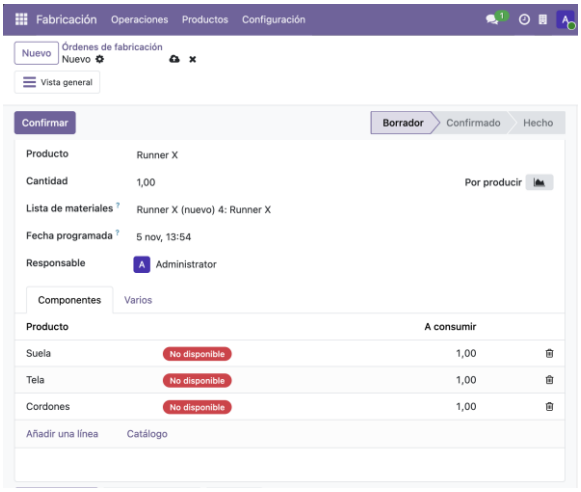


Figura 51. Orden nueva en borrador, sin componentes en stock (captura 7 del documento de Berme).

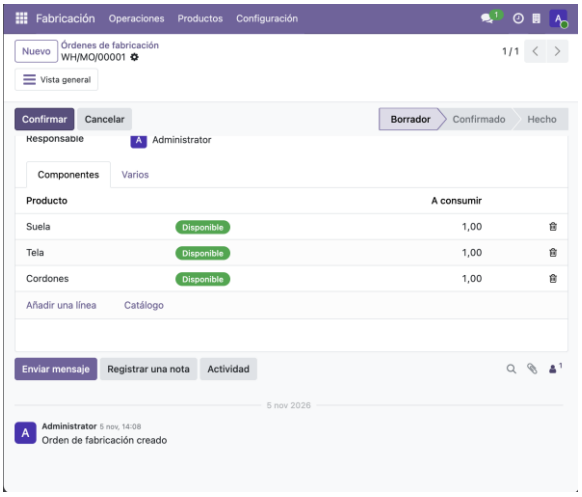


Figura 52. WH/MO/00001 con los tres componentes disponibles (captura 8 del documento de Berme).

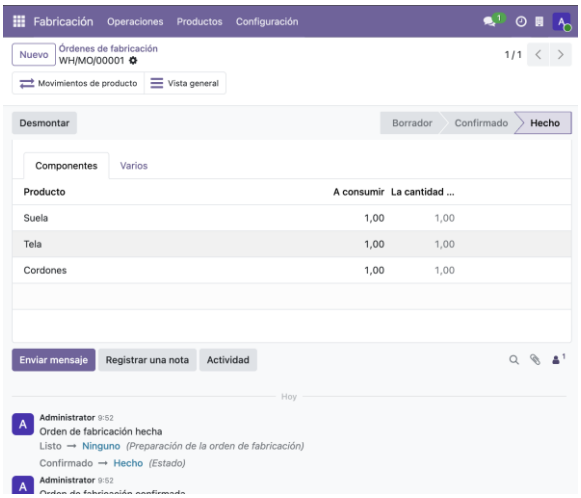


Figura 53. WH/MO/00001 en estado Hecho: consumida una unidad de cada componente (captura 9 del documento de Berme).

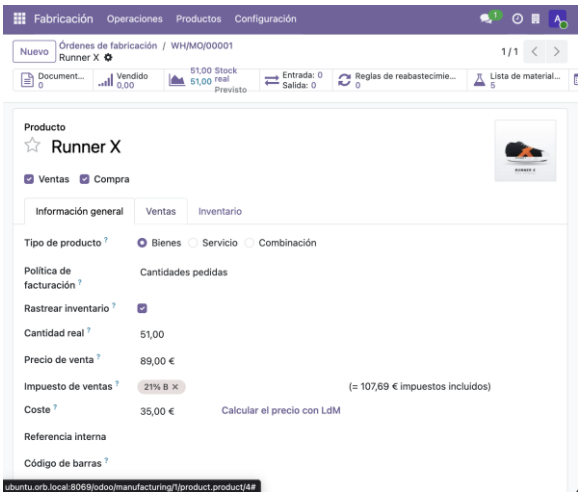


Figura 54. La Runner X pasa de 50 a 51 unidades (captura 10 del documento de Berme).

Esta prueba demuestra tres cosas: que la lista de materiales funciona, porque al terminar la orden se consumen exactamente los componentes previstos; que el inventario se actualiza solo, porque el producto terminado sube una unidad y los movimientos quedan registrados, y que el historial de la orden (*chatter*) guarda quién la confirmó y la terminó y cuándo.

Recorrido completo del pedido S00005

El caso que el equipo presenta para Odoo es la venta **S00005**, un pedido de 40 pares de Runner X para el cliente **Club Atletico NOVA**, que Berme recorrió entero dentro de Odoo el 13 de noviembre de 2026: venta, fabricación, entrega y factura. La tabla 21 resume los cuatro documentos que genera y que quedan enlazados entre sí por el origen S00005.

Tabla 21. Documentos del recorrido S00005 en Odoo.

Paso	Documento	Resultado
1. Venta (OD-03)	S00005	40 Runner X a 89,00 €: 3.560,00 € sin IVA, 747,60 € de IVA y 4.307,60 € en total.
2. Fabricación (OD-04)	WH/MO/00002	40 de 40 producidas; consume 40 suelas, 40 telas y 40 cordones.
3. Entrega (OD-05)	WH/OUT/00005	40 Runner X entregadas al cliente, con origen S00005.
4. Factura (OD-06)	INV/2026/00001	3.560,00 € más IVA, en estado Pagado.

Venta. En *Ventas → Pedidos → Presupuestos → Nuevo* se indicó el cliente, una línea con la Runner X y la cantidad 40 (figura 55). Al pulsar *Confirmar*, el presupuesto se convirtió en el pedido S00005 y apareció el botón *Entrega*, con la entrega ya creada por Odoo (figura 56). El pedido se hizo cuando la Runner X todavía costaba 89 €, por eso su importe es 3.560 € y no el que resultaría con el precio posterior de 99,90 €.

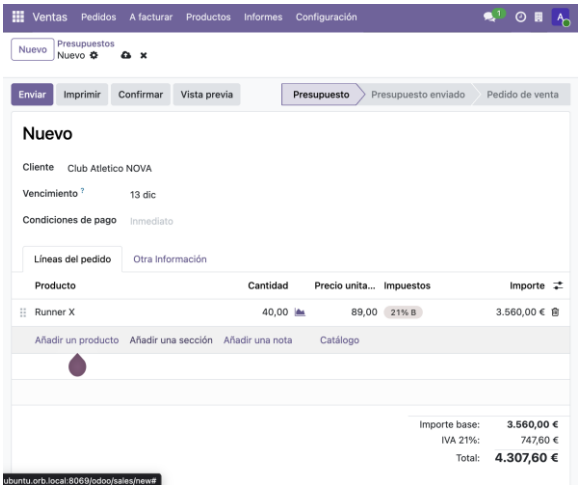


Figura 55. Presupuesto con 40 Runner X para el Club Atletico NOVA (captura 17 del documento de Berme).

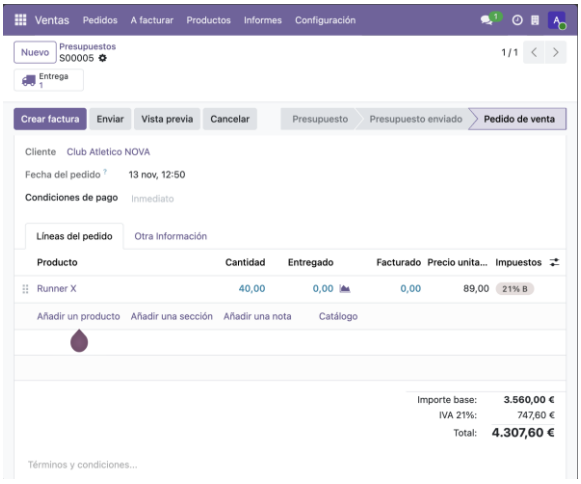


Figura 56. Pedido S00005 confirmado, con el botón de entrega (captura 18 del documento de Berme).

Fabricación. Se creó una orden de 40 unidades de Runner X. Odoo cargó la lista de materiales «Runner X (nuevo) 4» y reservó los componentes, que aparecían en verde porque había 99 unidades de cada uno (figura 57). Tras *Confirmar* (figura 58) y *Producir todo*, la orden WH/MO/00002 terminó con 40 de 40 unidades producidas (figura 59). El documento indica expresamente que la orden se creó a mano, con el pedido como origen: la venta no genera sola la fabricación, y automatizarlo queda como mejora.

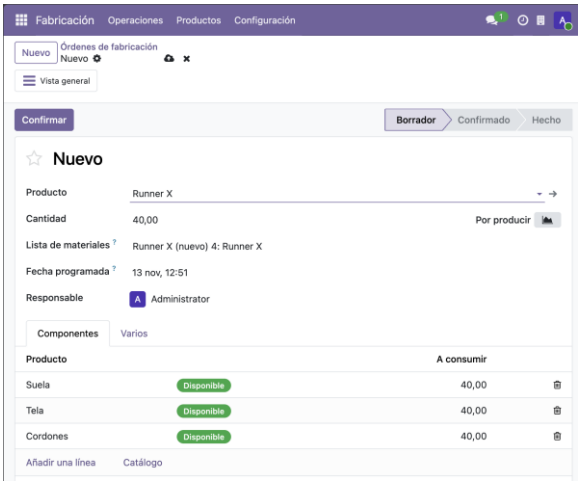


Figura 57. Orden de 40 unidades en borrador, con los componentes disponibles (captura 19 del documento de Berme).

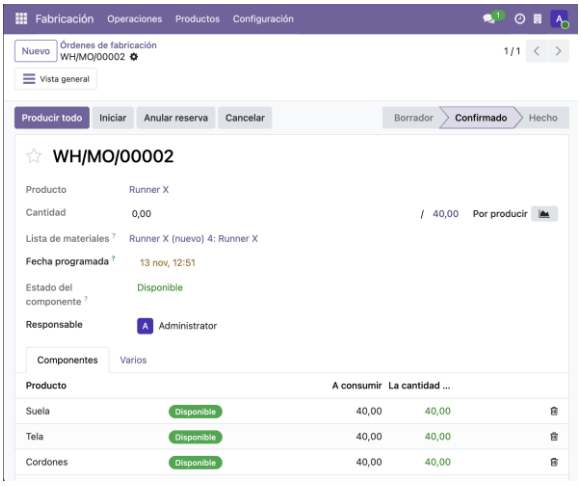


Figura 58. WH/MO/00002 confirmada (captura 20 del documento de Berme).

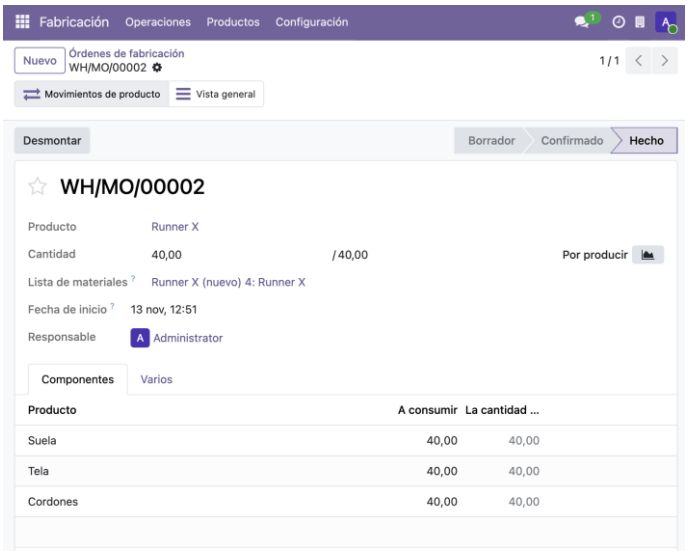


Figura 59. WH/MO/00002 en estado Hecho: 40/40 unidades (captura 21 del documento de Berme).

Entrega. Desde el pedido, el botón *Entrega* abre el albarán WH/OUT/00005, en estado *Listo* (figura 60). Al pulsar *Validar* pasó a *Hecho* y salieron del stock las 40 Runner X (figura 61).

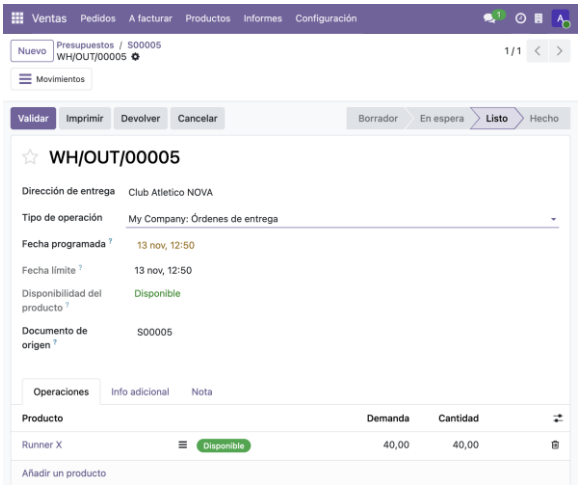


Figura 60. WH/OUT/00005 en estado Listo, preparada para validar (captura 22 del documento de Berme).

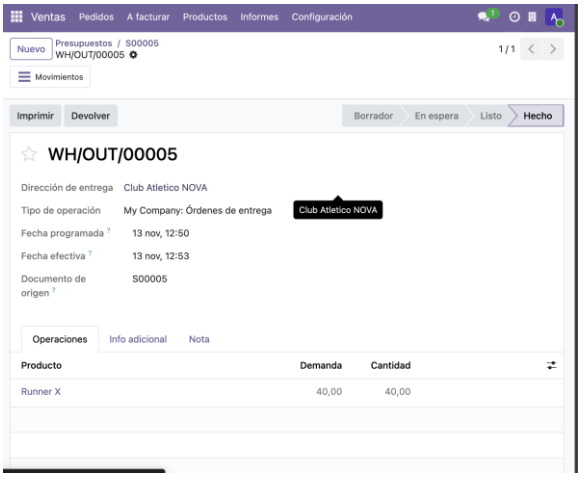


Figura 61. WH/OUT/00005 en estado Hecho (13 nov, 12:53) (captura 23 del documento de Berme).

Factura. Desde el pedido se siguió la secuencia *Crear factura → Confirmar → Registrar pago*. La factura INV/2026/00001 del Club Atletico NOVA quedó registrada y marcada como pagada, enlazada al pedido S00005 y a su pago (figura 62). Se trata de una facturación dentro del entorno de pruebas de Odoo, sin ningún cobro real.

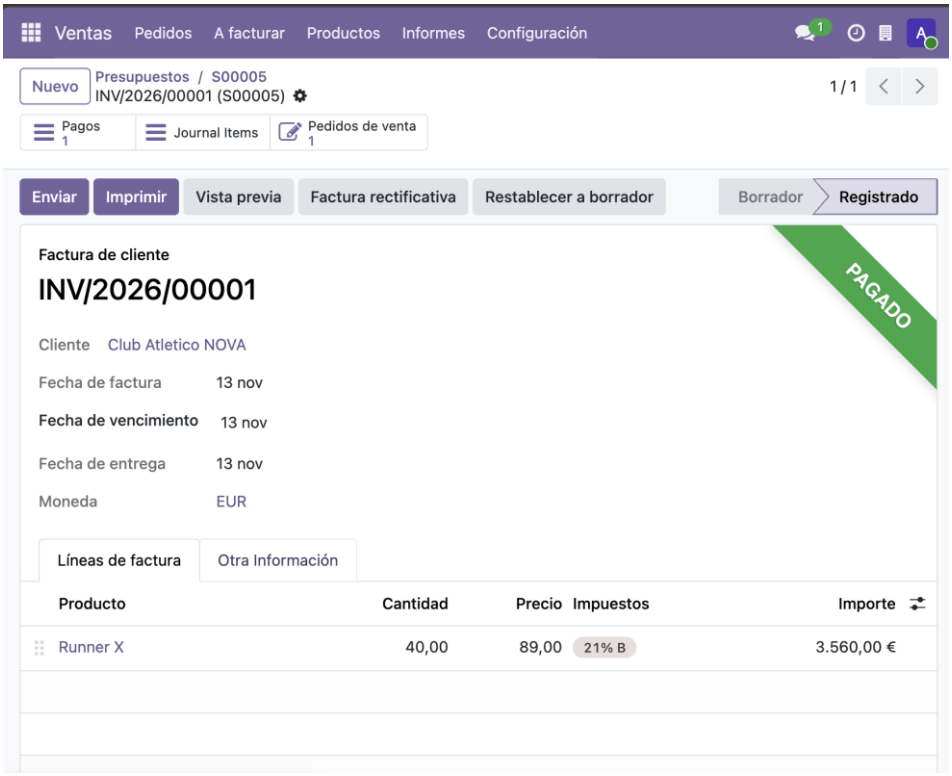


Figura 62. Factura INV/2026/00001 del Club Atletico NOVA, en estado Pagado (captura 24 del documento de Berme).

Trazabilidad del stock. El historial de movimientos de la Runner X recoge todo el recorrido (figura 63): la entrada inicial de 50 unidades, la fabricación de prueba (+1), la fabricación del pedido (+40) y su entrega (−40), con un saldo final de 51 unidades. Es la prueba de que cada documento movió el inventario en el sentido correcto y una sola vez.

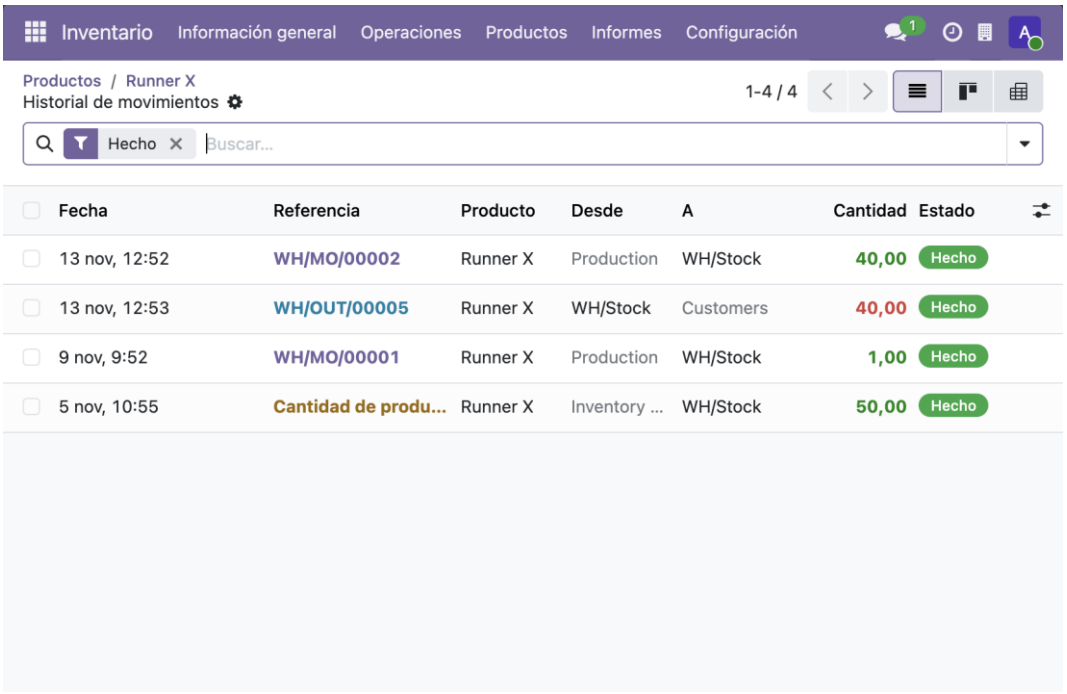


Figura 63. Historial de movimientos de la Runner X: +50, +1 (WH/MO/00001), +40 (WH/MO/00002) y −40 (WH/OUT/00005), con saldo de 51 (captura 25 del documento de Berme).

Conexión del servidor con Odoo

Los apartados 6 y 7 del documento describen una mejora que el enunciado no exige y que funcionó en el entorno de Berme: un conector, escrito por él en `servidor/odoo.js` y `servidor/pedidos.js` (hoy en `servidor-admin/`), que envía a Odoo como pedido de venta cada pedido creado en PostgreSQL. Usa la API JSON-2 de Odoo 19 con `fetch`, sin librerías, y se autentica con una clave de API creada en las preferencias del usuario (figura 64). La clave solo se muestra una vez y se guarda en el archivo `.env` del servidor, nunca en el repositorio.

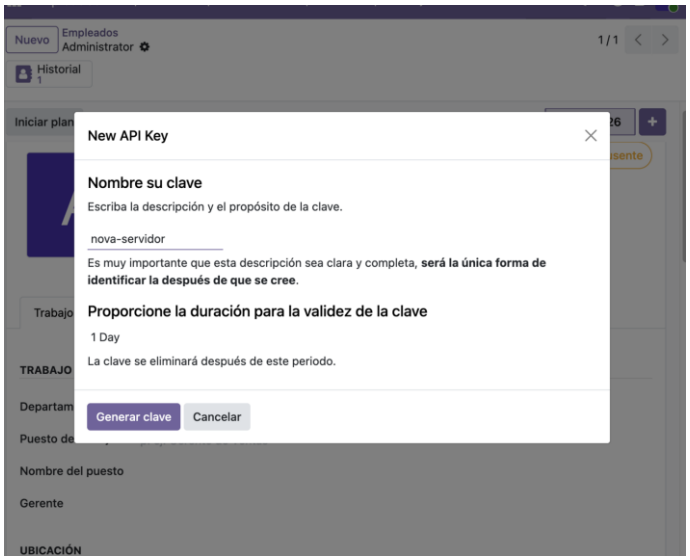


Figura 64. Creación de la clave de API «nova-servidor» en Odoo (captura 14 del documento de Berme).

```
POST {ODOO_URL}/json/2/<modelo>/<método>
Authorization: bearer <clave de API>
X-Odoo-Database: nova_sneaker
Content-Type: application/json

{ "domain": [["default_code", "=", "NOVA-4-42"]], "fields": ["id", "name"] }
```

Listado 8. Formato de las llamadas a la API JSON-2 de Odoo (pág. 14 del documento de Berme).

Al recibir un pedido, el conector sigue seis pasos: busca si ya existe una venta con la misma referencia de cliente (`client_order_ref`) para no duplicarla; busca el cliente por su correo o lo crea; localiza cada producto por su referencia interna (`default_code`); crea la venta con sus líneas y la personalización en la nota; la confirma, y lee el importe sin IVA para compararlo con el total de PostgreSQL. Si Odoo falla, el pedido queda guardado y confirmado en PostgreSQL, se registra como pendiente en la auditoría y se puede reenviar más tarde: Odoo nunca bloquea una venta.

En el entorno de Berme, la conexión creó en Odoo los pedidos de la tabla 22. Odoo numera sus ventas por su cuenta y guarda la referencia del proyecto en el campo «Referencia del cliente», de modo que cada pedido se encuentra en los dos sistemas con la misma referencia. Los pedidos de la web S00003 y S00004 se crearon y confirmaron desde el servidor con importes iguales a los de PostgreSQL; una segunda petición con la misma referencia no duplicó la venta, y el pedido NOVA-0012, que quedó pendiente cuando Odoo rechazó la clave (error 401), se creó como S00003 al reintentarlo.

Tabla 22. Pedidos creados en Odoo desde la integración, en el equipo de Berme (pág. 13 de su documento).

Pedido	Referencia del cliente	Origen	Importe sin IVA
S00001	NOVA-0011	Script de Python con un pedido de prueba de la base de Omar	4.796,00 €
S00002	NOVA-PRUEBA-1	Prueba del conector, repetida sin duplicarse	—
S00003	NOVA-0012	Web → servidor, tras un fallo y un reintento	119,90 €
S00004	NOVA-0013	Web → servidor, dos líneas y personalización	599,50 €

El documento recoge también un problema práctico que hubo que resolver: había que usar `127.0.0.1` en lugar de `localhost` en `ODOO_URL`, porque Node resolvía `localhost` a IPv6 y Odoo escuchaba en IPv4, y la conexión fallaba con el error «fetch failed».

Limitaciones de la integración

La evidencia de Odoo es sólida dentro del propio ERP, pero tiene límites que deben exponerse con claridad, porque afectan a la relación entre esta asignatura y las otras dos:

- **Dependencia del entorno de Berme.** Odoo y la prueba de conexión con el servidor se ejecutaron en el ordenador de Berme; desde los demás equipos la instancia no es accesible. Para la demostración en directo hace falta su máquina, y la copia de seguridad de la base `nova_sneaker` no se ha registrado en el repositorio.
- **Conector no activado en el servidor principal.** En el commit 372f811, el servidor carga `odoo.js`, pero no monta las rutas de `servidor-admin/pedidos.js`, porque escriben directamente en las tablas de pedidos y de auditoría sin pasar por los roles y funciones que usa el resto del servidor; además, `odoo.js` necesita `ODOO_URL`, `ODOO_DB` y `ODOO_API_KEY` en el `.env`. Para activar en esa versión el envío de pedidos que funcionó en el entorno de Berme, habría que adaptar esas rutas a los roles actuales.

- **Colisión de referencias.** En Odoo, las variantes de la Runner X usan las referencias `NOVA-4-38` a `NOVA-4-46`, pero en los datos de PostgreSQL del proyecto `NOVA-4-40` a `NOVA-4-43` corresponden al modelo NOVA Velocity, porque el SKU se forma con el identificador de la variante. Como el conector busca por igualdad exacta de la referencia, un pedido de NOVA Velocity podría entrar en Odoo como Runner X. No hay una equivalencia de SKU validada entre los dos sistemas, y se decidió no cambiar códigos para forzarla.
- **S00005 no ha pasado por los demás módulos.** No existe una exportación de S00005 desde Odoo, ni en PostgreSQL, ni en PSP, ni en Python. El pedido CUSTOM-40 que procesa PSP tiene el mismo nombre de cliente y también suma 40 pares, pero es un pedido de los datos de prueba de PostgreSQL (NOVA Velocity, tallas 40 a 43), no la misma venta.
- **Procesos que siguen siendo manuales.** La orden de fabricación se crea a mano desde la venta, y el stock de Odoo y el de PostgreSQL no se sincronizan: cada sistema lleva el suyo. Solo la Runner X tiene variantes; el resto del catálogo no se ha cargado en Odoo.
- **Pendientes de la rúbrica.** No se acredita el CRM previo a la venta, una exportación de datos desde Odoo ni una copia de seguridad restaurada, y no están todos los perfiles de usuario que menciona la rúbrica.

Por último, el ejemplo SO024 del enunciado (Club Halcones, Runner X, 40 pares de la 38 a la 46) no se ha creado en Odoo. El equipo decidió presentar S00005 como caso documentado sin crear ni renombrar ninguna venta, y tratar SO024 como lo que es en el enunciado: un ejemplo de pedido Custom.

CAPÍTULO 4. IMPLEMENTACIÓN E INTEGRACIÓN DEL PROYECTO

Las tres asignaturas se desarrollaron en paralelo, cada una con su tecnología, pero sobre una misma empresa y unos mismos datos. Este capítulo explica qué información pasa de una parte a otra en la versión entregada, cómo se reconoce un mismo pedido en los distintos sistemas y en qué orden se demuestra.

4.1 Qué viaja entre las partes

Tabla 23. Intercambios de datos entre las partes del proyecto y su estado.

De → a	Qué viaja	Cómo	Estado
Web de clientes → servidor → PostgreSQL	Cuentas, carrito, pedidos y pago simulado	API `/api/tienda` y funciones del script 20	Funciona (compra en Edge, 139/139)
Formulario Custom → empresa	Club, contacto, modelo, personalización y pares por talla	Mensaje de simulación en la página	Simulado: no se envía
Panel y portal ↔ PostgreSQL	Consultas, «Marcar en reparto», avance de estados y ajuste de stock	APIs `/api/admin` y `/api/trabajador` con roles mínimos	Funciona (82/82 y 29/29)
PostgreSQL → PSP	La orden de fabricación de un pedido confirmado	CSV de 8 columnas generado por el script 15	Funciona (CUSTOM-40)
Servidor → Odoo	Pedidos de la web como ventas de Odoo	API JSON-2 (`odoo.js`, `pedidos.js`)	Funcionó en el entorno de Berme (S00003, S00004); no activado en `main`
Odoo → PSP	La venta S00005 como CSV de fabricación	—	No existe

La tabla muestra que la parte técnica de la cadena (web, base de datos y PSP) está conectada y probada con datos de prueba, y que el ERP funciona por sí mismo; el envío de pedidos desde el servidor funcionó en el entorno de Berme, aunque no está activado en el servidor principal. El eslabón que falta es el de vuelta: exportar una venta de Odoo con el formato que lee PSP. Con ese archivo, PSP podría procesarla sin cambiar su código, porque solo necesita un CSV con las ocho columnas acordadas.

4.2 Modelo de datos común del pedido

Un pedido Custom se describe con los mismos datos en todas las partes del proyecto, aunque cada sistema los guarde a su manera. La tabla 24 relaciona las ocho columnas del CSV de fabricación, que es el formato de intercambio entre PostgreSQL y PSP, con los campos de Odoo que usa el conector de Berme según su documento (pág. 15). El intercambio con PSP se hace en CSV (UTF-8 y coma como separador) y el de Odoo, en JSON a través de su API.

Tabla 24. Datos comunes del pedido en el CSV de fabricación y en Odoo.

Dato	CSV de fabricación (PostgreSQL → PSP)	Odoo (conector de Berme)
Referencia del pedido	`referencia` (por ejemplo, `CUSTOM-40`)	«Referencia del cliente» de la venta (`client_order_ref`)
Cliente	`cliente` (nombre)	Contacto (`res.partner`), que se busca por su correo o se crea
Producto y variante	`sku`, `modelo`, `color` y `talla`	Variante de producto (`product.product`), localizada por su referencia interna (`default_code`)
Cantidad	`cantidad` de cada línea	Cantidad de cada línea del pedido de venta
Personalización	`personalizacion`	Nota del pedido de venta

Dos de estos datos permiten reconocer un mismo pedido en todos los sistemas. El primero es la **referencia del pedido**: en PostgreSQL es el campo de referencia (por ejemplo, `CUSTOM-40` o `WEB-…` para las compras de la tienda); en PSP aparece en el CSV, en cada respuesta `OK;`, en el nombre de la ficha y en el registro; y en Odoo se guarda en la «Referencia del cliente» (`client_order_ref`) de la venta, que es lo que usa el conector para no duplicar pedidos.

El segundo identificador es el **SKU** de cada combinación de modelo, color y talla. Aquí está el principal problema de integración pendiente: el SKU de PostgreSQL se forma como `NOVA-<id_variante>-<talla>` y depende de un identificador generado por la base, mientras que en Odoo las variantes de la Runner X recibieron las referencias `NOVA-4-38` a `NOVA-4-46`, que en los datos de prueba de PostgreSQL corresponden en parte a otro modelo, la NOVA Velocity. Antes de usar la integración con datos reales habría que acordar una tabla de equivalencias o unos códigos estables.

4.3 Orden de la demostración

Para demostrar el funcionamiento del proyecto, el orden más claro sigue el recorrido de un pedido:

1. **Web de clientes** en escritorio y en móvil: catálogo, ficha, probador y solicitud Custom simulada; después, una compra completa en la tienda.
2. **Panel de administración**: el pedido recién comprado aparece en Pedidos y en la Auditoría; se marca en reparto y el cliente lo ve como «En reparto».
3. **PSP**: procesamiento de CUSTOM-40 con código 0, un error controlado y la batería de 27 pruebas.
4. **Odoo**, en el entorno de Berme: el recorrido S00005 (venta, fabricación, entrega y factura) y la conexión con el servidor que se probó allí.

Los pasos 1 a 3 se ejecutan con el servidor y una base de datos montada con los scripts del proyecto, según el manual `06_Intermodular/manuales/MANUAL_INSTALACION.md`. El paso 4 depende de la máquina virtual de Berme.

CAPÍTULO 5. CONCLUSIONES

5.1 Conclusiones del trabajo

El proyecto ha alcanzado los objetivos principales de las tres asignaturas. En Desarrollo de Interfaces se ha construido una web completa, sin dependencias externas, organizada con BEM, adaptable desde 320 px y sin violaciones de accesibilidad detectadas por las herramientas automáticas; las ampliaciones (tienda, panel y portal) demuestran además la conexión de la interfaz con datos reales. En Programación de Servicios y Procesos, la aplicación coordina tres procesos hijos con `ProcessBuilder`, los comunica por la entrada y la salida estándar y controla códigos de salida, errores y tiempos de espera, con una batería de pruebas que lanza los procesos de verdad. En Sistemas de Gestión Empresarial, Odoo está implantado y configurado para la empresa y el pedido S00005 se ha llevado de la venta a la factura, con su fabricación, su entrega y la trazabilidad del stock.

El valor del caso común está en que obliga a mirar el proyecto como un sistema. Al juntar las partes aparecieron problemas que no se ven trabajando por separado: dos ramas que modifican los mismos archivos, una referencia de producto que significa cosas distintas en dos bases de datos o un mismo nombre de cliente en dos pedidos que no son el mismo. Resolverlos, o al menos identificarlos y documentarlos con precisión, ha sido una parte importante del trabajo.

5.2 Reparto del trabajo

La tabla 25 recoge las aportaciones de cada miembro del equipo que se pueden comprobar en el historial de Git (con `git log --follow` y `git blame`) y en los documentos del repositorio. Hay que leerla con dos precauciones: el autor de un commit no siempre es el autor de todo el trabajo que contiene (por ejemplo, las fusiones de las ramas de Álvaro y de Berme las hizo Omar), y el reparto inicial de tareas no coincide en todo con lo que finalmente subió cada uno, como ocurre con las clases PSP-01 y PSP-04.

Tabla 25. Aportaciones verificables de cada miembro del equipo en las tres asignaturas.

	Desarrollo de Interfaces	PSP	Odoo
Omar	Web de clientes (estructura, CSS, catálogo, probador, Custom, contacto y tienda), renombrado a BEM y corrección de contrastes; servidor Node/Express con el login único y la integración del panel y del portal; pruebas y evidencias.	Validador, lector de CSV, primera versión del lanzador, proceso padre, `GestorProcesos`, generador de la ficha y batería `PruebasPsp`.	Documentación del caso S00005 y de los límites de la integración en `05_Odoo` y `06_Intermodular`; integración del conector de Berme en `servidor-admin`, sin activarlo.
Álvaro	Diseño y CSS del panel de administración, sus páginas, sus gráficos con Chart.js y los prototipos; diálogo de acceso de clientes; colores por colección de las tarjetas; incorporación del logotipo de NOVA.	Analizador de tallas (`Psp.AnalizadorTallas`), su lanzador, `ResultadoProceso` y su prueba de ocho comprobaciones.	—
Berme	Portal del trabajador: páginas, JavaScript y rutas del servidor; comprobador de recursos de la web (`scripts/comprobar_web.py`).	Comprobador rápido (`scripts/comprobar_psp.py`) y su corrección, CSV sintético de prueba y descripción de los formatos de fabricación.	Instalación y configuración de Odoo, productos, lista de materiales, variantes, usuarios, recorrido S00005, conector con el servidor y documento `Odoo-NOVA-Sneakers.pdf`.

5.3 Aprendizajes del proyecto

- **La comunicación entre procesos tiene trampas que no se ven en un ejemplo pequeño.** El bloqueo por tuberías llenas solo apareció con un CSV grande, y un tiempo de espera puede no actuar si se lee la salida antes de llamar a `waitFor`. Probar con casos extremos y medir los tiempos fue la forma de descubrirlo.
- **La accesibilidad se diseña, no se añade al final.** Los colores por colección eran atractivos, pero dejaban textos ilegibles; corregirlos obligó a cambiar la estructura de la tarjeta, no solo un color.
- **Una convención de nombres facilita el trabajo en equipo.** BEM hace evidente a qué componente pertenece cada clase, y renombrarlas en un solo commit, comprobando que los estilos calculados no cambiaban, permitió hacerlo sin romper nada.
- **En un ERP, el orden de configuración importa.** Las variantes no se generaban con una línea por talla, había listas de materiales duplicadas y productos con referencias repetidas; el flujo de venta solo fue limpio después de ordenar los datos maestros.
- **Documentar los límites es parte del resultado.** Separar lo que funciona y está probado de lo que depende de un equipo concreto, o de lo que todavía no existe, permite defender el proyecto con seguridad.

5.4 Líneas de mejora

- Realizar la prueba de usabilidad con tres a cinco personas siguiendo el procedimiento ya preparado, y repetir Lighthouse sobre la versión final.
- Exportar desde Odoo la venta de un pedido Custom con el formato del CSV de PSP y procesarla, para demostrar la cadena completa con un mismo pedido.
- Acordar una equivalencia estable de SKU entre Odoo y PostgreSQL y adaptar las rutas del conector a los roles del servidor principal para activarlo.
- Estudiar la ruta de fabricación bajo pedido (MTO) de Odoo, para que la venta confirmada genere sola la orden de fabricación, y sincronizar el stock entre los dos sistemas.
- Completar en Odoo el resto del catálogo como variantes, los perfiles de usuario de la rúbrica y una copia de seguridad restaurada y documentada.
- En PSP, procesar varios pedidos a la vez, que es la ampliación que propone el enunciado.

CAPÍTULO 6. ANEXOS

6.1 Repositorio y versión documentada

El código del proyecto está en el repositorio `omar1676/NOVA` de GitHub (privado). Esta memoria documenta la rama `main` en el commit **372f811**, la fusión de la pull request #3 del 13 de noviembre de 2026. Las carpetas del repositorio se organizan por asignatura: `01_Interfaces`, `02_Acceso_Datos`, `03_PSP`, `04_Python`, `05_Odoo` y `06_Intermodular`, además de `servidor-admin` (servidor Node/Express) y `odoo/documentacion` (documento de Berme).

6.2 Cómo reproducir las demostraciones

PSP

```
& "$env:JAVA_HOME\bin\javac.exe" -encoding UTF-8 -d clases *.java pruebas\*.java src\Psp\*.java
& "$env:JAVA_HOME\bin\java.exe" -cp clases ProcesadorPedidosCustom datos\custom40_fabricacion_psp.csv
& "$env:JAVA_HOME\bin\java.exe" -cp clases ProcesadorPedidosCustom datos\prueba_cantidad_cero.csv
& "$env:JAVA_HOME\bin\java.exe" -cp clases PruebasPsp
& "$env:JAVA_HOME\bin\java.exe" -cp clases Psp.PruebasAnalizador
```

Listado 9. Compilación, ejecución y pruebas de PSP en Windows con el JDK 17 (desde la carpeta `03_PSP`).

Interfaces

La web de clientes se puede abrir directamente desde `01_Interfaces/cliente/index.html`: sin servidor funcionan el catálogo, la ficha con el precio orientativo, el probador y el formulario Custom, y la tienda indica que no está disponible. Para la tienda, el panel y el portal hay que montar la base con los scripts de `02_Acceso_Datos/sql`, instalar las dependencias del servidor con `npm ci` y arrancarlo con `npm start` desde `servidor-admin`, según `06_Intermodular/manuales/MANUAL_INSTALACION.md`. Las direcciones son `http://127.0.0.1:3000/tienda/` para la tienda y `http://127.0.0.1:3000/login.html` para el acceso del personal.

Odoo

Odoo se ejecuta en la máquina virtual del ordenador de Berme (`http://ubuntu.orb.local:8069`, base `nova_sneaker`). Los comandos de servicio y la prueba de la clave de API están en la página 19 de su documento.

6.3 Relación de fuentes

Tabla 26. Fuentes utilizadas para elaborar la memoria.

Fuente	Ubicación y uso
Repositorio del proyecto	`omar1676/NOVA` (GitHub, privado), rama `main`, commit 372f811 (fusión de la PR #3, 13/11/2026).
Documento de Odoo	`odoo/documentacion/Odoo-NOVA-Sneakers.pdf`, de Berme: 19 páginas y 28 capturas (SHA-256 8df824d5…f854). Se conserva sin modificar.
Interfaces: código	`01_Interfaces/cliente/`, `01_Interfaces/admin/`, `01_Interfaces/trabajador/` y `servidor-admin/`.
Interfaces: documentación y pruebas	`01_Interfaces/README.md`, `01_Interfaces/cliente/README.md`, `01_Interfaces/memoria/ESTRATEGIA_PRUEBAS_INTERFACES.md`, `01_Interfaces/memoria/PROCEDIMIENTO_USABILIDAD.md` y `01_Interfaces/evidencias/`.
PSP: código y documentación	`03_PSP/` (clases, `pruebas/`, `src/Psp/`, `datos/`), `03_PSP/README.md`, `03_PSP/memoria/DOCUMENTACION_PSP.md` y `03_PSP/evidencias/`.
Odoo: documentos del equipo	`05_Odoo/README.md`, `06_Intermodular/CASO_COMUN_S00005.md` y `06_Intermodular/SO024/INTEGRACION_ODOO_SO024.md`.
Requisitos y estado	`06_Intermodular/rubricas/MATRIZ_RUBRICAS.md` (apartados 2.1, 2.3 y 2.5), que cita página a página el enunciado intermodular v0.3.
Regresiones	`06_Intermodular/evidencias/regresion-final/` (13/11/2026) y `06_Intermodular/evidencias/cierre-entrega-20261113/`.
Historial de Git	`git log --follow` y `git blame -C -w` sobre 372f811, para la autoría de cada archivo.
Ejecuciones de verificación	PSP (`ProcesadorPedidosCustom`, `PruebasPsp`, `Psp.PruebasAnalizador`) el 13/11/2026 con el JDK 17.0.12 sobre una copia de 372f811: 27/27 y 8/8.

6.4 Relación de figuras y su origen

Todas las figuras son capturas reales del proyecto o del documento de Berme, salvo el esquema de la figura 1, que es de elaboración propia. Algunas capturas se han recortado para que el diálogo o la tabla que interesa sea legible; el recorte no altera su contenido. Las rutas son relativas a la raíz del repositorio.

Tabla 27. Origen de cada figura de la memoria.

Fig.	Contenido	Origen
1	Relación entre las tres asignaturas en la versión entregada (esquema de elaboración propia…	Elaboración propia
2	Portada de la web de clientes en escritorio (1440 px)	`06_Intermodular/evidencias/cierre-entrega-20261113/web-1440.png`
3	Portada en un móvil de 390 px	`06_Intermodular/evidencias/cierre-entrega-20261113/web-390.png`
4	Catálogo en móvil (480 px) con el precio y la disponibilidad de cada modelo	`01_Interfaces/evidencias/tienda/capturas-2026-11-13/movil_01_catalogo_disponibilidad.png`
5	«Mis pedidos» en móvil con un pedido «En reparto»	`01_Interfaces/evidencias/reparto-y-correo/capturas/mis_pedidos_en_reparto_movil.png`
6	Dashboard del panel en móvil	`01_Interfaces/evidencias/panel-admin/capturas/dashboard_movil.png`; recortada
7	Catálogo completo	`06_Intermodular/evidencias/regresion-final/05_web/capturas_interfaces/escritorio_01_catalogo_con_precios.png`
8	Ficha del modelo NOVA Street Vortex	`06_Intermodular/evidencias/regresion-final/05_web/capturas_interfaces/escritorio_02_ficha_street.png`; recortada
9	Probador con el modelo NOVA Casual Marea puesto, en el escenario Madrid	Captura tomada para esta memoria (13/11/2026, Edge) sobre la web de 372f811; recorte del diálogo
10	Figura de complexión media con calzado genérico y la zapatilla elegida al lado, en el…	Captura tomada para esta memoria (13/11/2026, Edge) sobre la web de 372f811; recorte del diálogo
11	Sección Custom	`06_Intermodular/evidencias/regresion-final/05_web/capturas_interfaces/escritorio_04_custom_simulado.png`
12	Validación del formulario Custom	`01_Interfaces/evidencias/web/capturas/escritorio_05_custom_aviso_sin_pares.png`
13	Diálogo de acceso de clientes, diseñado por Álvaro, con el error de credenciales	`01_Interfaces/evidencias/tienda/capturas-2026-11-13/escritorio_05_login_alvaro.png`; recortada

Fig.	Contenido	Origen
14	Registro de una cuenta con el aviso de la longitud mínima de la contraseña	01_Interfaces/evidencias/tienda/capturas-2026-11-13/escritorio_06_registro.png; recortada
15	Carrito con seis modelos de las seis colecciones y su total estimado	01_Interfaces/evidencias/tienda/capturas-2026-11-13/escritorio_16_carrito_modelos_nuevos.png; recortada
16	Paso de pago simulado	01_Interfaces/evidencias/tienda/capturas-2026-11-13/escritorio_08_checkout_pago_simulado.png; recortada
17	«Mis pedidos»	01_Interfaces/evidencias/tienda/capturas-2026-11-13/escritorio_17_mis_pedidos_modelos_nuevos.png; recortada
18	Recibo de un pedido, generado por el servidor tras el pago simulado	01_Interfaces/evidencias/reparto-y-correo/capturas/recibo_ejemplo.png; recortada
19	Login único del personal	01_Interfaces/evidencias/portal-trabajador/capturas/login_comercial_denegado.png; recortada
20	Dashboard del panel de administración con datos de la base de prueba (captura del…	01_Interfaces/evidencias/panel-admin/capturas/dashboard_escritorio.png
21	Página de estadísticas	01_Interfaces/evidencias/panel-admin/capturas/estadisticas_escritorio.png
22	Pedidos tras marcar el pedido WEB-F056… en reparto	01_Interfaces/evidencias/reparto-y-correo/capturas/panel_tras_marcar_en_reparto.png
23	Auditoría del panel con los movimientos generados por compras de la tienda online	01_Interfaces/evidencias/tienda/capturas-2026-11-13/panel_15_auditoria_con_compras_web.png
24	Página «Análisis de datos» con el panel de Streamlit integrado, sobre la exportación de…	06_Intermodular/evidencias/cierre-entrega-20261113/dashboard-integrado-1440.png
25	Resumen del portal del trabajador con sus cuatro indicadores	01_Interfaces/evidencias/portal-trabajador/capturas/portal_trabajador.png; recortada
26	Pedidos del portal después de avanzar NOVA-0002, que pasa a «en_preparacion»	01_Interfaces/evidencias/portal-trabajador/capturas/portal_pedidos_tras_avanzar.png
27	Inventario del portal	01_Interfaces/evidencias/portal-trabajador/capturas/portal_inventarioTrabajador.png
28	Pedidos personalizados y configuraciones X-Nova (esta última tabla, solo de lectura y sin…	01_Interfaces/evidencias/portal-trabajador/capturas/portal_personalizadosTrabajador.png; recortada
29	Informe de Lighthouse 12.8.2 en escritorio	Vista del informe 01_Interfaces/evidencias/web/lighthouse/lighthouse_escritorio.html (11/11/2026), capturada el 13/11/2026; parte superior
30	Informe de Lighthouse 12.8.2 en móvil, con las mismas puntuaciones	Vista del informe 01_Interfaces/evidencias/web/lighthouse/lighthouse_movil.html (11/11/2026), capturada el 13/11/2026; parte superior
31	Ejecución del proceso padre con el pedido CUSTOM-40	Captura de consola tomada para esta memoria (13/11/2026): ejecución real sobre una copia de 372f811 con el JDK 17.0.12
32	Dos errores controlados	Captura de consola tomada para esta memoria (13/11/2026): ejecución real sobre una copia de 372f811 con el JDK 17.0.12
33	El validador lanzado por separado con una tubería	Captura de consola tomada para esta memoria (13/11/2026): ejecución real sobre una copia de 372f811 con el JDK 17.0.12
34	Ejecución de PruebasPsp (primera parte)	Captura de consola tomada para esta memoria (13/11/2026): ejecución real sobre una copia de 372f811 con el JDK 17.0.12
35	Ejecución de PruebasPsp (segunda parte)	Captura de consola tomada para esta memoria (13/11/2026): ejecución real sobre una copia de 372f811 con el JDK 17.0.12
36	Prueba propia del analizador de tallas de Álvaro	Captura de consola tomada para esta memoria (13/11/2026): ejecución real sobre una copia de 372f811 con el JDK 17.0.12
37	Máquina ubuntu en OrbStack, en ejecución	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 3, captura 1 (imagen original extraída sin modificar)
38	apt update leyendo el repositorio nightly.odoo.com/19.0 (captura 2 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 4, captura 2 (imagen original extraída sin modificar)
39	Instalación y actualización del paquete odoo (captura 3 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 4, captura 3 (imagen original extraída sin modificar)
40	Servicio odoo activo y habilitado al arranque (captura 4 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 5, captura 4 (imagen original extraída sin modificar)
41	Creación de la base nova_sneaker en español, España y sin datos de demostración (captura…	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 6, captura 11 (imagen original extraída sin modificar)
42	Menú con las aplicaciones instaladas (captura 5 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 6, captura 5 (imagen original extraída sin modificar)
43	Empresa «Nova Sneaker» con su logotipo, en Ajustes → Compañías (captura 12 del documento de…	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 6, captura 12 (imagen original extraída sin modificar)
44	Ficha de la Runner X en Inventario	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 7, captura 6 (imagen original extraída sin modificar)
45	Componente «Suela»	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 7, captura 13 (imagen original extraída sin modificar)
46	Antes	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 8, captura 16 (imagen original extraída sin modificar)

Fig.	Contenido	Origen
47	Después	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 8, captura 15 (imagen original extraída sin modificar)
48	Runner X con el atributo Talla en una sola línea de nueve valores (captura 26 del documento…	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 16, captura 26 (imagen original extraída sin modificar)
49	Variantes de la Runner X con su referencia interna NOVA-4-38 a NOVA-4-46 (captura 27…	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 16, captura 27 (imagen original extraída sin modificar)
50	Usuarios internos de Odoo	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 17, captura 28 (imagen original extraída sin modificar)
51	Orden nueva en borrador, sin componentes en stock (captura 7 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 8, captura 7 (imagen original extraída sin modificar)
52	WH/MO/00001 con los tres componentes disponibles (captura 8 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 8, captura 8 (imagen original extraída sin modificar)
53	WH/MO/00001 en estado Hecho	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 9, captura 9 (imagen original extraída sin modificar)
54	La Runner X pasa de 50 a 51 unidades (captura 10 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 9, captura 10 (imagen original extraída sin modificar)
55	Presupuesto con 40 Runner X para el Club Atletico NOVA (captura 17 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 10, captura 17 (imagen original extraída sin modificar)
56	Pedido S00005 confirmado, con el botón de entrega (captura 18 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 10, captura 18 (imagen original extraída sin modificar)
57	Orden de 40 unidades en borrador, con los componentes disponibles (captura 19 del documento…	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 11, captura 19 (imagen original extraída sin modificar)
58	WH/MO/00002 confirmada (captura 20 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 11, captura 20 (imagen original extraída sin modificar)
59	WH/MO/00002 en estado Hecho	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 11, captura 21 (imagen original extraída sin modificar)
60	WH/OUT/00005 en estado Listo, preparada para validar (captura 22 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 11, captura 22 (imagen original extraída sin modificar)
61	WH/OUT/00005 en estado Hecho (13 nov, 12:53) (captura 23 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 11, captura 23 (imagen original extraída sin modificar)
62	Factura INV/2026/00001 del Club Atletico NOVA, en estado Pagado (captura 24 del documento…	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 12, captura 24 (imagen original extraída sin modificar)
63	Historial de movimientos de la Runner X	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 17, captura 25 (imagen original extraída sin modificar)
64	Creación de la clave de API «nova-servidor» en Odoo (captura 14 del documento de Berme)	odoo/documentacion/Odoo-NOVA-Sneakers.pdf, pág. 14, captura 14 (imagen original extraída sin modificar)